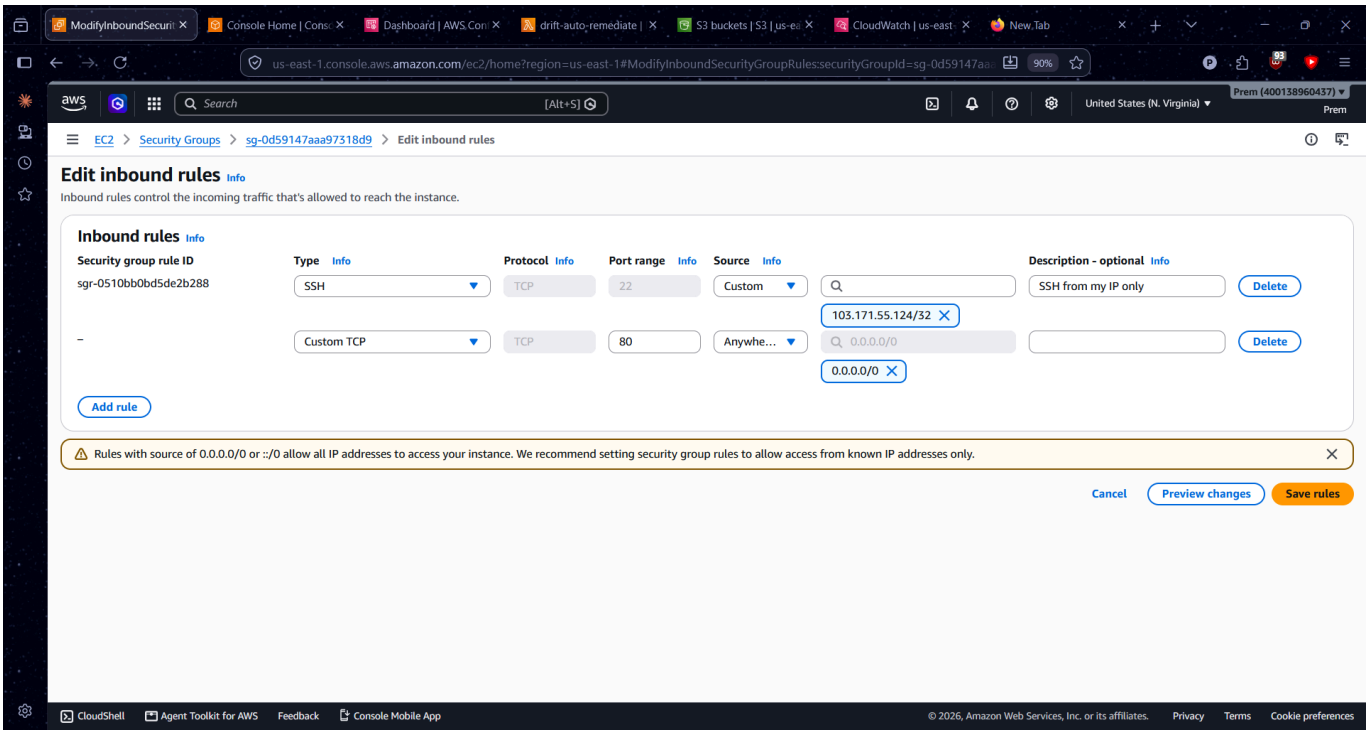
Infrastructure Drift Detection and Automated Remediation using Terraform and AWS Lambda

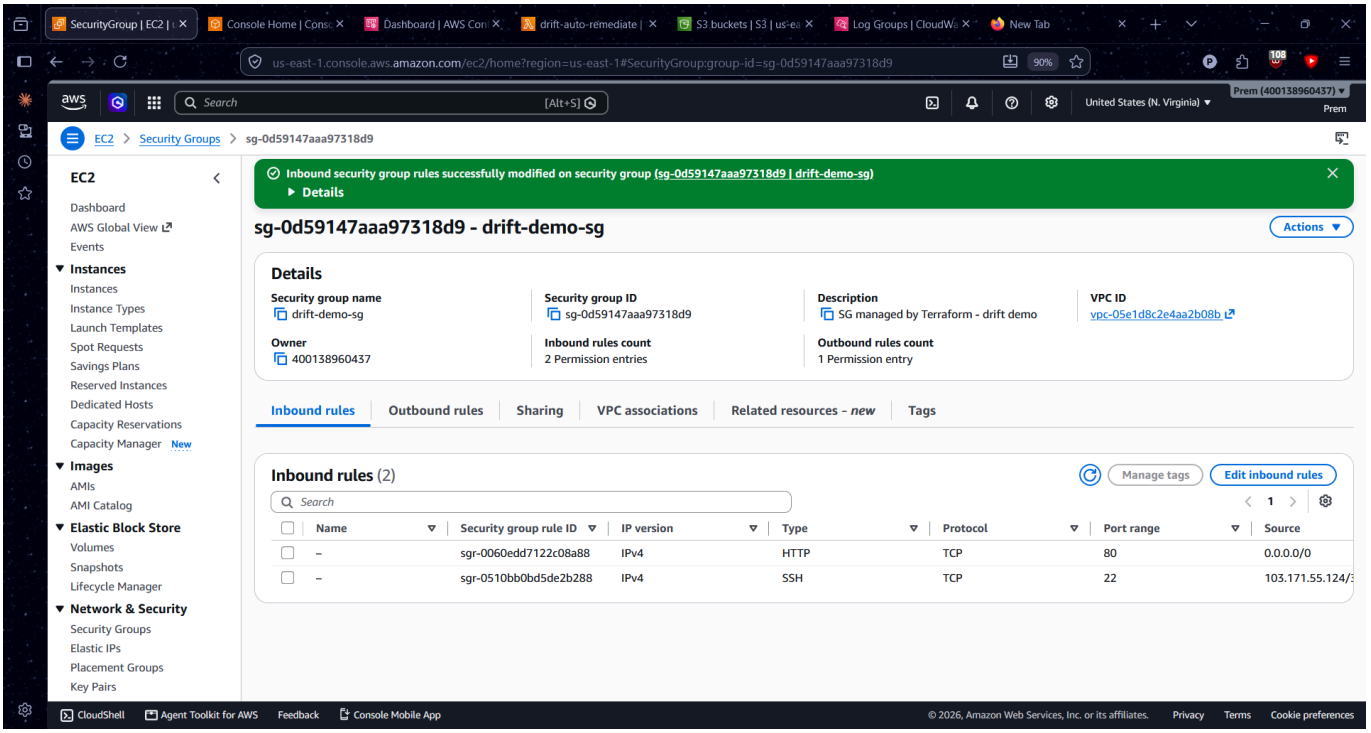
> A production-style Cloud Automation project that solves a problem every growing engineering team eventually faces — someone makes a "quick" console change that bypasses Terraform, and suddenly nobody knows what the infrastructure actually looks like anymore.

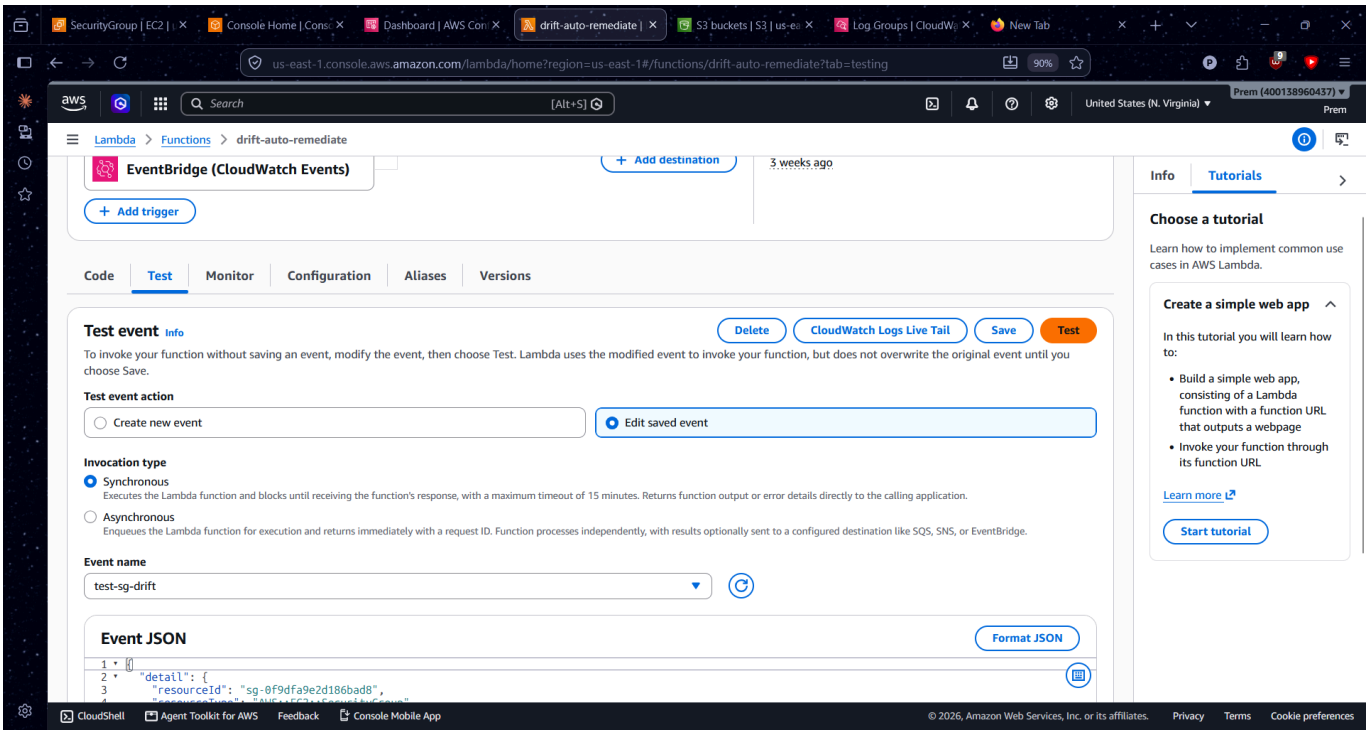


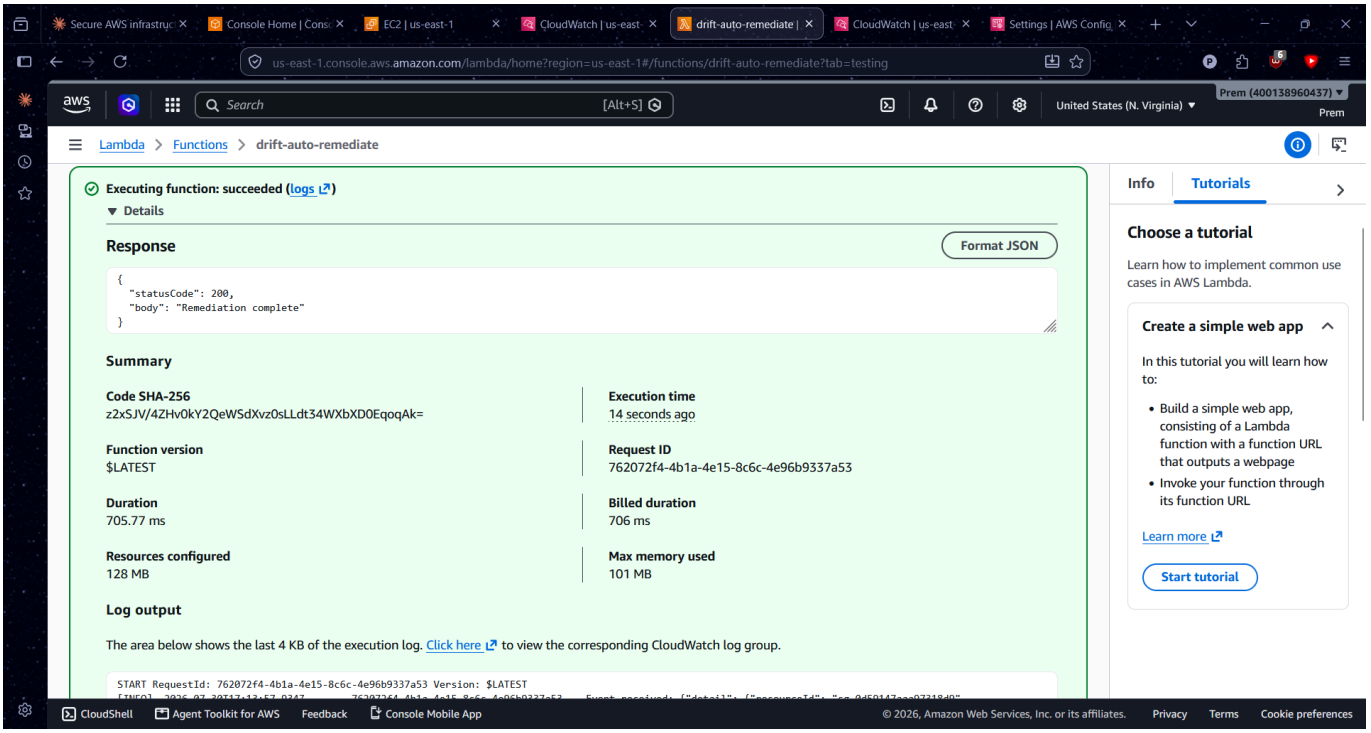
Project Preview

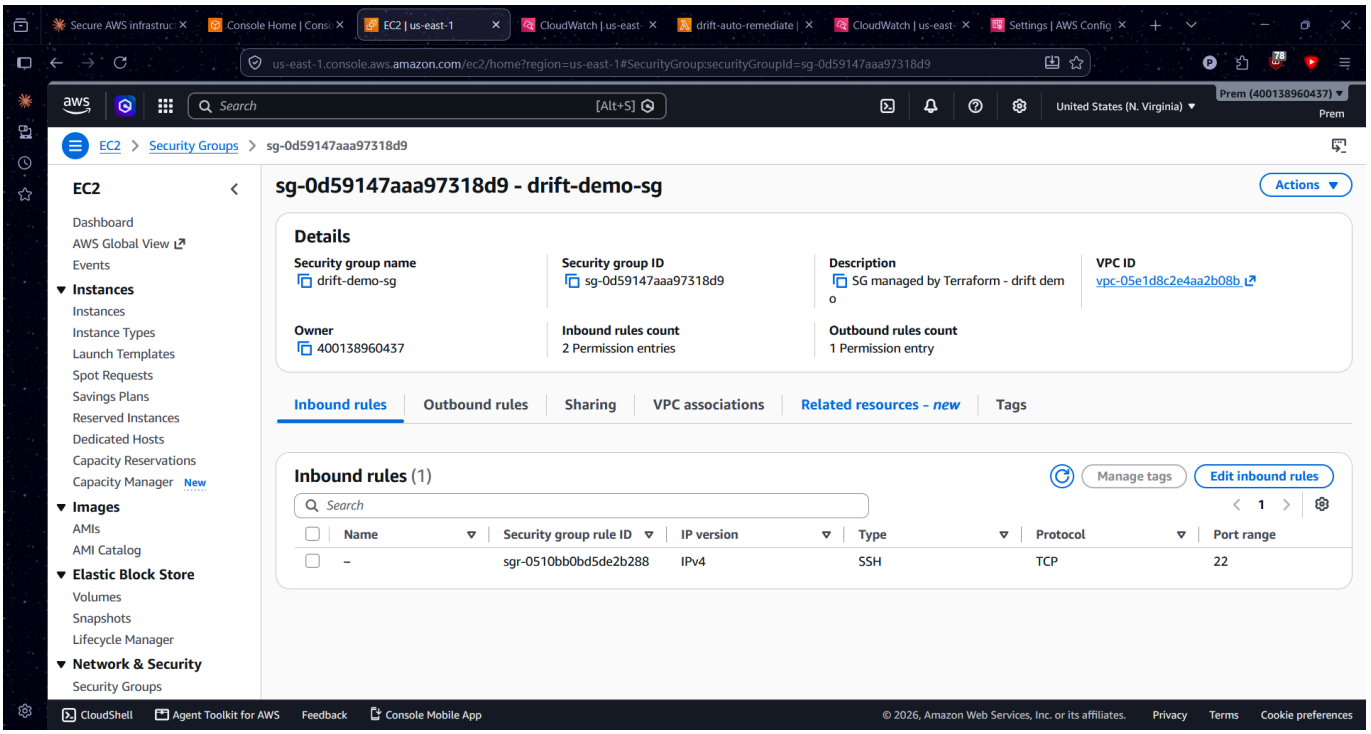
> **CloudWatch Logs** showing the complete automated cycle in action — drift detected, remediation triggered, infrastructure restored. No human touched anything.

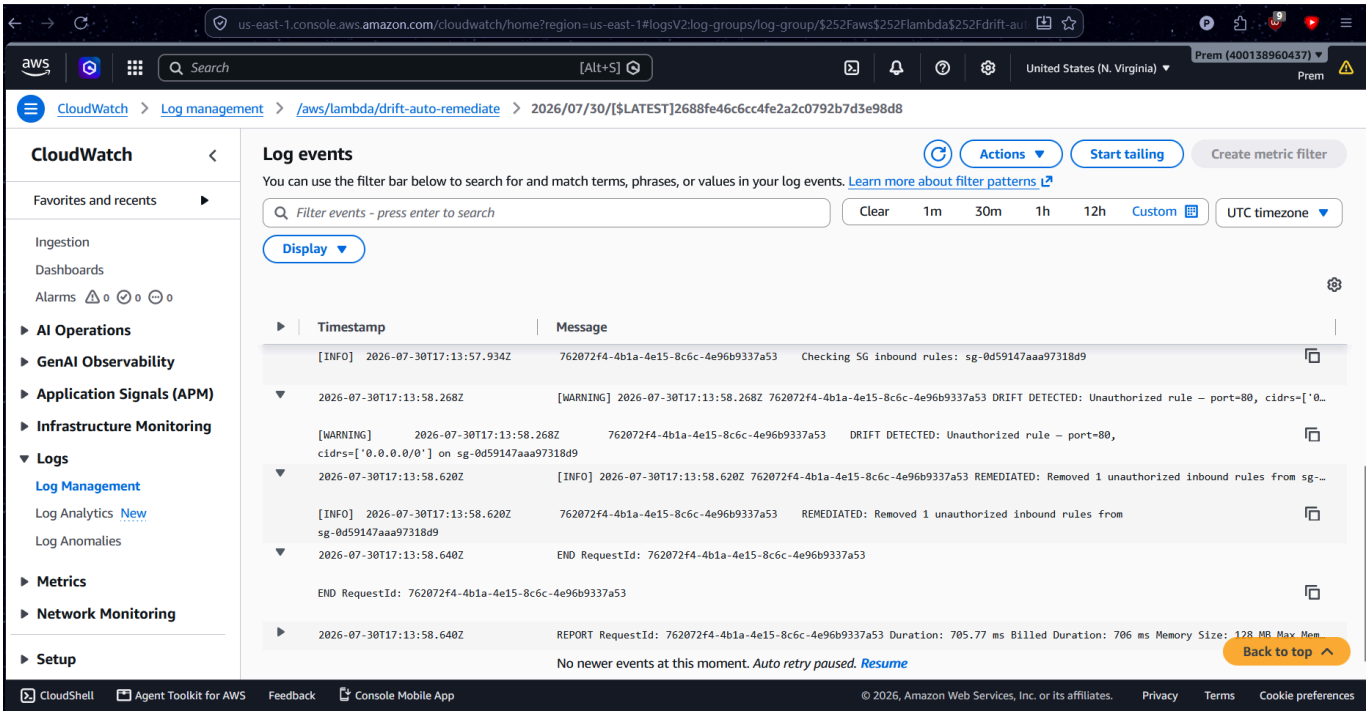

Edit inbound rules
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules
Security group rule ID
Type
Protocol
Port range
Source
Description - optional
sgr-0510bb0bd5de2b288
SSH
TCP
22
Custom
103.171.55.124/32
SSH from my IP only
Delete
Custom TCP
TCP
80
Anywhe...
0.0.0.0/0
Delete
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.
Cancel
Preview changes
Save rules


Inbound security group rules successfully modified on security group (sg-0d59147aaa97318d9 | drift-demo-sg)
Details
sg-0d59147aaa97318d9 - drift-demo-sg
Actions
Details
Security group name
drift-demo-sg
Security group ID
sg-0d59147aaa97318d9
Description
SG managed by Terraform - drift demo
VPC ID
vpc-05e1d8c2e4aa2b08b
Owner
400138960437
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Name
Security group rule ID
IP version
Type
Protocol
Port range
Source
sgr-0060edd7122c08a88
IPv4
HTTP
TCP
80
0.0.0.0/0
sgr-0510bb0bd5de2b288
IPv4
SSH
TCP
22


EventBridge (CloudWatch Events)
Add trigger
Add destination
3 weeks ago
Code
Test
Monitor
Configuration
Aliases
Versions
Test event Info
Delete
CloudWatch Logs Live Tail
Save
Test
To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.
Test event action
Create new event
Edit saved event
Invocation type
Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.
Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.
Event name
test-sg-drift
Event JSON
Format JSON
"detail": {
"resourceId": "sg-0f9dfa9e2d186bad8",
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial


Executing function: succeeded (logs)
Details
Response
Format JSON
{
"statusCode": 200,
"body": "Remediation complete"
}
Summary
Code SHA-256
z2xSJV/4ZHv0kY2QeWSdXvz0sLLdt34WXbXD0EqoqAk=
Execution time
14 seconds ago
Function version
$LATEST
Request ID
762072f4-4b1a-4e15-8c6c-4e96b9337a53
Duration
705.77 ms
Billed duration
706 ms
Resources configured
128 MB
Max memory used
101 MB
Log output
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.
START RequestId: 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Version: $LATEST
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial





Introduction

Here is a scenario that happens more often than most teams want to admit.

It's 11 PM and there's a production incident. An engineer needs to open a port to unblock things quickly. They log into the AWS Console, make the change in 30 seconds, and the incident is resolved. Everyone goes back to sleep.

The port never gets closed.

Weeks later, during an audit, the team finds that the AWS environment no longer matches the Terraform configuration in Git. The change isn't in the commit history because it was never made through code — it

went straight into the AWS Console. At that point, it's often unclear who made the change, why it was made, or exactly when it happened.

This is configuration drift. What starts as a quick operational change can eventually become a security, governance, and audit problem.

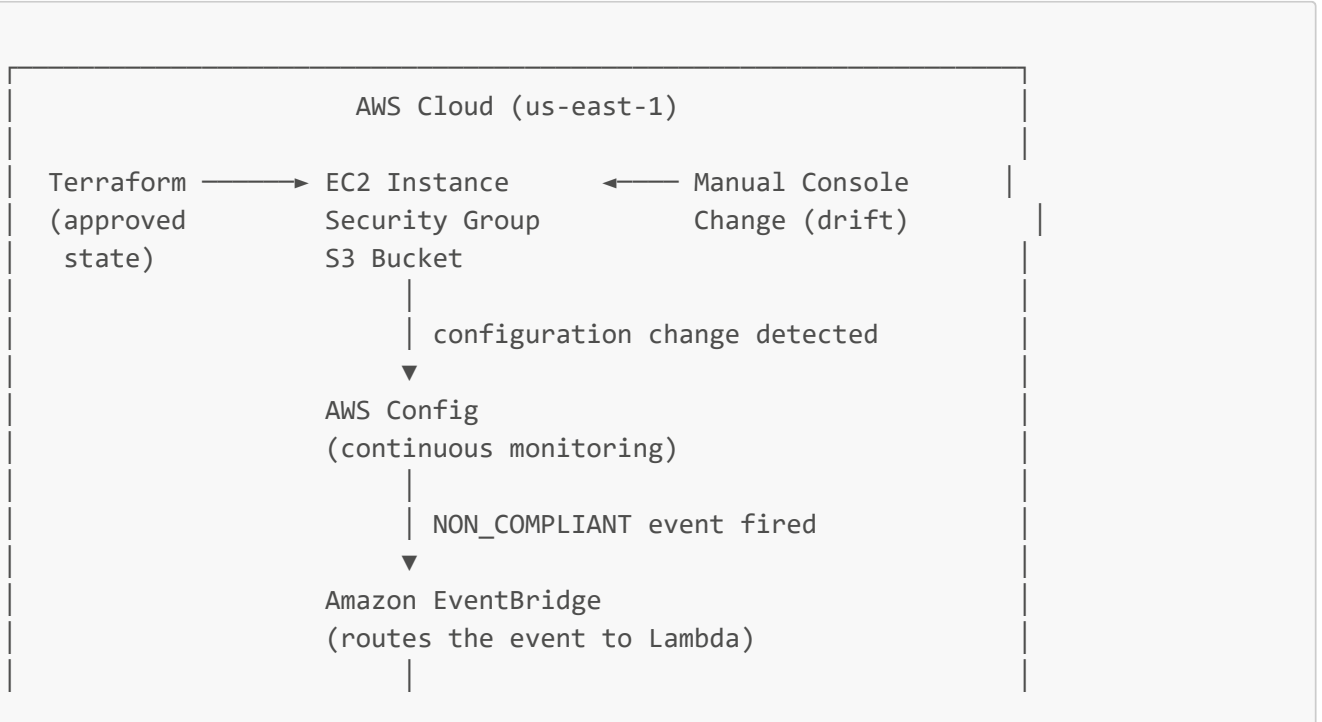
This project is designed to address that problem with an automated system that continuously monitors AWS resources, detects when something deviates from the expected Terraform-managed state, and automatically corrects it — while maintaining a clear audit trail in CloudWatch of what changed, when it changed, and what action was taken.

No one has to manually catch every change or follow a runbook when drift occurs. The system detects the change, triggers remediation, and logs the full process automatically.

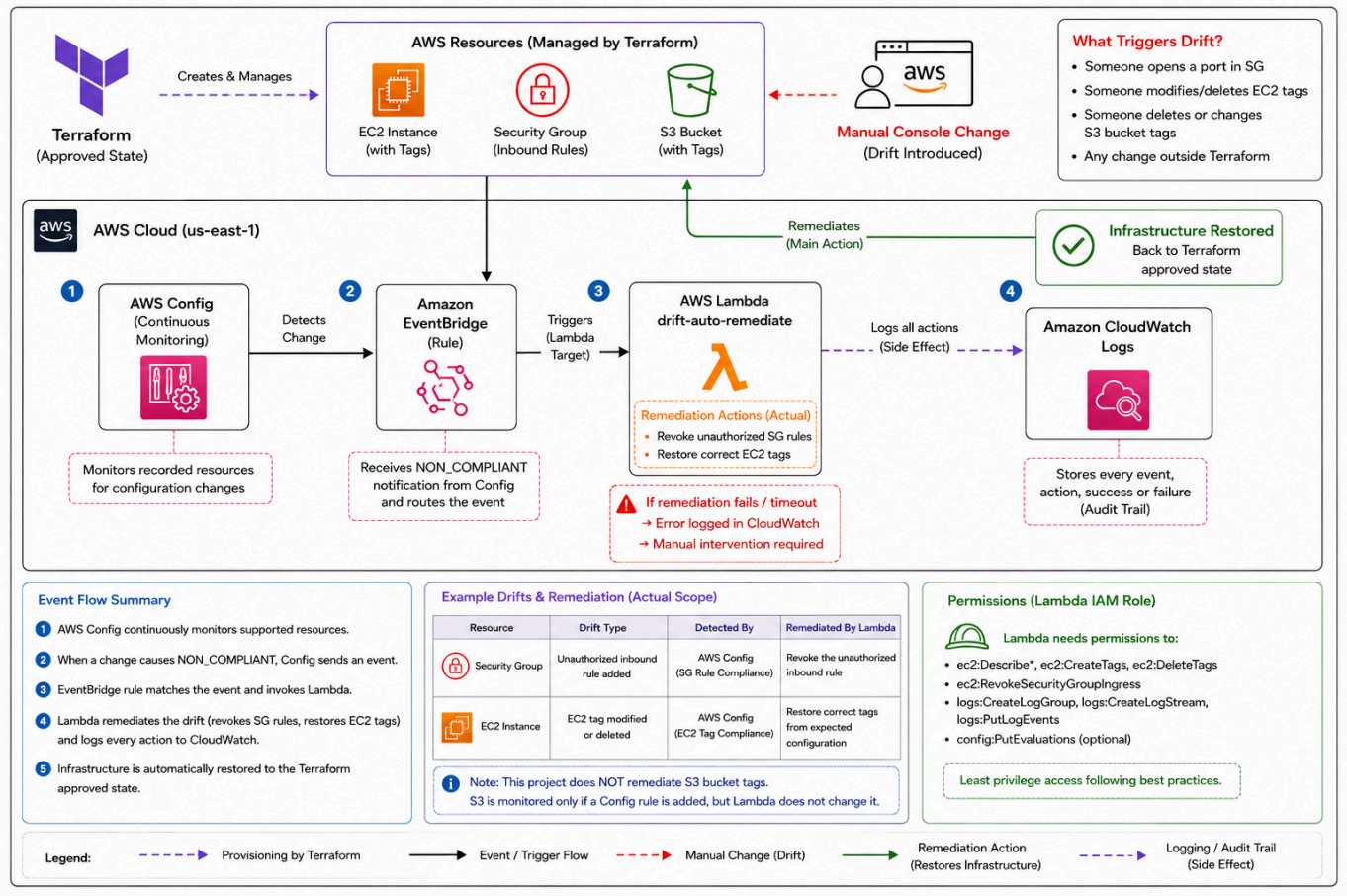
Project Overview

What	Details
Type	Infrastructure Drift Detection + Automated Remediation
Role	Cloud Automation Engineer
Resources monitored	EC2 Instance, Security Group, S3 Bucket
Drift types handled	Unauthorized inbound rules, EC2 tag violations
Remediation	Lambda automatically revokes bad rules and restores tags
Audit trail	CloudWatch Logs records every event with timestamps
Region	us-east-1 (N. Virginia)

Architecture



```
│                    │ triggers                                  │
│                    ▼                                           │
│                 Lambda Function                                │
│                 drift-auto-remediate                           │
│                  ├── revokes unauthorized SG rules             │
│                  └── logs every action taken                   │
│                    │                                           │
│                    ▼                                           │
│                 CloudWatch Logs                                │
│                 (permanent audit trail)                        │
│                    │                                           │
│                    ▼                                           │
│                 Infrastructure Restored ✅                     │
│                 (back to Terraform approved state)             │
└────────────────────────────────────────────────────────────────┘
```



🔄 The Automated Flow — End to End

Here is exactly what happens from the moment someone makes an unauthorized console change to the moment it is fixed:

```
1.  Engineer manually adds port 80 via AWS Console
2.  AWS Config detects the configuration change
3.  Config Rule evaluates → marks Security Group NON_COMPLIANT
```

```
4.  EventBridge receives the compliance change event
5.  EventBridge triggers the Lambda function automatically
6.  Lambda identifies the unauthorized port 80 rule
7.  Lambda calls revoke_security_group_ingress via boto3
8.  Port 80 rule is removed — approved state restored
9.  CloudWatch Logs records every step with timestamps
10. Zero human intervention required throughout
```

The whole thing from detection to remediation takes between 2 and 15 minutes depending on how AWS Config evaluation timing lines up. For demos, triggering a manual re-evaluate in Config drops that to under 30 seconds.

Services Used

Service	Role
Terraform	Defines the approved state — EC2, Security Group, S3
AWS Config	Continuously watches resources and evaluates compliance
Amazon EventBridge	Routes NON_COMPLIANT events from Config to Lambda
AWS Lambda	The brain — auto-remediates drift when triggered
CloudWatch Logs	Records everything — the permanent audit trail
IAM	Scoped role that gives Lambda exactly the permissions it needs

---

Technologies and Tools

```
Terraform       Infrastructure as Code — provisions and defines approved state
Python 3.12     Lambda function runtime
boto3           AWS SDK for Python — makes the actual EC2 API calls
AWS Config      Drift detection and compliance evaluation engine
EventBridge     Serverless event routing between Config and Lambda
Lambda          Serverless auto-remediation — no servers to manage
CloudWatch      Audit logging and monitoring
IAM             Permission management — least privilege throughout
```

Project Structure

```
drift-detection-project/
│
├── terraform/
│   ├── provider.tf          ← AWS provider and version constraints
│   ├── main.tf              ← EC2, Security Group, S3 resources
│   ├── variables.tf         ← Your IP and configurable values
```

```
│   └── outputs.tf           ← Resource IDs and ARNs as output
│
├── lambda/
│   └── lambda_function.py   ← Auto-remediation logic
│
├── image/                   ← All 18 project screenshots
│   ├── 01-terraform-apply-output.png
│   ├── 02-aws-resources-created.png
│   └── ... (all screenshots)
│
└── README.md
```

🚀 Step-by-Step Deployment

Phase 1 — Provision Infrastructure with Terraform

Terraform creates the three AWS resources that form the approved baseline. Every resource gets a `ManagedBy = Terraform` tag — this is how AWS Config knows which resources belong to this project.

provider.tf

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

main.tf

```
resource "aws_security_group" "app_sg" {
  name        = "drift-demo-sg"
  description = "SG managed by Terraform - drift detection demo"

  ingress {
    description = "SSH from authorized IP only"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.your_ip]
```

```
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name        = "drift-demo-sg"
    ManagedBy   = "Terraform"
    Environment = "lab"
  }
}

resource "aws_instance" "app_server" {
  ami             = "ami-0c101f26f147fa7fd"
  instance_type   = "t2.micro"
  security_groups = [aws_security_group.app_sg.name]

  tags = {
    Name        = "drift-demo-ec2"
    ManagedBy   = "Terraform"
    Environment = "lab"
  }
}

resource "aws_s3_bucket" "drift_bucket" {
  bucket = "drift-demo-bucket-prem-lab"

  tags = {
    Name      = "drift-demo-bucket"
    ManagedBy = "Terraform"
  }
}
```

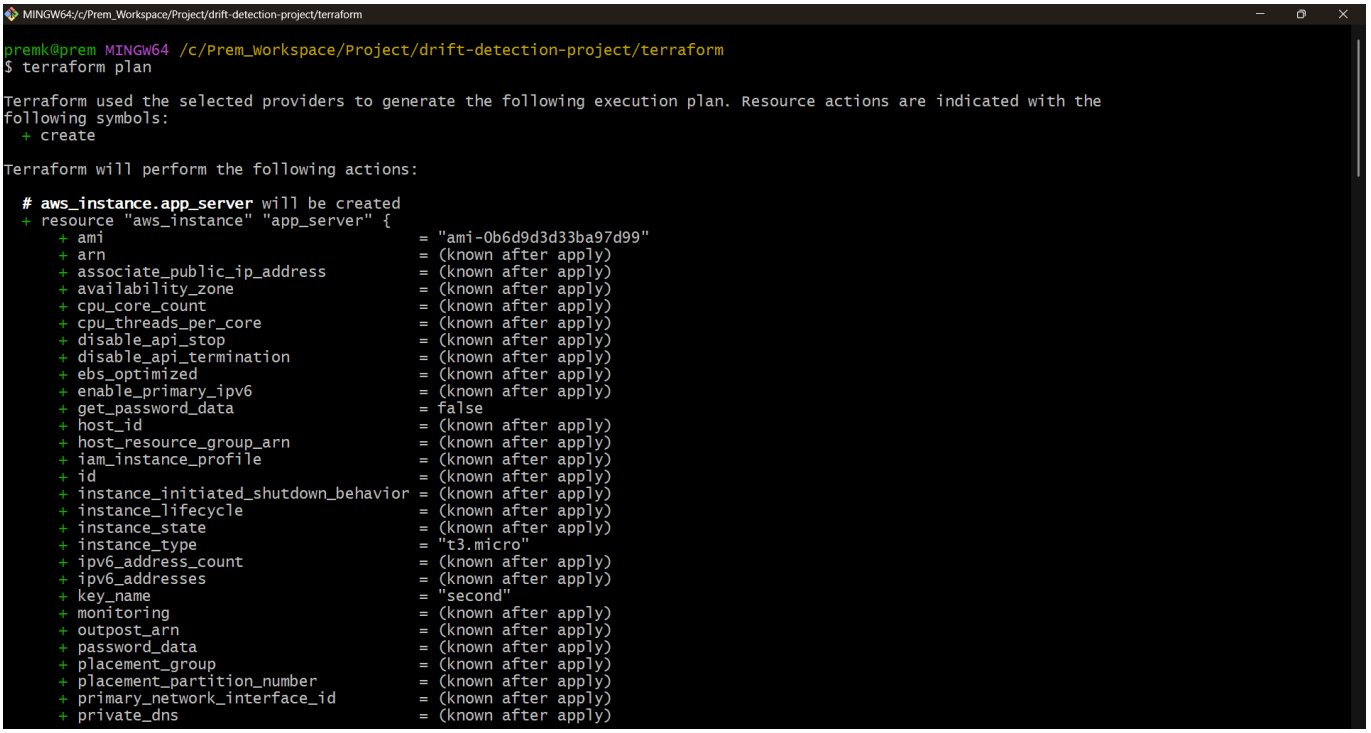
variables.tf

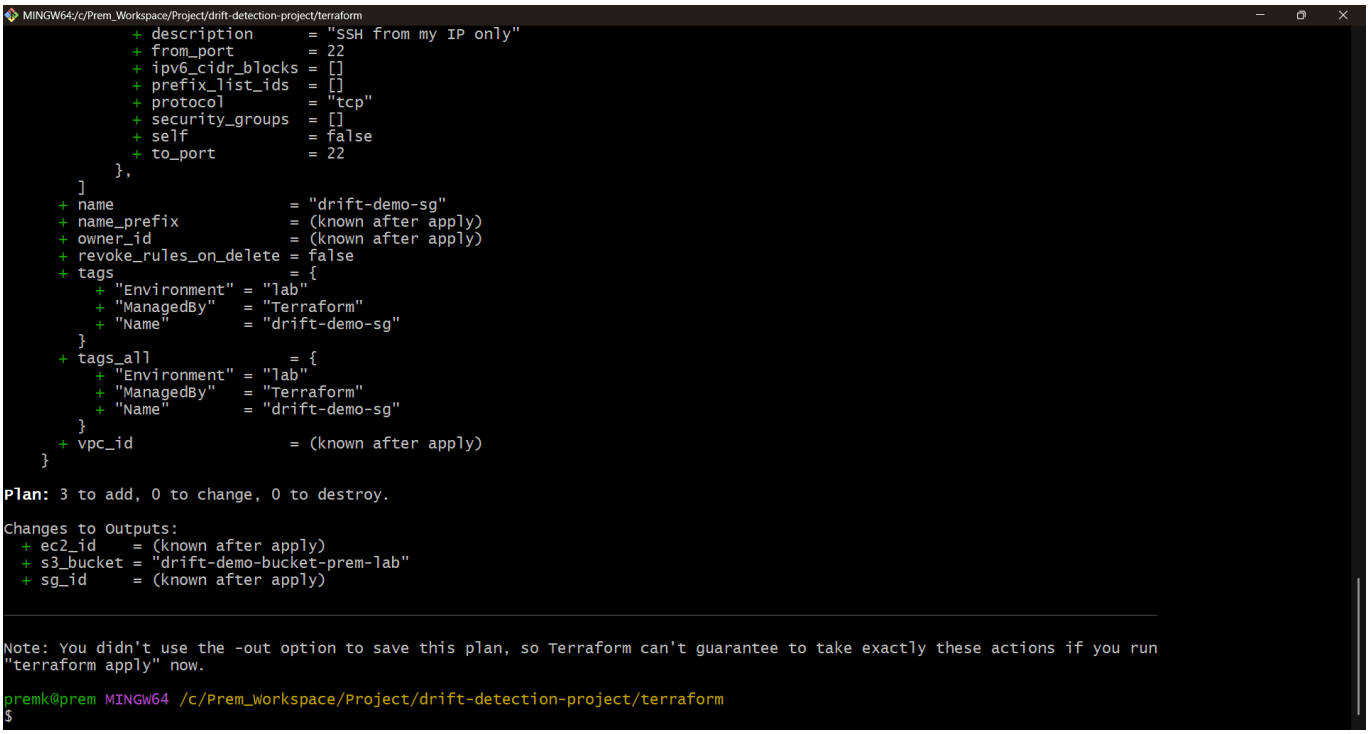
```
variable "your_ip" {
  description = "Your authorized IP address for SSH"
  type        = string
  default     = "YOUR_IP/32"
}
```

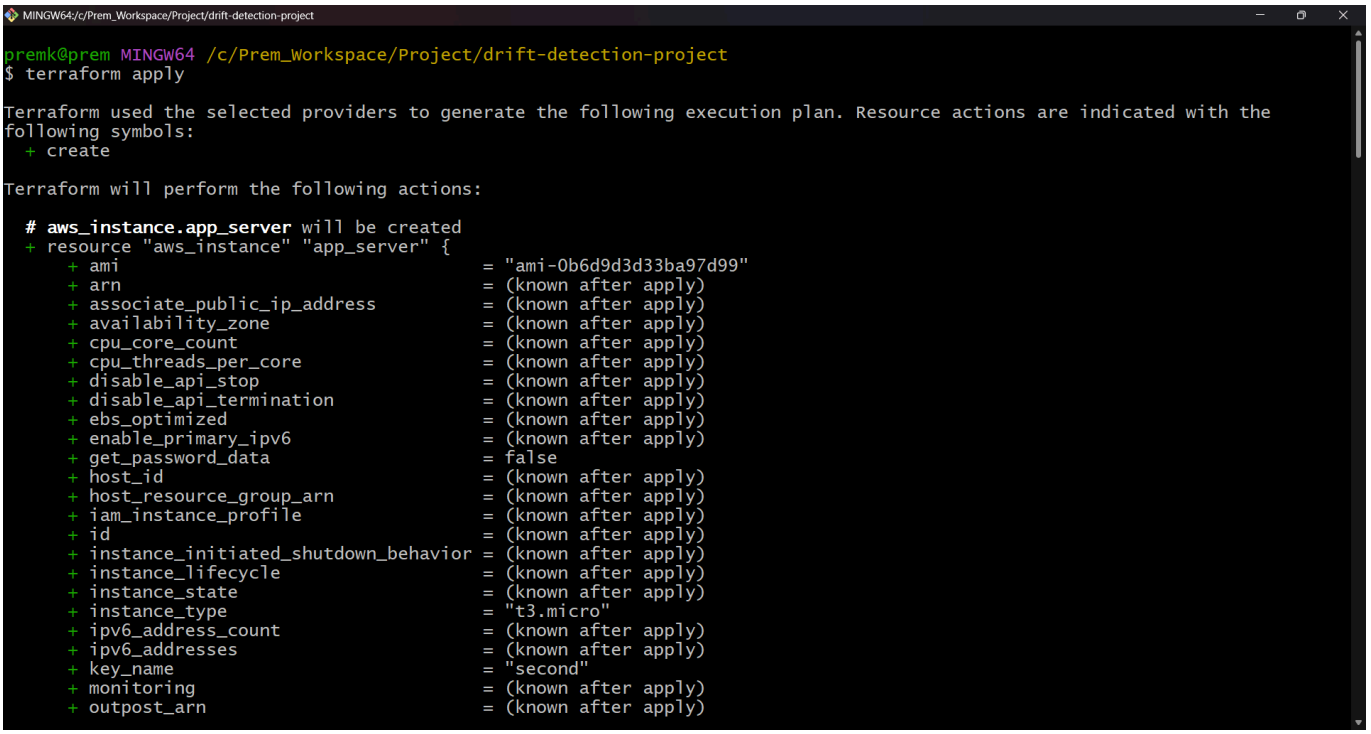
Run it

```
terraform init
terraform plan
terraform apply -auto-approve
```

After apply completes, you have your baseline. Three resources, all tagged, all matching the Terraform definition exactly. This is the approved state everything will be compared against.







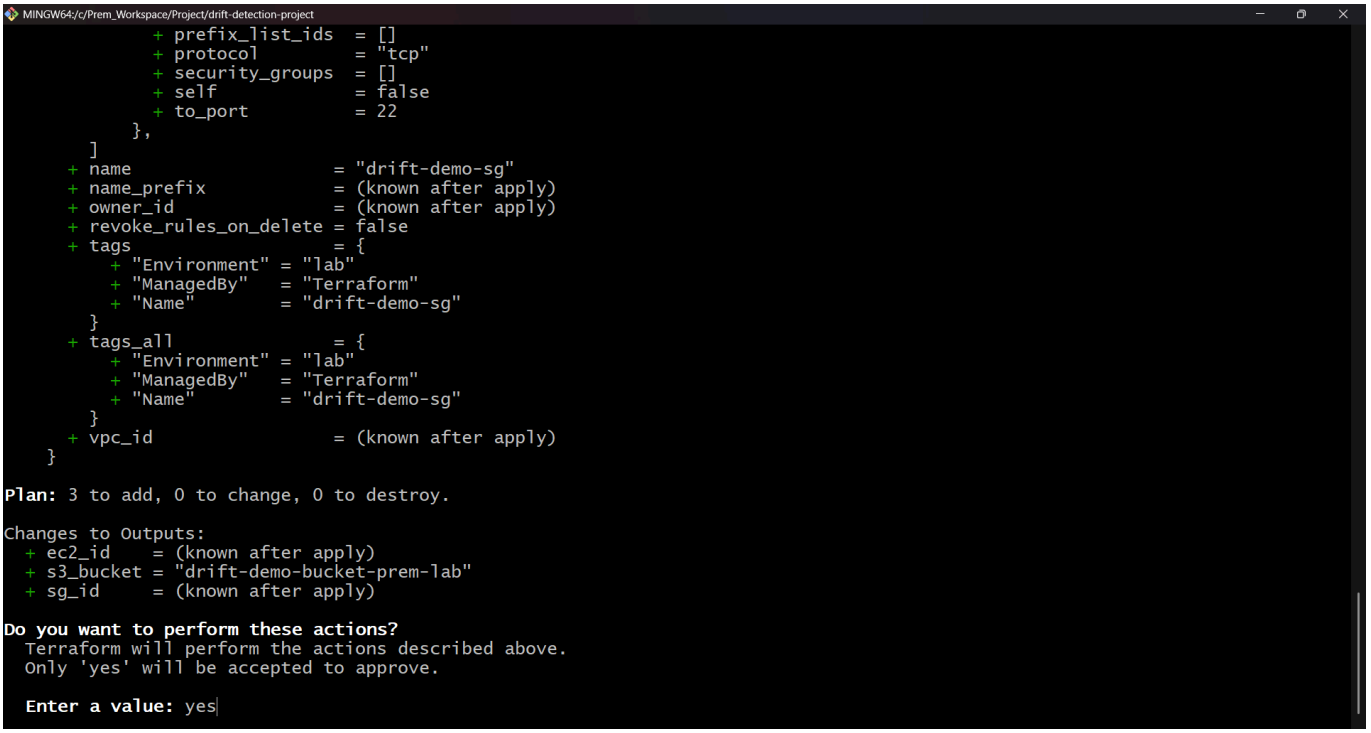
```
MINGW64:/c/Prem_Workspace/Project/drift-detection-project

premk@prem MINGW64 /c/Prem_Workspace/Project/drift-detection-project
$ terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # aws_instance.app_server will be created
  + resource "aws_instance" "app_server" {
      + ami                                  = "ami-0b6d9d3d33ba97d99"
      + arn                                  = (known after apply)
      + associate_public_ip_address          = (known after apply)
      + availability_zone                    = (known after apply)
      + cpu_core_count                       = (known after apply)
      + cpu_threads_per_core                 = (known after apply)
      + disable_api_stop                     = (known after apply)
      + disable_api_termination              = (known after apply)
      + ebs_optimized                        = (known after apply)
      + enable_primary_ipv6                  = (known after apply)
      + get_password_data                    = false
      + host_id                              = (known after apply)
      + host_resource_group_arn              = (known after apply)
      + iam_instance_profile                 = (known after apply)
      + id                                   = (known after apply)
      + instance_initiated_shutdown_behavior = (known after apply)
      + instance_lifecycle                   = (known after apply)
      + instance_state                       = (known after apply)
      + instance_type                        = "t3.micro"
      + ipv6_address_count                   = (known after apply)
      + ipv6_addresses                       = (known after apply)
      + key_name                             = "second"
      + monitoring                           = (known after apply)
      + outpost_arn                          = (known after apply)
```



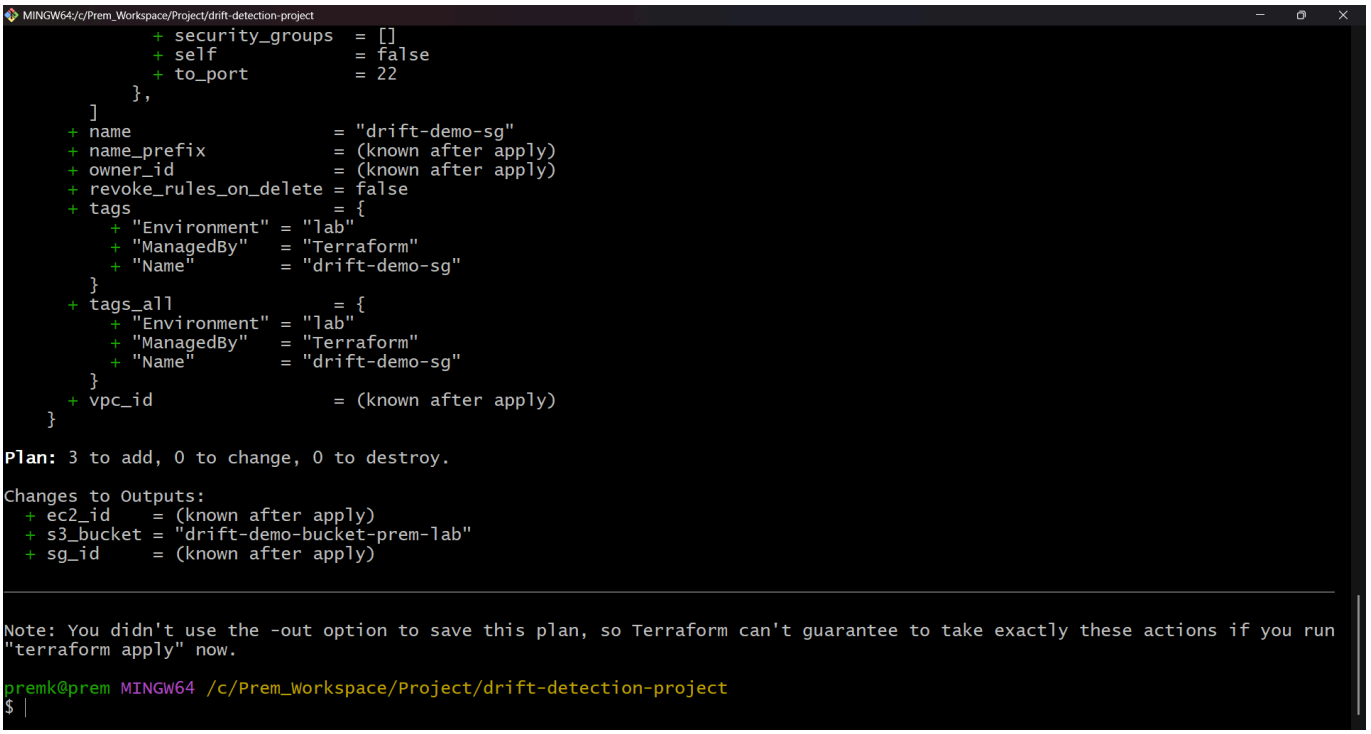
```
MINGW64:/c/Prem_Workspace/Project/drift-detection-project
            + prefix_list_ids  = []
            + protocol         = "tcp"
            + security_groups  = []
            + self             = false
            + to_port          = 22
          },
      ]
      + name                   = "drift-demo-sg"
      + name_prefix            = (known after apply)
      + owner_id               = (known after apply)
      + revoke_rules_on_delete = false
      + tags                   = {
          + "Environment" = "lab"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "drift-demo-sg"
        }
      + tags_all               = {
          + "Environment" = "lab"
          + "ManagedBy"   = "Terraform"
          + "Name"        = "drift-demo-sg"
        }
      + vpc_id                 = (known after apply)
    }

Plan: 3 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + ec2_id    = (known after apply)
  + s3_bucket = "drift-demo-bucket-prem-lab"
  + sg_id     = (known after apply)

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```



Phase 2 — Simulate Configuration Drift

Now for the fun part — breaking things on purpose to prove the system catches it.

These two manual changes simulate exactly what happens in real engineering teams under pressure. Someone adds a rule "temporarily." Someone updates a tag "just to test something." Neither change makes it into the Terraform code. Drift is born.

Drift 1 — Unauthorized Security Group Rule

```
AWS Console → EC2 → Security Groups → drift-demo-sg
→ Edit inbound rules → Add rule

Type:        HTTP
Port:        80
Source:      0.0.0.0/0
Description: temp-manual-change

→ Save rules
```

Drift 2 — EC2 Tag Violation

```
AWS Console → EC2 → Instances → drift-demo-ec2
→ Tags → Manage tags → Add tag

Key:   Owner
Value: manually-added-not-in-terraform
```

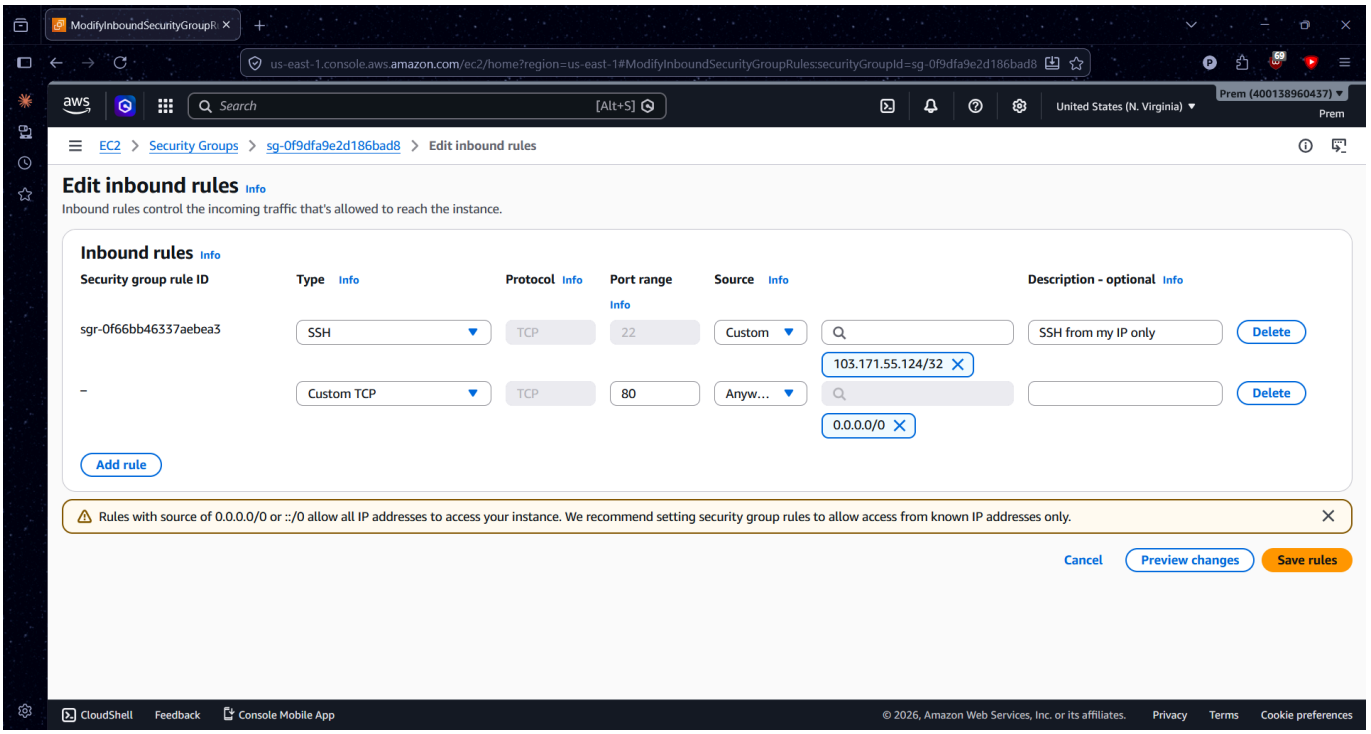
```
→ Save
```

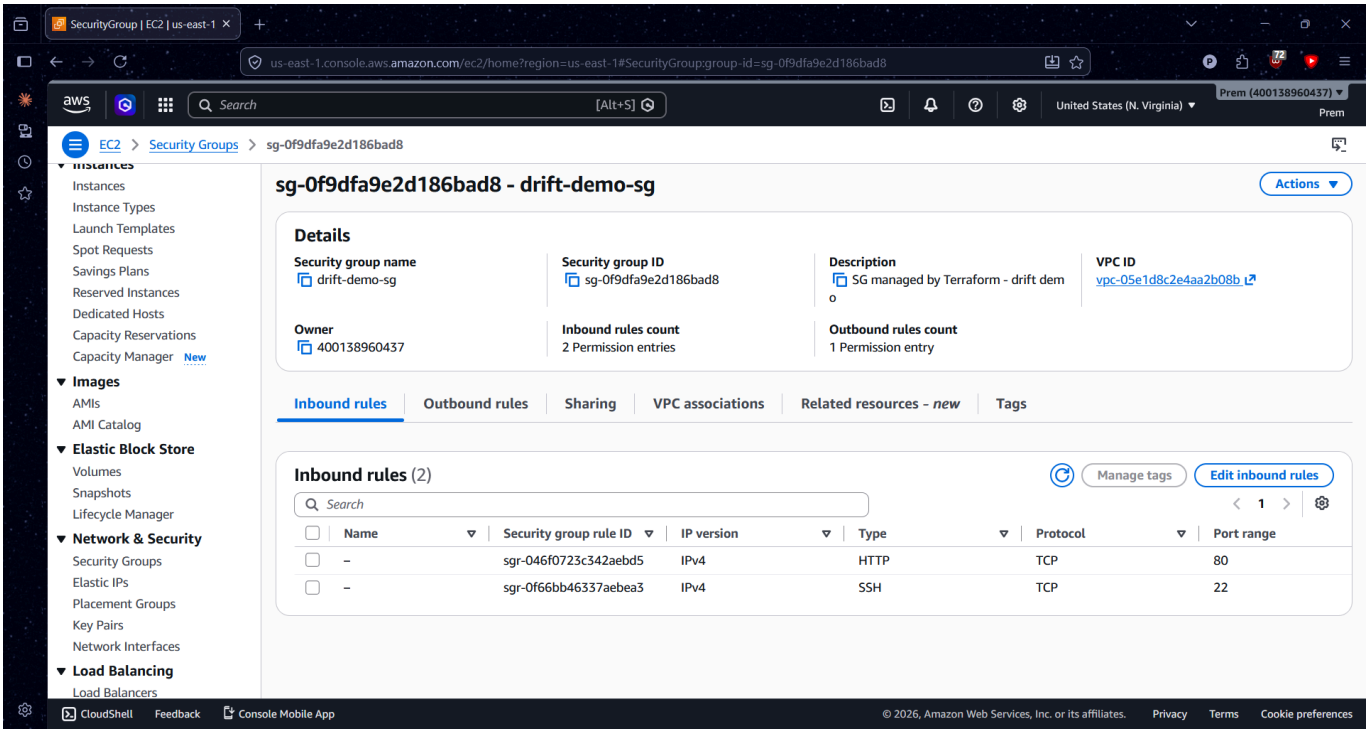
Confirm drift with terraform plan

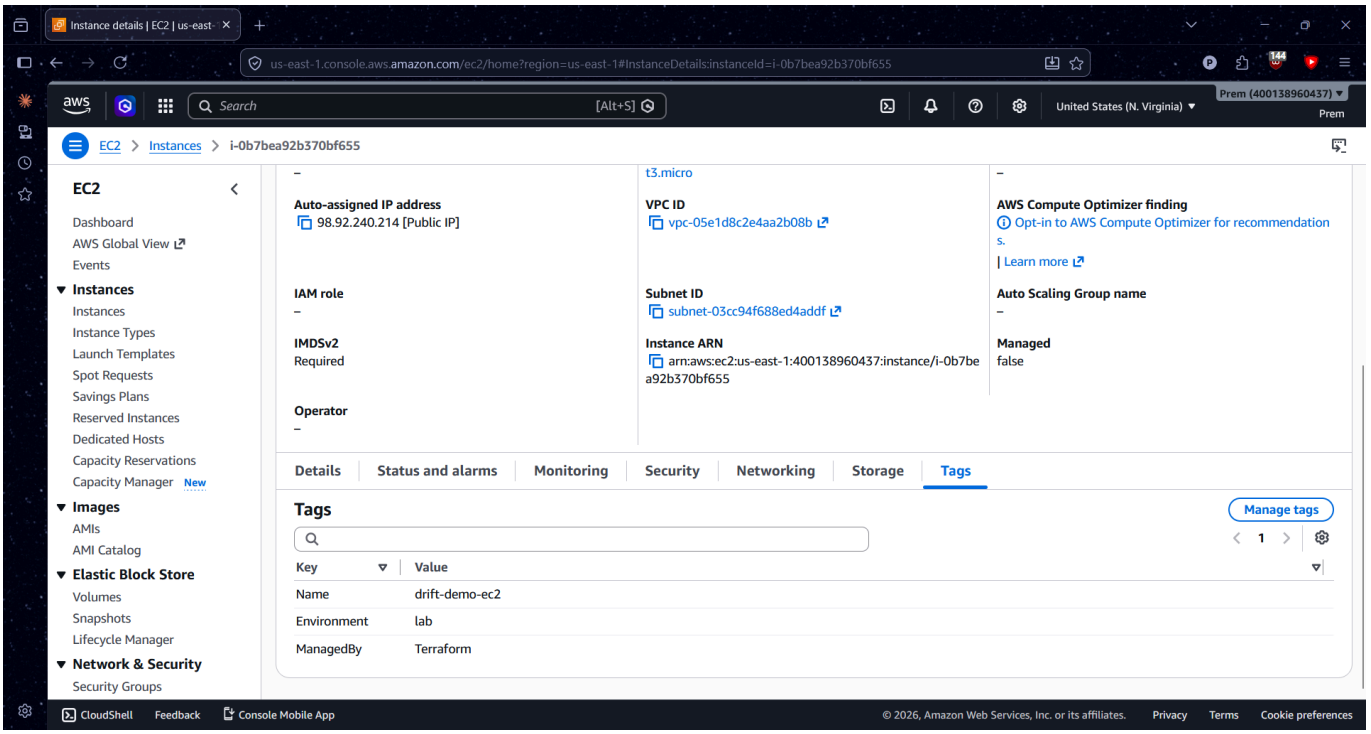
```
terraform plan

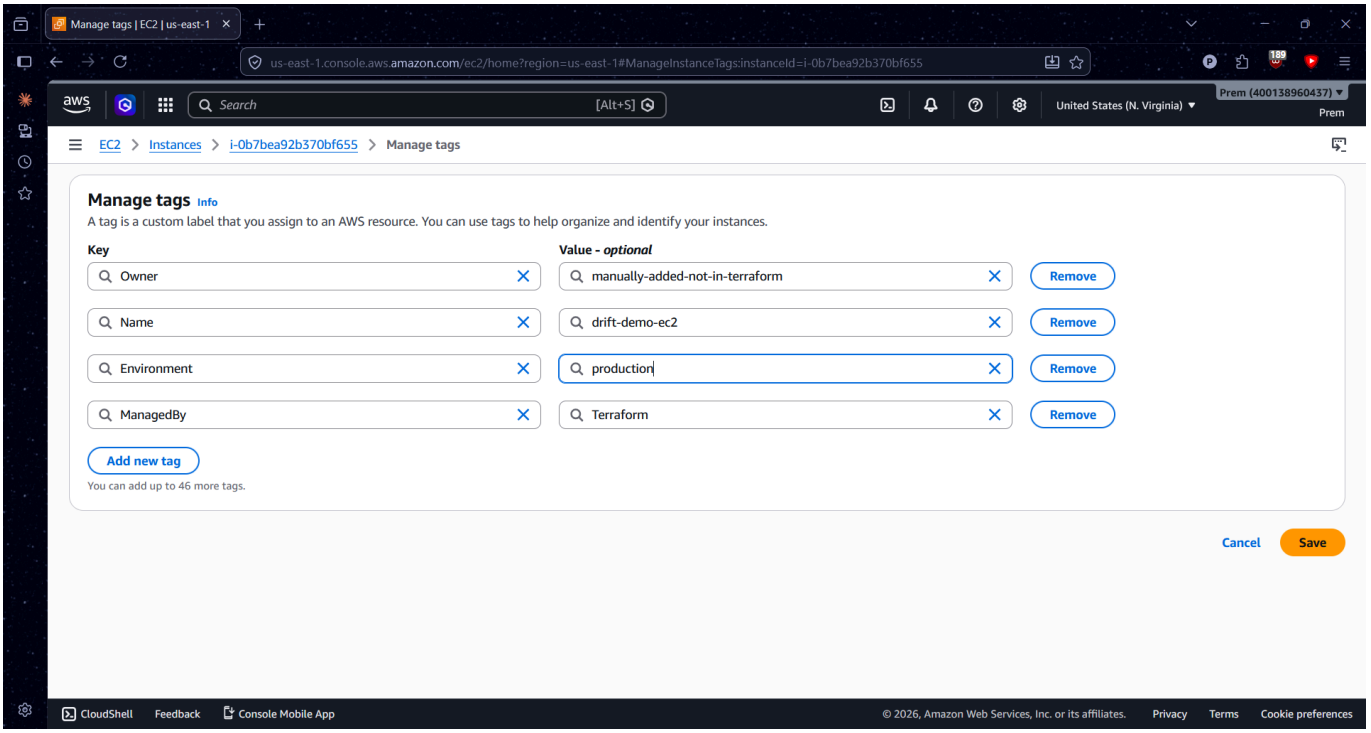
# You should see:
# ~ aws_security_group.app_sg will be updated
#   ingress rules differ from approved state
# ~ aws_instance.app_server will be updated
#   tags differ from approved state
```

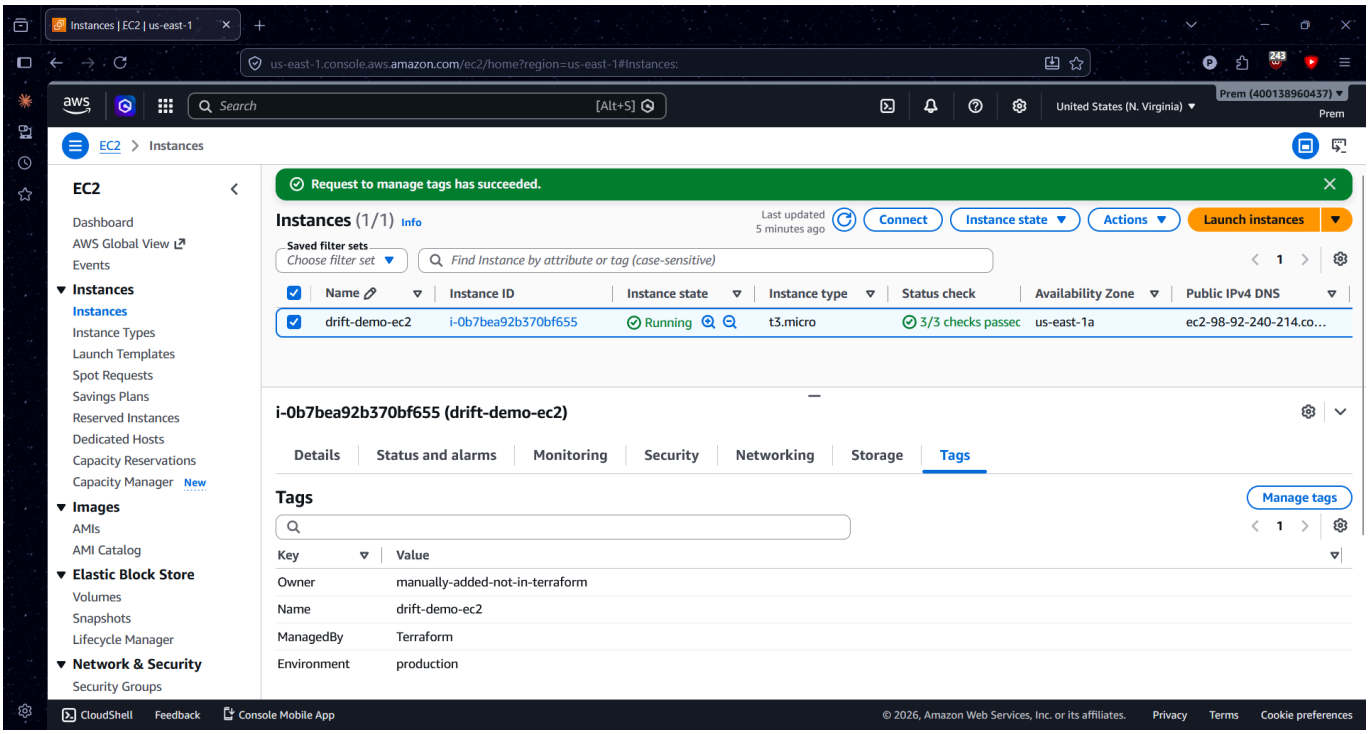
The `terraform plan` output is your evidence that drift exists. Terraform can see the gap between its state file and what AWS actually has right now.

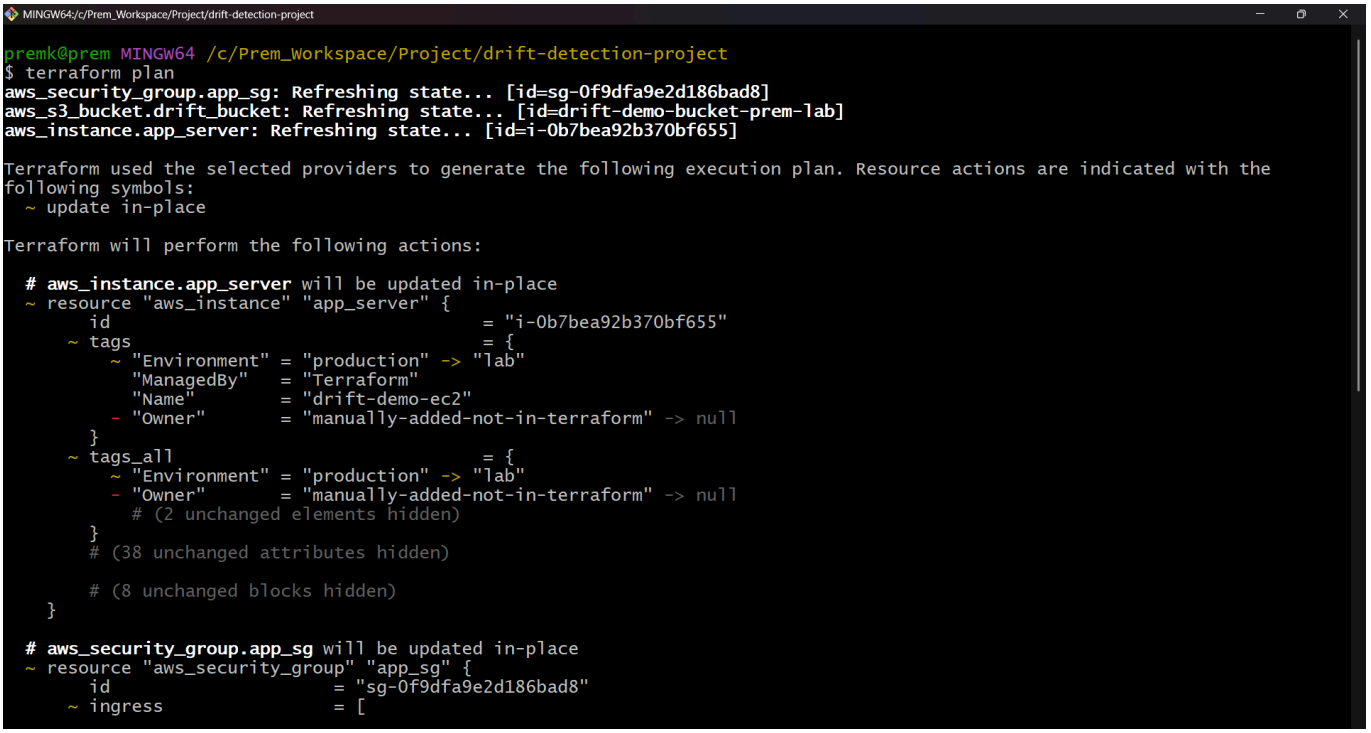

ModifyInboundSecurityGroupR
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ModifyInboundSecurityGroupRules:securityGroupId=sg-0f9dfa9e2d186bad8
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
EC2 > Security Groups > sg-0f9dfa9e2d186bad8 > Edit inbound rules
Edit inbound rules Info
Inbound rules control the incoming traffic that's allowed to reach the instance.
Inbound rules Info
Security group rule ID
Type Info
Protocol Info
Port range Info
Source Info
Description - optional Info
sgr-0f66bb46337aebea3
SSH
TCP
22
Custom
103.171.55.124/32
SSH from my IP only
Delete
-
Custom TCP
TCP
80
Anyw...
0.0.0.0/0
Delete
Add rule
Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.
Cancel
Preview changes
Save rules
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

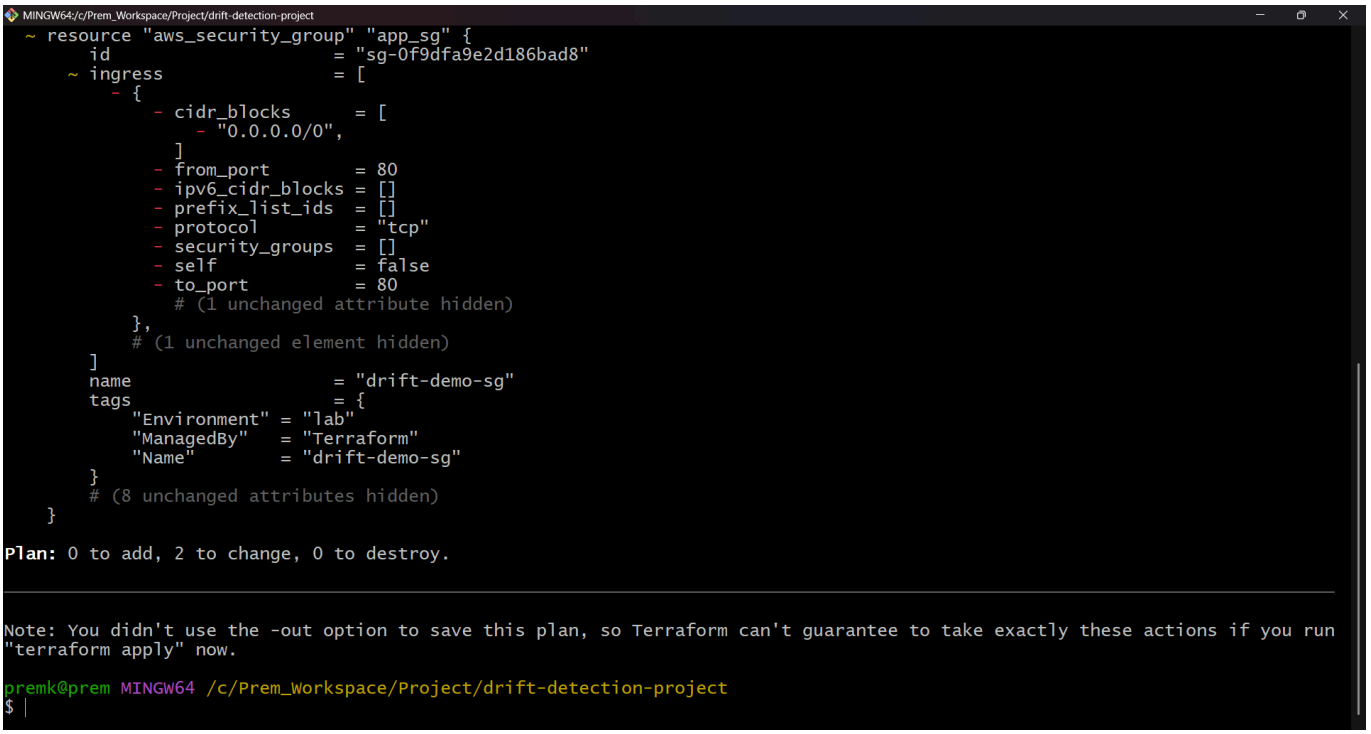

SecurityGroup | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:group-id=sg-0f9dfa9e2d186bad8
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
EC2 > Security Groups > sg-0f9dfa9e2d186bad8
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
Network Interfaces
Load Balancing
Load Balancers
sg-0f9dfa9e2d186bad8 - drift-demo-sg
Actions
Details
Security group name
drift-demo-sg
Security group ID
sg-0f9dfa9e2d186bad8
Description
SG managed by Terraform - drift demo
VPC ID
vpc-05e1d8c2e4aa2b08b
Owner
400138960437
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Search
Name
Security group rule ID
IP version
Type
Protocol
Port range
-
sgr-046f0723c342aebd5
IPv4
HTTP
TCP
80
-
sgr-0f66bb46337aebea3
IPv4
SSH
TCP
22
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Instance details | EC2 | us-east-
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#InstanceDetails:instanceId=i-0b7bea92b370bf655
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > i-0b7bea92b370bf655
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
t3.micro
Auto-assigned IP address
98.92.240.214 [Public IP]
VPC ID
vpc-05e1d8c2e4aa2b08b
AWS Compute Optimizer finding
Opt-in to AWS Compute Optimizer for recommendations.
Learn more
IAM role
Subnet ID
subnet-03cc94f688ed4addf
Auto Scaling Group name
IMDSv2
Required
Instance ARN
arn:aws:ec2:us-east-1:400138960437:instance/i-0b7bea92b370bf655
Managed
false
Operator
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Tags
Manage tags
Key
Value
Name
drift-demo-ec2
Environment
lab
ManagedBy
Terraform
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Manage tags | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#ManageInstanceTags:instanceId=i-0b7bea92b370bf655
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances > i-0b7bea92b370bf655 > Manage tags
Manage tags Info
A tag is a custom label that you assign to an AWS resource. You can use tags to help organize and identify your instances.
Key
Value - optional
Owner
manually-added-not-in-terraform
Remove
Name
drift-demo-ec2
Remove
Environment
production
Remove
ManagedBy
Terraform
Remove
Add new tag
You can add up to 46 more tags.
Cancel
Save
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Instances | EC2 | us-east-1
us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
EC2 > Instances
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Request to manage tags has succeeded.
Instances (1/1) Info
Last updated 5 minutes ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Find Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
drift-demo-ec2
i-0b7bea92b370bf655
Running
t3.micro
3/3 checks passed
us-east-1a
ec2-98-92-240-214.co...
i-0b7bea92b370bf655 (drift-demo-ec2)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Tags
Manage tags
Key
Value
Owner
manually-added-not-in-terraform
Name
drift-demo-ec2
ManagedBy
Terraform
Environment
production
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


MINGW64:/c/Prem_Workspace/Project/drift-detection-project
premk@prem MINGW64 /c/Prem_Workspace/Project/drift-detection-project
$ terraform plan
aws_security_group.app_sg: Refreshing state... [id=sg-0f9dfa9e2d186bad8]
aws_s3_bucket.drift_bucket: Refreshing state... [id=drift-demo-bucket-prem-lab]
aws_instance.app_server: Refreshing state... [id=i-0b7bea92b370bf655]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
~ update in-place
Terraform will perform the following actions:
aws_instance.app_server will be updated in-place
~ resource "aws_instance" "app_server" {
id = "i-0b7bea92b370bf655"
~ tags = {
~ "Environment" = "production" -> "lab"
"ManagedBy" = "Terraform"
"Name" = "drift-demo-ec2"
- "Owner" = "manually-added-not-in-terraform" -> null
}
~ tags_all = {
~ "Environment" = "production" -> "lab"
- "Owner" = "manually-added-not-in-terraform" -> null
(2 unchanged elements hidden)
}
(38 unchanged attributes hidden)
(8 unchanged blocks hidden)
}
aws_security_group.app_sg will be updated in-place
~ resource "aws_security_group" "app_sg" {
id = "sg-0f9dfa9e2d186bad8"
~ ingress = [



Phase 3 — AWS Config Drift Detection

AWS Config is the continuous monitoring layer. It records a snapshot every time a resource configuration changes — and evaluates those snapshots against compliance rules you define.

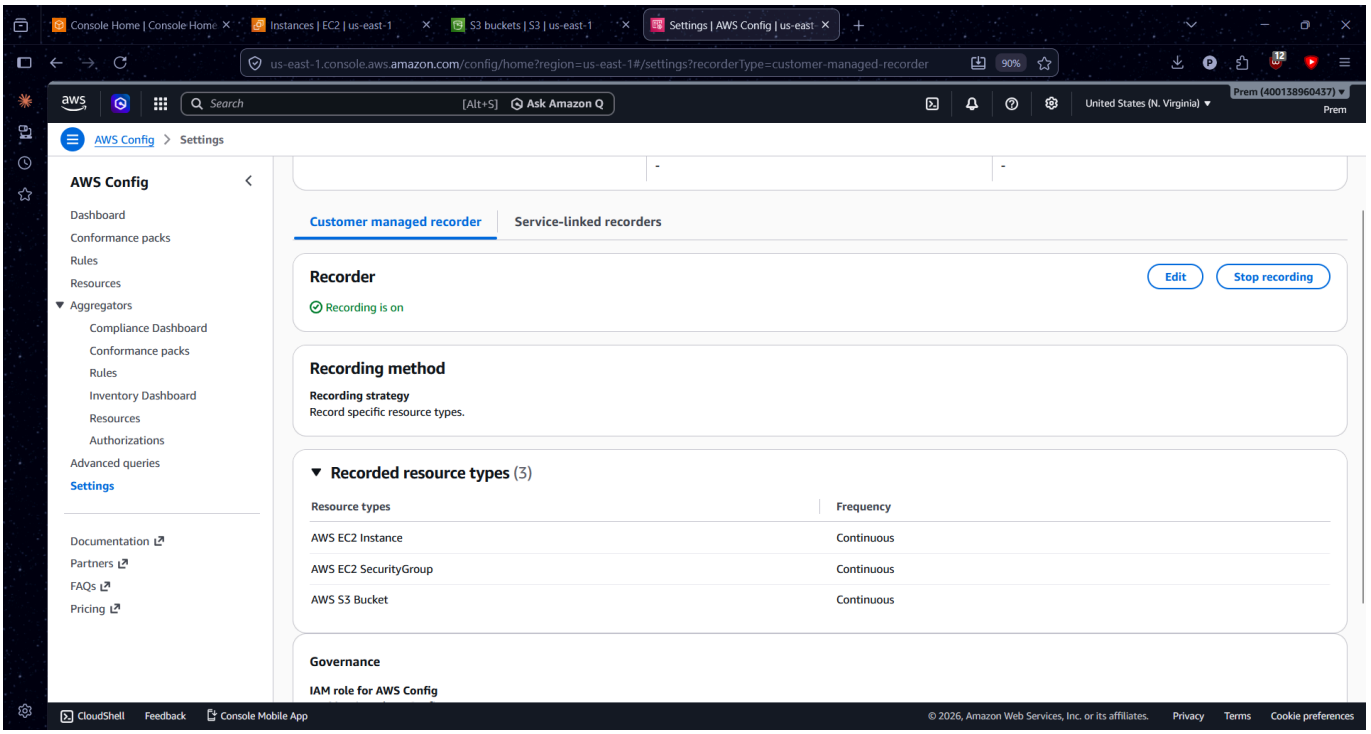
Enable AWS Config Recording

```
AWS Config → Settings → Edit

Resource types to record:
  AWS::EC2::Instance
  AWS::EC2::SecurityGroup
  AWS::S3::Bucket

Delivery method: S3 bucket → config-delivery-prem-lab
Service role: Create new → AWSServiceRoleForConfig

→ Save
```



Create Config Rule

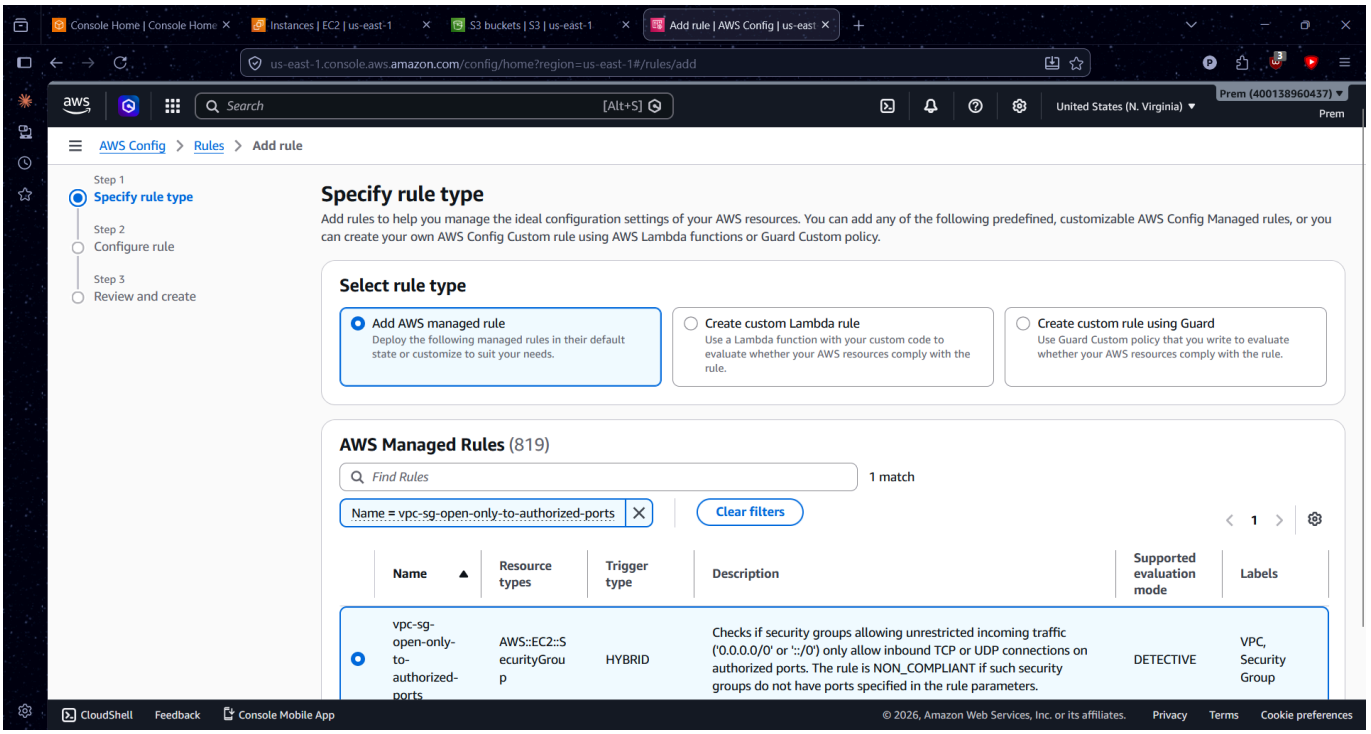
```
AWS Config → Rules → Add rule

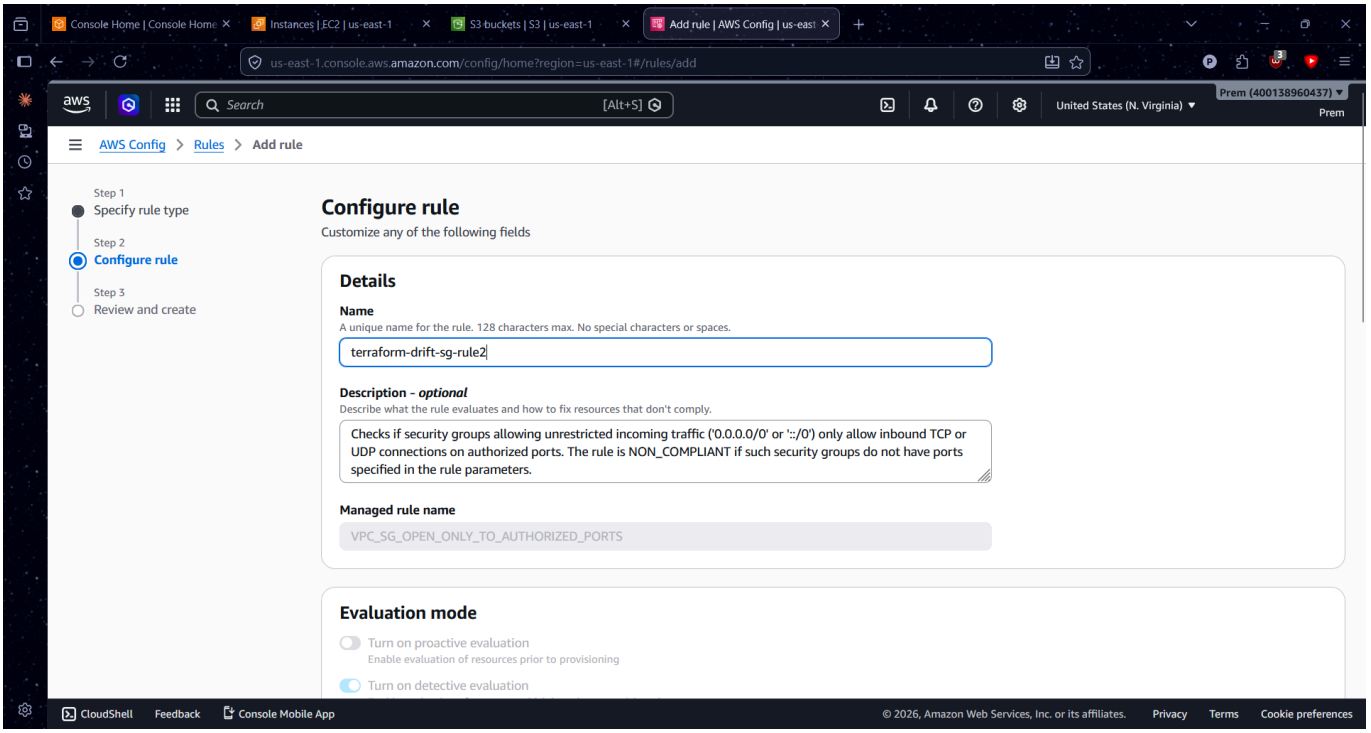
Managed rule: vpc-sg-open-only-to-authorized-ports
Rule name:    terraform-drift-sg-rule2

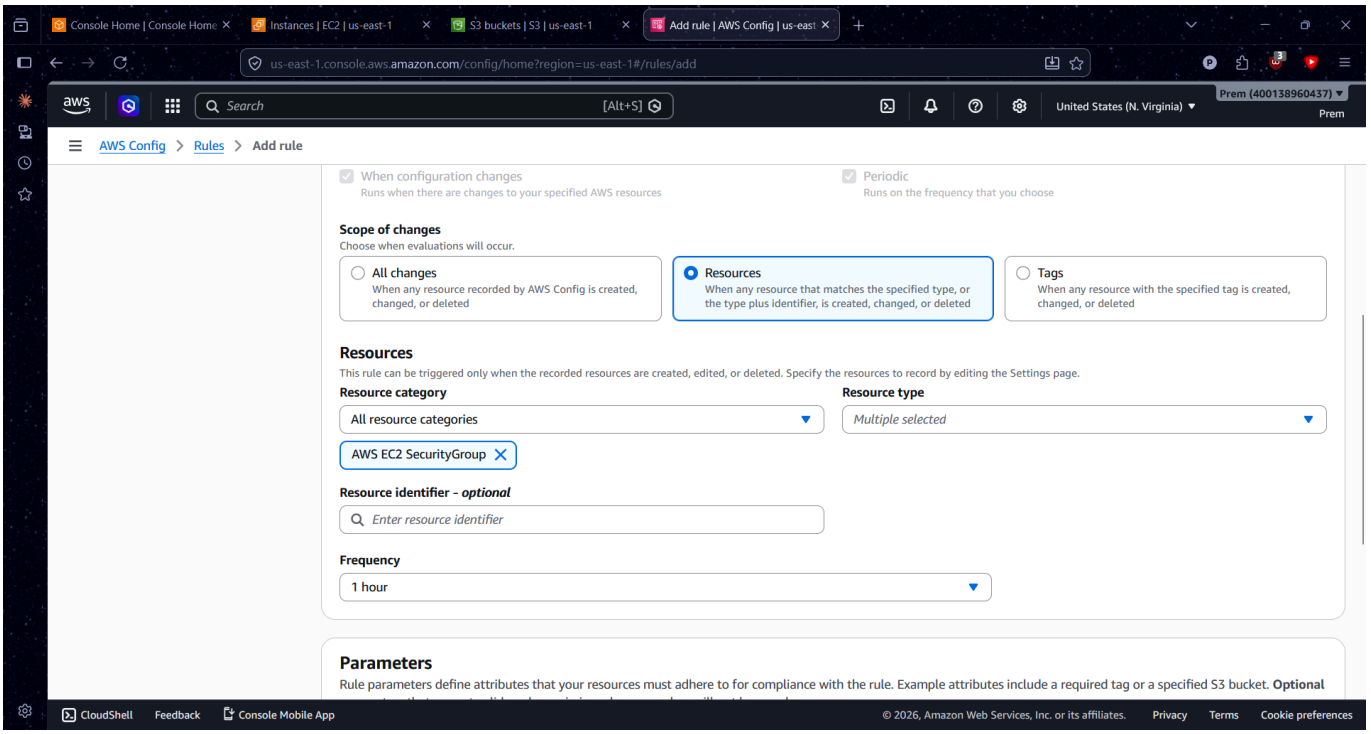
Parameters:
  authorizedTcpPorts: 22

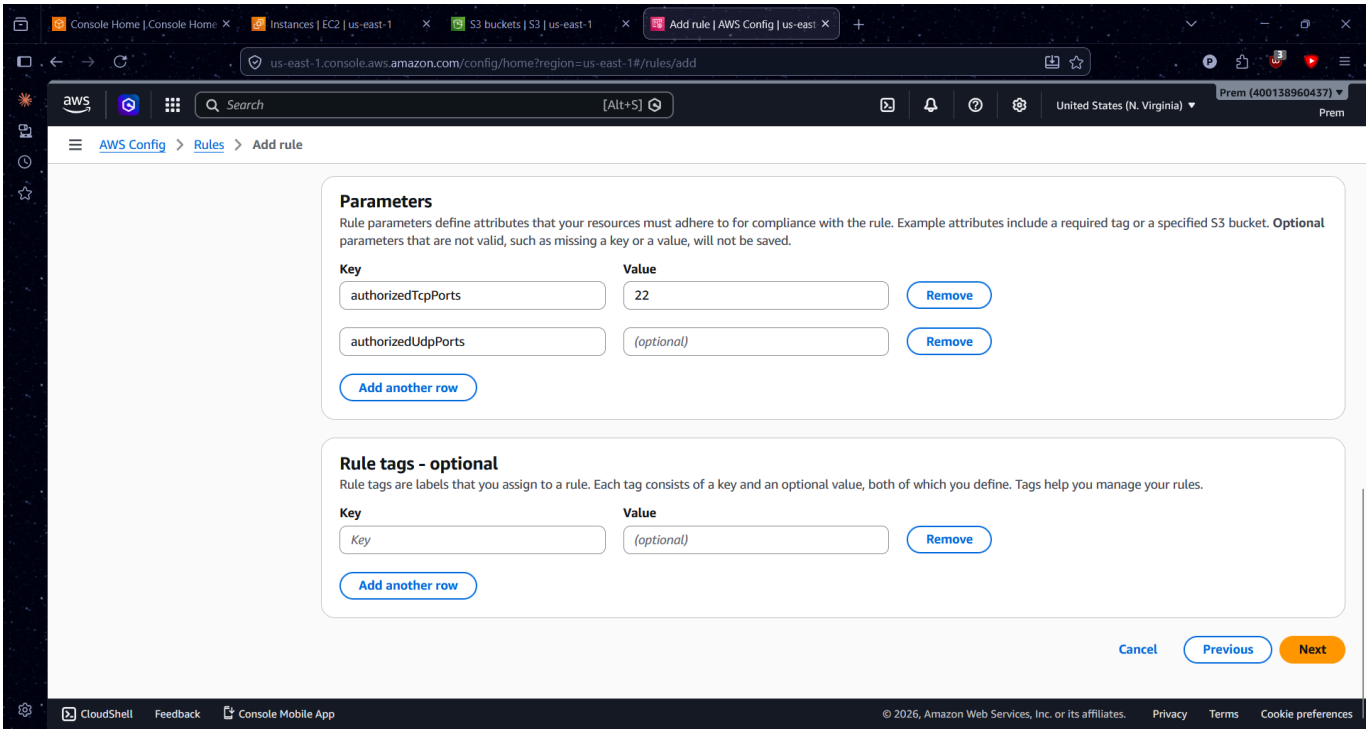
Trigger: When configuration changes
Scope:   EC2 SecurityGroup

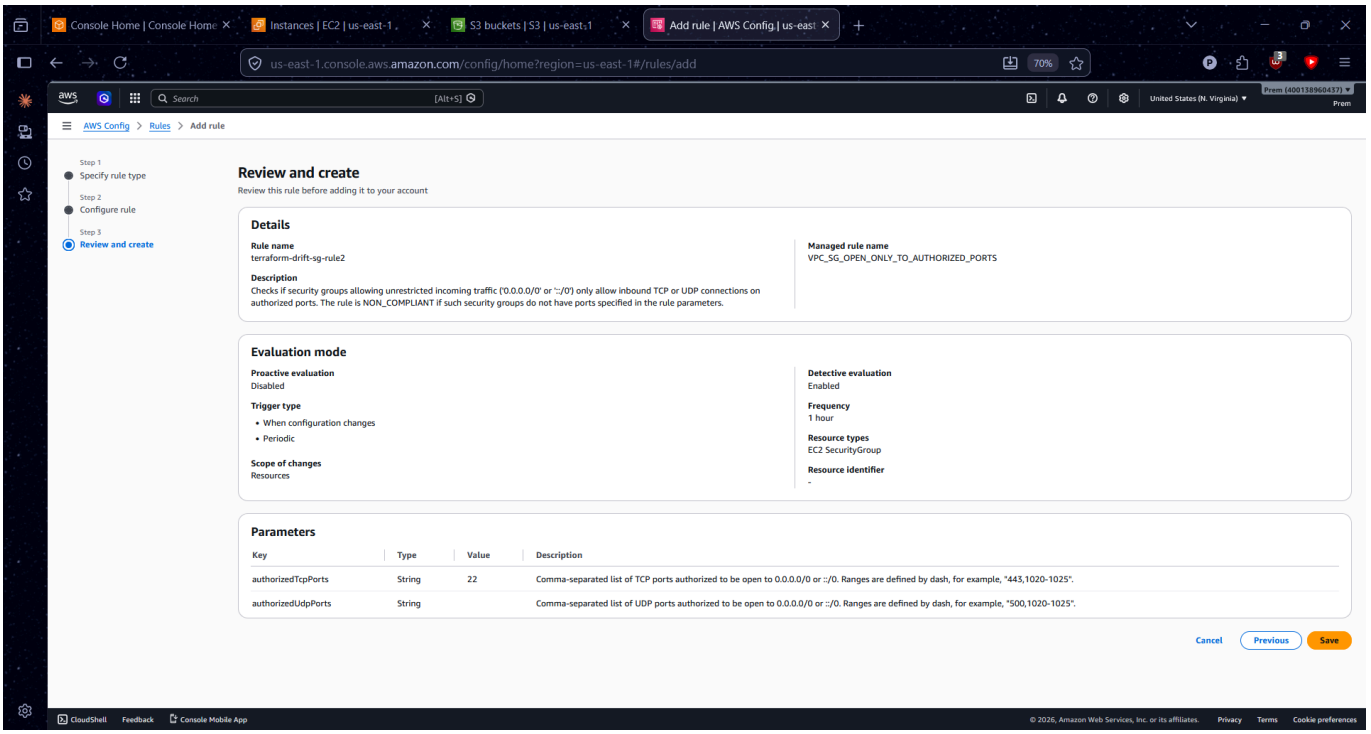
→ Add rule
```

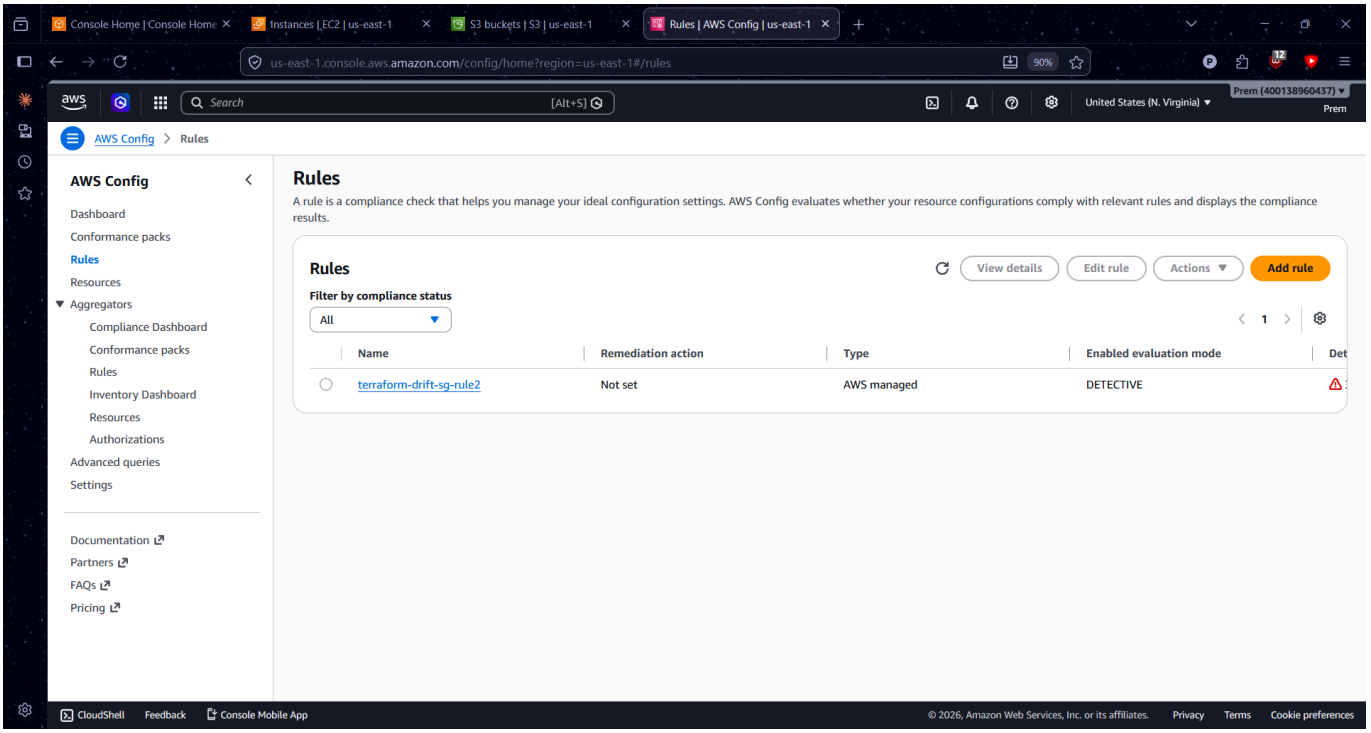

Console Home | Console Home
Instances | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Add rule | AWS Config | us-east
us-east-1.console.aws.amazon.com/config/home?region=us-east-1#/rules/add
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
AWS Config > Rules > Add rule
Step 1
Specify rule type
Step 2
Configure rule
Step 3
Review and create
Specify rule type
Add rules to help you manage the ideal configuration settings of your AWS resources. You can add any of the following predefined, customizable AWS Config Managed rules, or you can create your own AWS Config Custom rule using AWS Lambda functions or Guard Custom policy.
Select rule type
Add AWS managed rule
Deploy the following managed rules in their default state or customize to suit your needs.
Create custom Lambda rule
Use a Lambda function with your custom code to evaluate whether your AWS resources comply with the rule.
Create custom rule using Guard
Use Guard Custom policy that you write to evaluate whether your AWS resources comply with the rule.
AWS Managed Rules (819)
Find Rules
1 match
Name = vpc-sg-open-only-to-authorized-ports
Clear filters
1
Name
Resource types
Trigger type
Description
Supported evaluation mode
Labels
vpc-sg-open-only-to-authorized-ports
AWS::EC2::SecurityGroup
HYBRID
Checks if security groups allowing unrestricted incoming traffic ('0.0.0.0/0' or '::/0') only allow inbound TCP or UDP connections on authorized ports. The rule is NON_COMPLIANT if such security groups do not have ports specified in the rule parameters.
DETECTIVE
VPC, Security Group
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Home
Instances | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Add rule | AWS Config | us-east
us-east-1.console.aws.amazon.com/config/home?region=us-east-1#/rules/add
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
AWS Config > Rules > Add rule
Step 1
Specify rule type
Step 2
Configure rule
Step 3
Review and create
Configure rule
Customize any of the following fields
Details
Name
A unique name for the rule. 128 characters max. No special characters or spaces.
terraform-drift-sg-rule2
Description - optional
Describe what the rule evaluates and how to fix resources that don't comply.
Checks if security groups allowing unrestricted incoming traffic ('0.0.0.0/0' or '::/0') only allow inbound TCP or UDP connections on authorized ports. The rule is NON_COMPLIANT if such security groups do not have ports specified in the rule parameters.
Managed rule name
VPC_SG_OPEN_ONLY_TO_AUTHORIZED_PORTS
Evaluation mode
Turn on proactive evaluation
Enable evaluation of resources prior to provisioning
Turn on detective evaluation
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Home
Instances | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Add rule | AWS Config | us-east
us-east-1.console.aws.amazon.com/config/home?region=us-east-1#/rules/add
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
AWS Config > Rules > Add rule
When configuration changes
Runs when there are changes to your specified AWS resources
Periodic
Runs on the frequency that you choose
Scope of changes
Choose when evaluations will occur.
All changes
When any resource recorded by AWS Config is created, changed, or deleted
Resources
When any resource that matches the specified type, or the type plus identifier, is created, changed, or deleted
Tags
When any resource with the specified tag is created, changed, or deleted
Resources
This rule can be triggered only when the recorded resources are created, edited, or deleted. Specify the resources to record by editing the Settings page.
Resource category
All resource categories
Resource type
Multiple selected
AWS EC2 SecurityGroup
Resource identifier - optional
Enter resource identifier
Frequency
1 hour
Parameters
Rule parameters define attributes that your resources must adhere to for compliance with the rule. Example attributes include a required tag or a specified S3 bucket. Optional
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Home
Instances | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Add rule | AWS Config | us-east
us-east-1.console.aws.amazon.com/config/home?region=us-east-1#/rules/add
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
AWS Config > Rules > Add rule
Parameters
Rule parameters define attributes that your resources must adhere to for compliance with the rule. Example attributes include a required tag or a specified S3 bucket. Optional parameters that are not valid, such as missing a key or a value, will not be saved.
Key
Value
authorizedTcpPorts
22
Remove
authorizedUdpPorts
(optional)
Remove
Add another row
Rule tags - optional
Rule tags are labels that you assign to a rule. Each tag consists of a key and an optional value, both of which you define. Tags help you manage your rules.
Key
Value
Key
(optional)
Remove
Add another row
Cancel
Previous
Next
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences





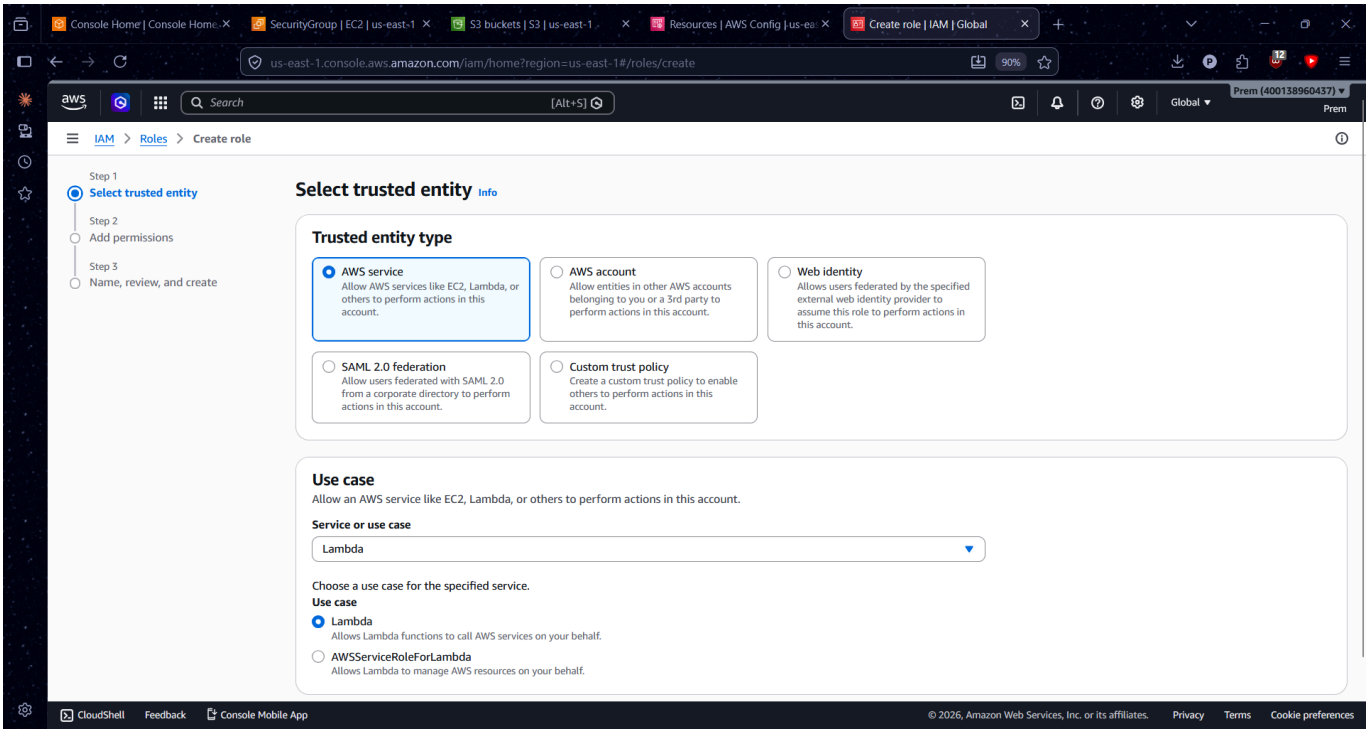
This rule is simple but powerful — it marks any Security Group NON_COMPLIANT the moment a port other than 22 has an inbound rule. The moment port 80 was added in Phase 2, this rule will fire.

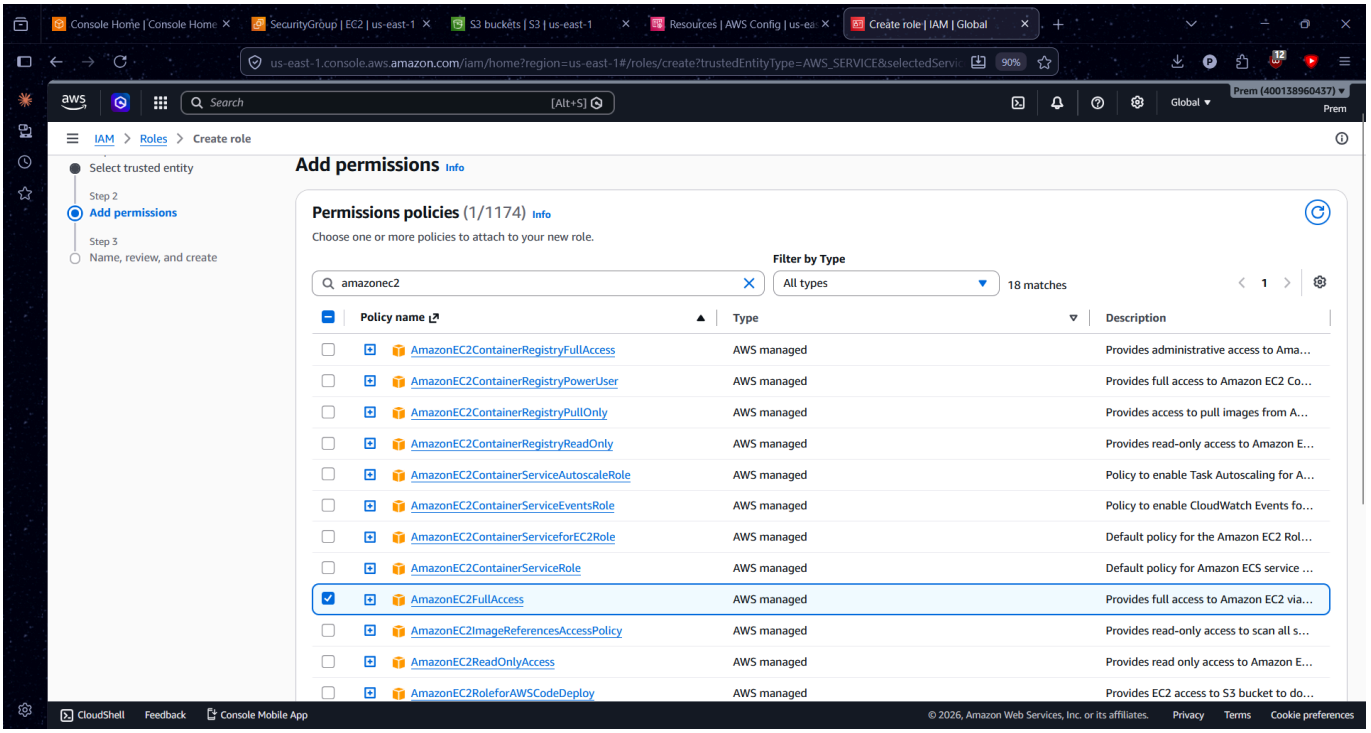
IAM Role for Lambda

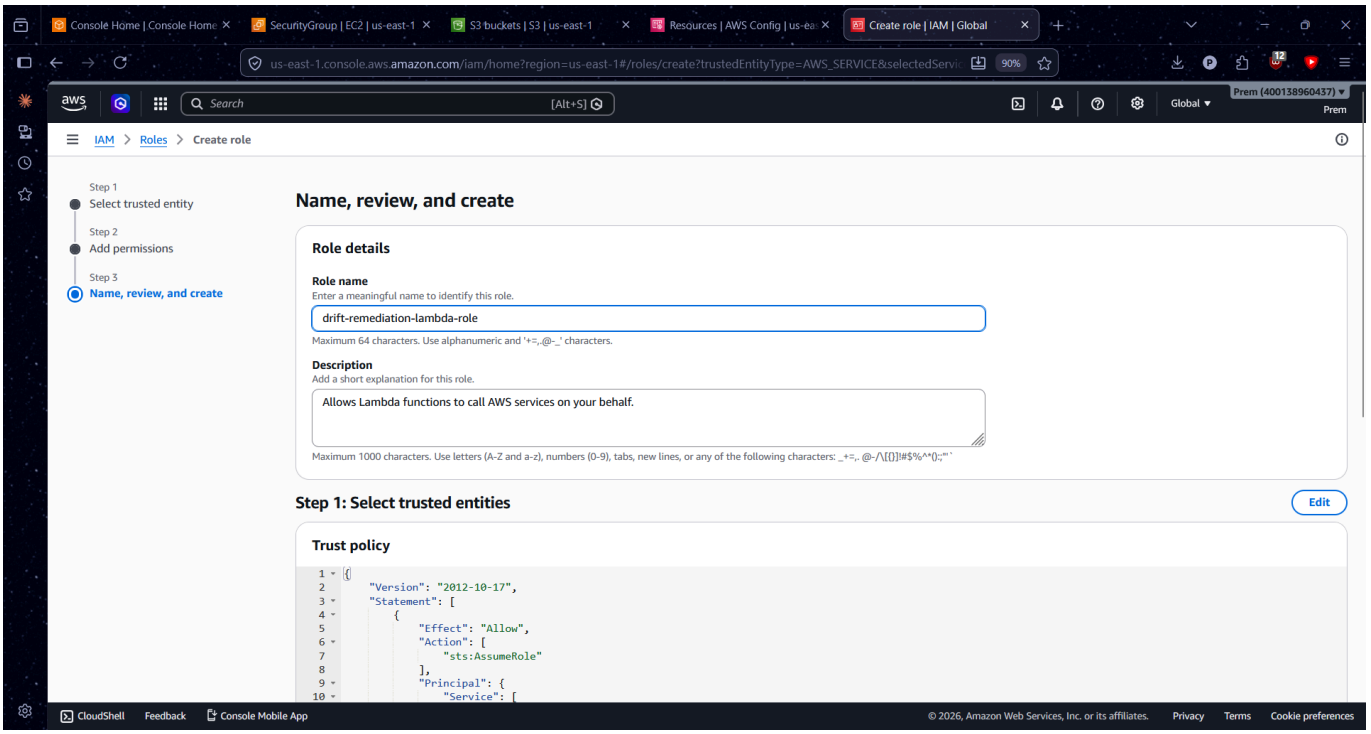
```
IAM → Roles → Create role

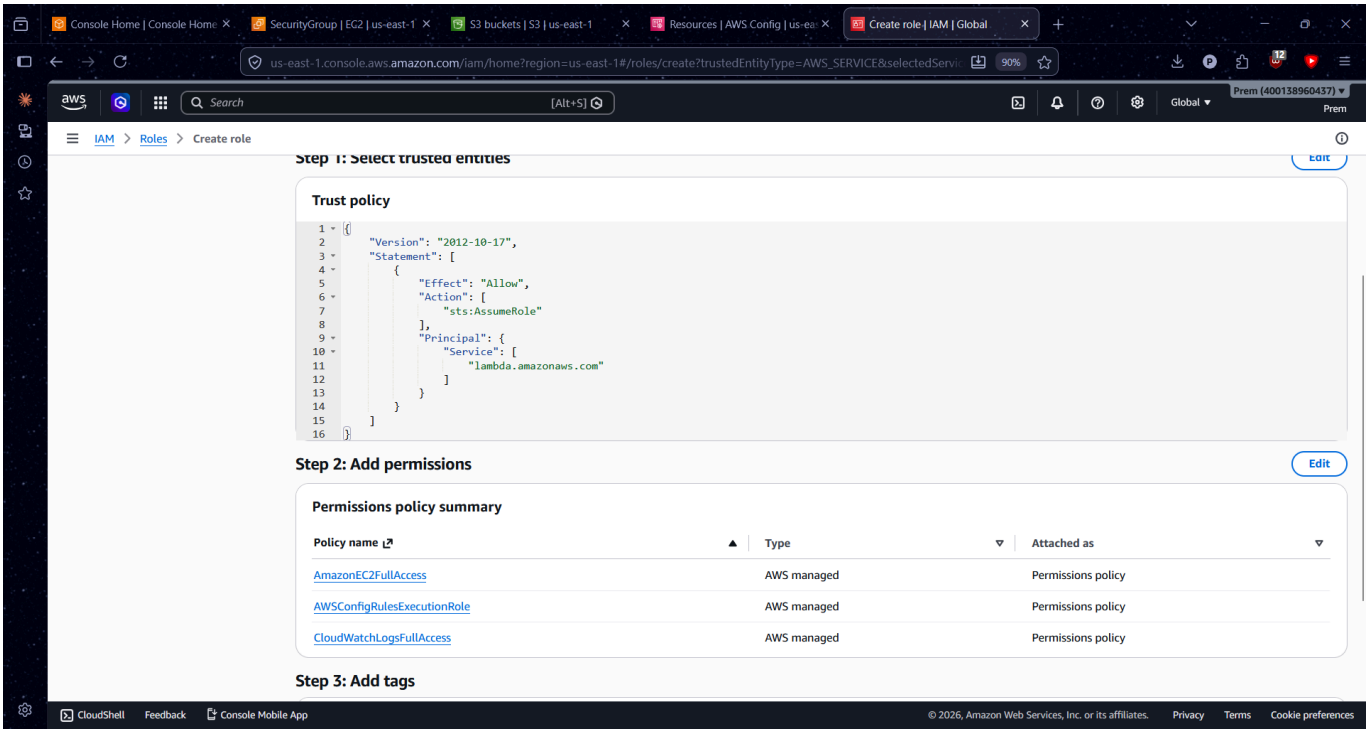
Trusted entity: Lambda
Policies:
  ✓ AWSConfigRulesExecutionRole
  ✓ AmazonEC2FullAccess
  ✓ CloudWatchLogsFullAccess
```

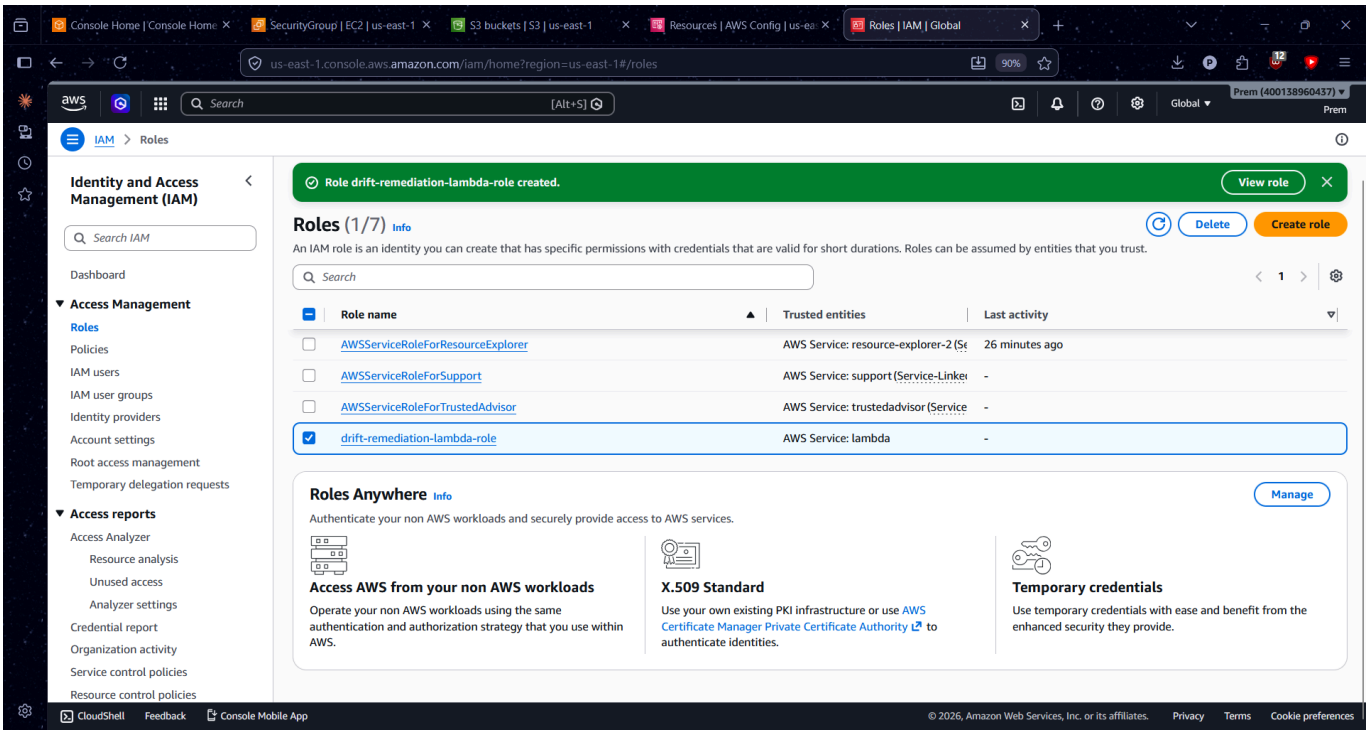
```
Role name: drift-remediation-lambda-role
```



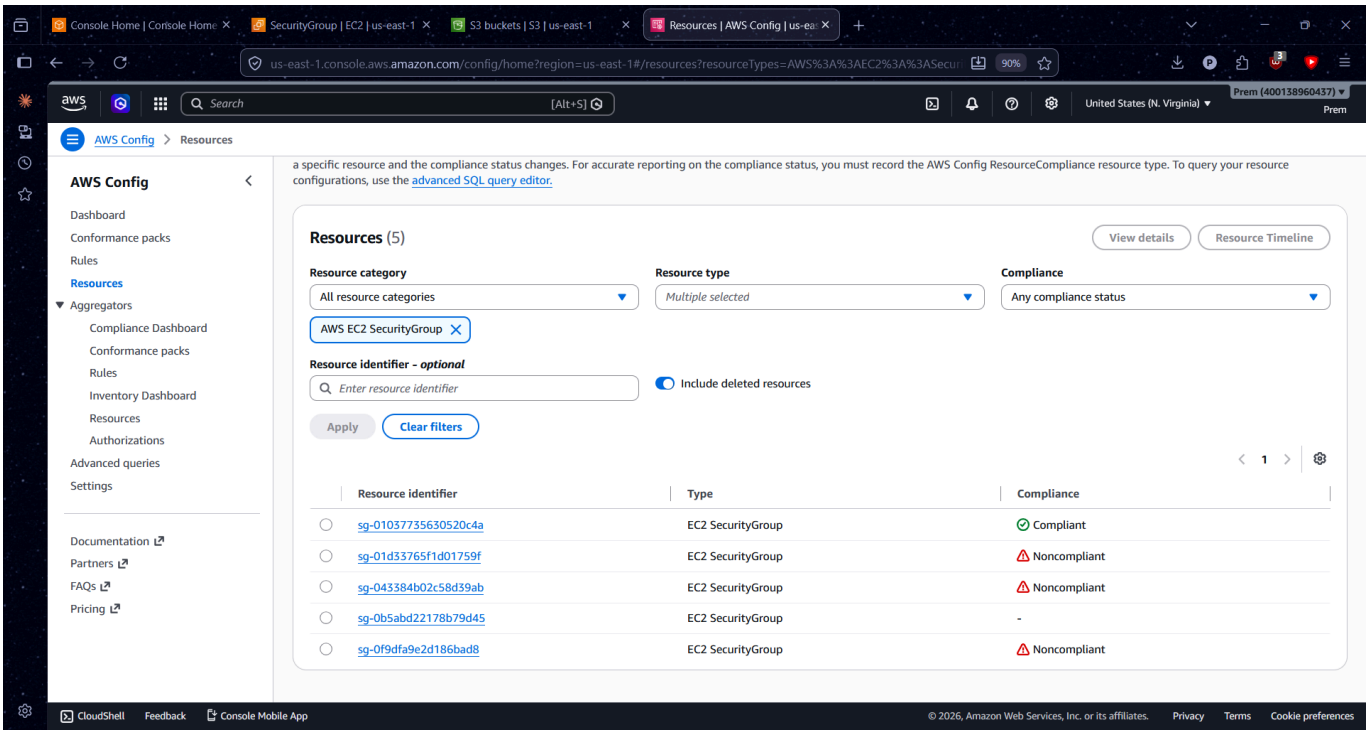


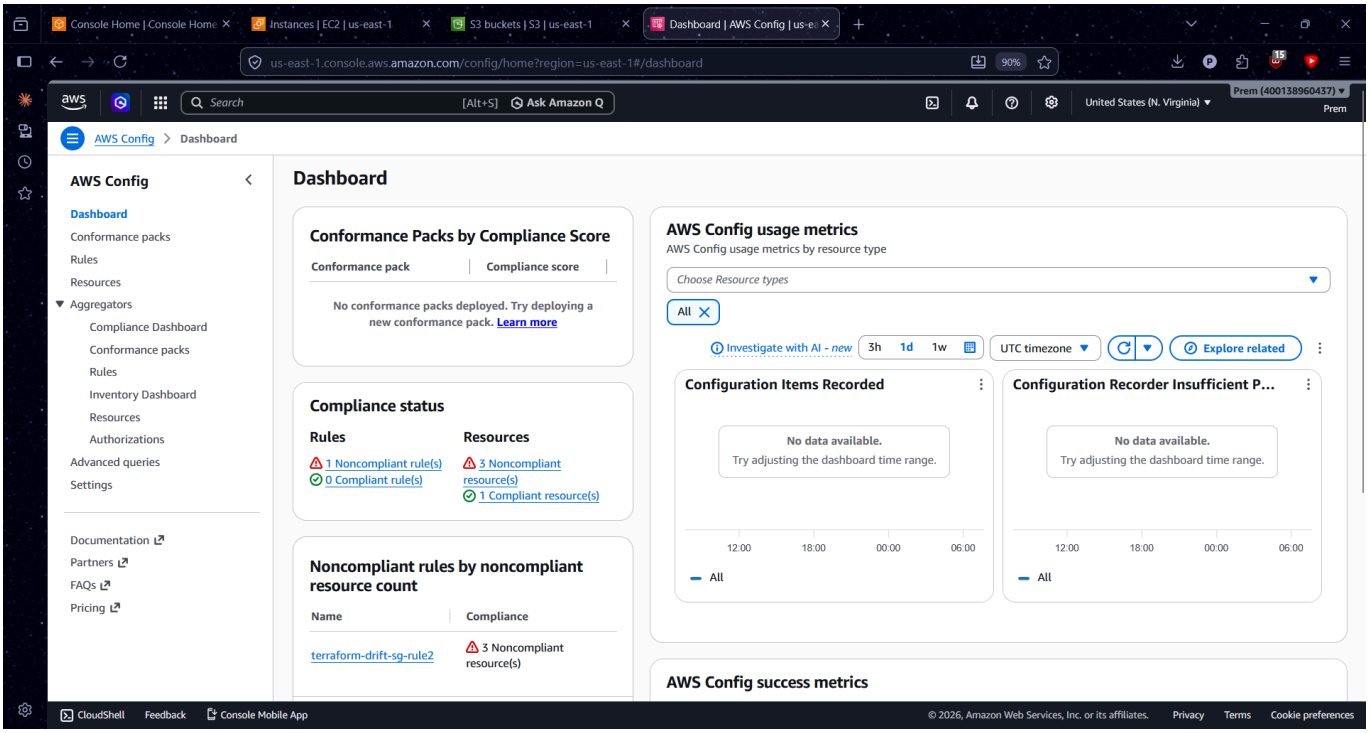

Console Home | Console Home
SecurityGroup | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Resources | AWS Config | us-ea
Create role | IAM | Global
us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/create?trustedEntityType=AWS_SERVICE&selectedServi
90%
Search
[Alt+S]
Global
Prem (400138960437)
Prem
IAM > Roles > Create role
Step 1
Select trusted entity
Step 2
Add permissions
Step 3
Name, review, and create
Name, review, and create
Role details
Role name
Enter a meaningful name to identify this role.
drift-remediation-lambda-role
Maximum 64 characters. Use alphanumeric and '+=,.@-_' characters.
Description
Add a short explanation for this role.
Allows Lambda functions to call AWS services on your behalf.
Maximum 1000 characters. Use letters (A-Z and a-z), numbers (0-9), tabs, new lines, or any of the following characters: _+=,. @-/\[{}]!#$%^*0;"'`
Step 1: Select trusted entities
Edit
Trust policy
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": [
"sts:AssumeRole"
],
"Principal": {
"Service": [
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Home
SecurityGroup | EC2 | us-east-1
S3 buckets | S3 | us-east-1
Resources | AWS Config | us-ea
Create role | IAM | Global
us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/create?trustedEntityType=AWS_SERVICE&selectedServi
90%
Search
[Alt+S]
Global
Prem (400138960437)
Prem
IAM > Roles > Create role
Trust policy
{
"Version": "2012-10-17",
"Statement": [
{
"Effect": "Allow",
"Action": [
"sts:AssumeRole"
],
"Principal": {
"Service": [
"lambda.amazonaws.com"
]
}
}
]
}
Step 2: Add permissions
Edit
Permissions policy summary
Policy name
Type
Attached as
AmazonEC2FullAccess
AWS managed
Permissions policy
AWSConfigRulesExecutionRole
AWS managed
Permissions policy
CloudWatchLogsFullAccess
AWS managed
Permissions policy
Step 3: Add tags
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



> ⚠️ `AmazonEC2FullAccess` is used here for lab simplicity. In production this would be scoped down to only `ec2:DescribeSecurityGroups`, `ec2:RevokeSecurityGroupIngress`, `ec2:DescribeInstances`, `ec2:CreateTags`, and `ec2:DeleteTags`. Always apply least privilege before production deployment.





Phase 4 — Lambda Auto-Remediation Function

This is the core of the project. When EventBridge fires the Lambda, it figures out exactly what drifted and uses boto3 to put it back the way Terraform defined it.

One design decision worth calling out — the `is_rule_approved()` function validates port, protocol, AND source CIDR together. An earlier version only checked port numbers, which would incorrectly approve `port 22 open to 0.0.0.0/0` because the port matched. That is a worse security posture than the drift it was trying to fix.

lambda_function.py

```
import boto3
import json
import logging

logger = logging.getLogger()
logger.setLevel(logging.INFO)

ec2 = boto3.client('ec2')

# Approved rules — port, protocol AND cidr must all match
APPROVED_RULES = [
    {
        'port':     22,
        'protocol': 'tcp',
        'cidr':     'YOUR_IP/32'
    }
]

APPROVED_TAGS = {
    'Name':        'drift-demo-ec2',
    'ManagedBy':   'Terraform',
    'Environment': 'lab'
}


def lambda_handler(event, context):
    try:
        logger.info(f"Event received: {json.dumps(event)}")

        detail        = event.get('detail', {})
        resource_id   = detail.get('resourceId', '')
        resource_type = detail.get('resourceType', '')

        compliance = (
            detail.get('newEvaluationResult', {})
                  .get('complianceType', '')
        )

        logger.info(f"Resource: {resource_type} | ID: {resource_id}")
        logger.info(f"Compliance status: {compliance}")

        if compliance != 'NON_COMPLIANT':
            logger.info("Resource is compliant. No action needed.")
            return {'statusCode': 200, 'body': 'No action needed'}

        if resource_type == 'AWS::EC2::SecurityGroup':
            remediate_security_group(resource_id)
        elif resource_type == 'AWS::EC2::Instance':
            remediate_ec2_tags(resource_id)
        else:
            logger.info(f"No handler for resource type: {resource_type}")
```

```
        return {'statusCode': 200, 'body': 'Remediation complete'}

    except Exception as e:
        logger.error(f"Lambda handler failed: {str(e)}")
        raise


def is_rule_approved(rule):
    """
    Validates port, protocol AND cidr — not just port.
    A rule is only approved when ALL three match an approved entry.
    This prevents port 22 open to 0.0.0.0/0 from passing as approved.
    """
    from_port = rule.get('FromPort', -1)
    to_port   = rule.get('ToPort', -1)
    protocol  = rule.get('IpProtocol', '').lower()
    cidrs     = [ip.get('CidrIp', '') for ip in rule.get('IpRanges', [])]

    for approved in APPROVED_RULES:
        port_ok     = (from_port == approved['port'] and
                       to_port   == approved['port'])
        protocol_ok = (protocol  == approved['protocol'])
        cidr_ok     = (approved['cidr'] in cidrs)

        if port_ok and protocol_ok and cidr_ok:
            return True

    return False


def remediate_security_group(sg_id):
    try:
        logger.info(f"Checking SG inbound rules: {sg_id}")
        response = ec2.describe_security_groups(GroupIds=[sg_id])
        sg = response['SecurityGroups'][0]
        current_rules = sg['IpPermissions']

        unauthorized = []
        for rule in current_rules:
            if not is_rule_approved(rule):
                from_port = rule.get('FromPort', 'ALL')
                cidrs = [ip.get('CidrIp') for ip in rule.get('IpRanges', [])]
                unauthorized.append(rule)
                logger.warning(
                    f"DRIFT DETECTED: Unauthorized rule — "
                    f"port={from_port}, cidrs={cidrs} on {sg_id}"
                )

        if unauthorized:
            ec2.revoke_security_group_ingress(
                GroupId=sg_id,
                IpPermissions=unauthorized
            )
            logger.info(
```

```
                f"REMEDIATED: Removed {len(unauthorized)} "
                f"unauthorized inbound rules from {sg_id}"
            )
        else:
            logger.info(f"All inbound rules are approved on {sg_id}")

    except Exception as e:
        logger.error(f"Error in remediate_security_group: {e}")
        raise


def remediate_ec2_tags(instance_id):
    try:
        logger.info(f"Checking EC2 tags: {instance_id}")
        response = ec2.describe_instances(InstanceIds=[instance_id])
        instance = response['Reservations'][0]['Instances'][0]
        current_tags = {
            t['Key']: t['Value']
            for t in instance.get('Tags', [])
        }

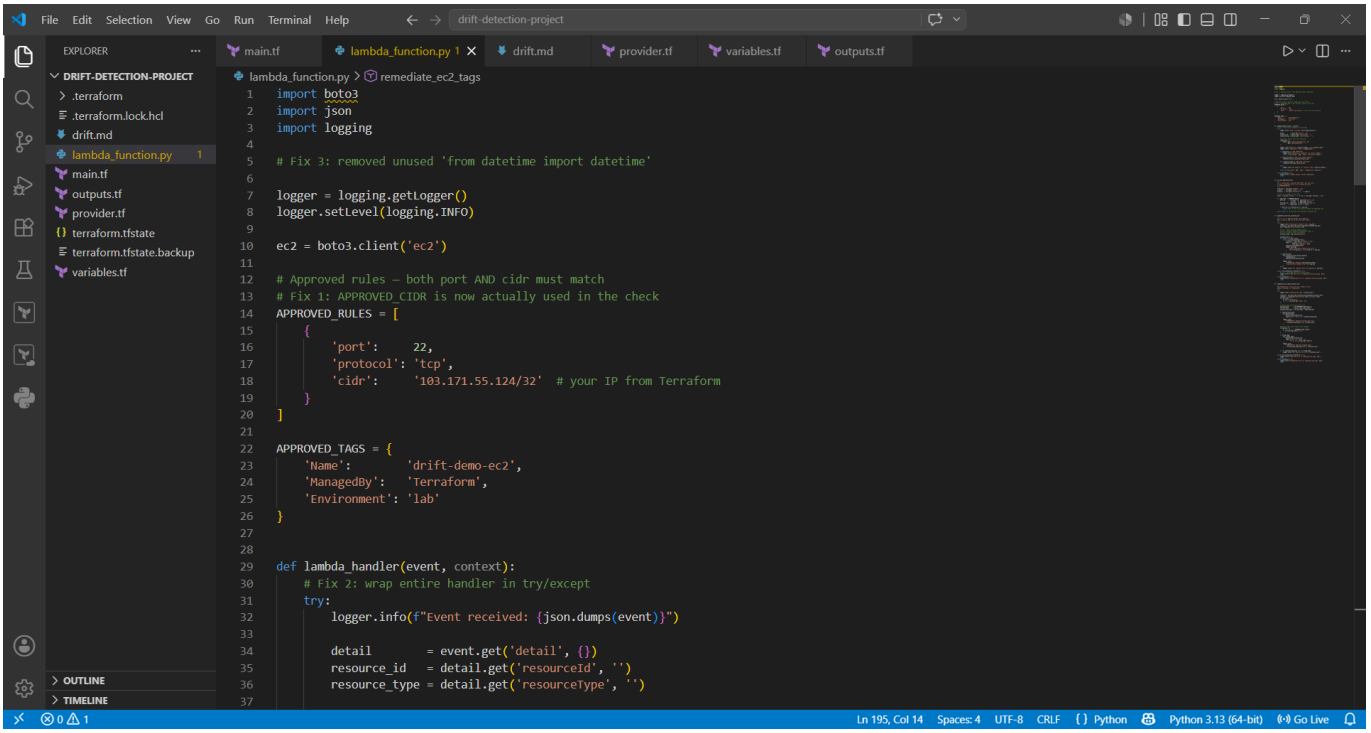
        approved_keys     = set(APPROVED_TAGS.keys())
        current_keys      = set(current_tags.keys())
        unauthorized_keys = current_keys - approved_keys

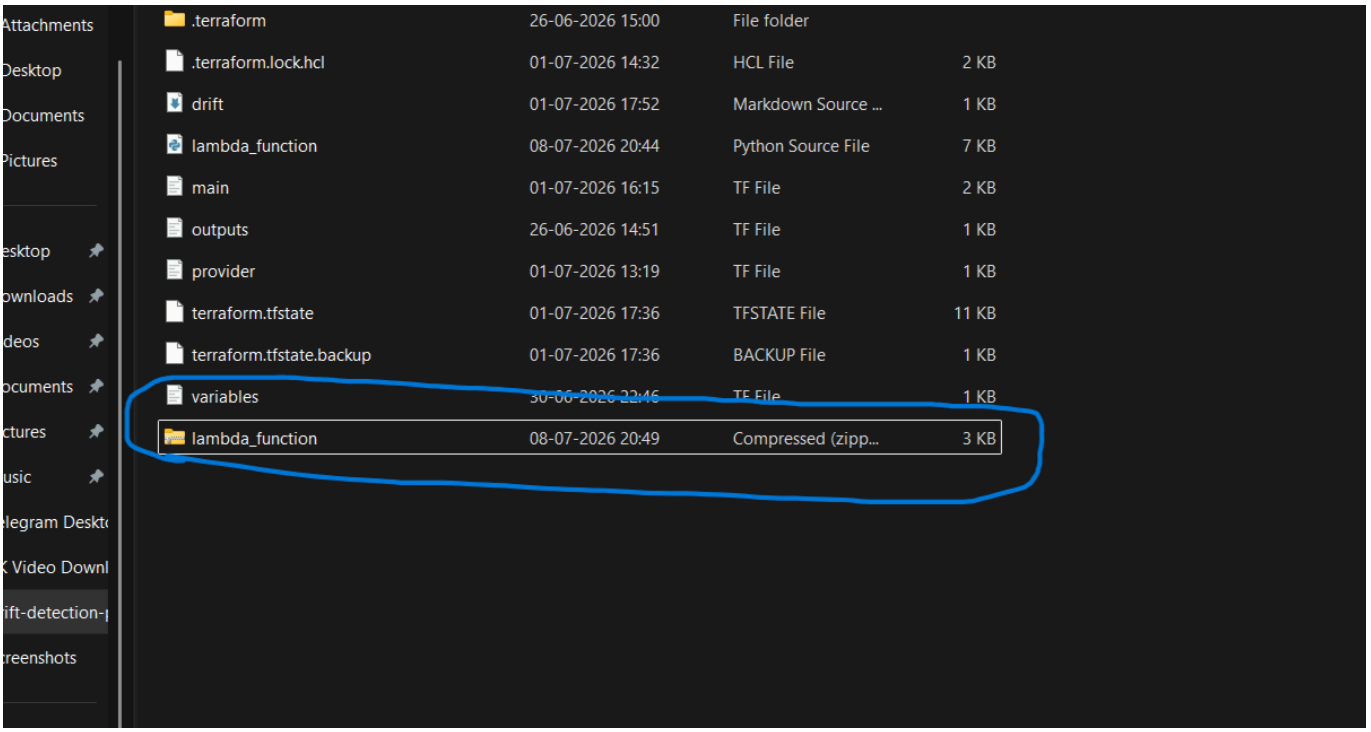
        if unauthorized_keys:
            ec2.delete_tags(
                Resources=[instance_id],
                Tags=[{'Key': k} for k in unauthorized_keys]
            )
            logger.info(
                f"REMEDIATED: Removed unauthorized tags "
                f"{unauthorized_keys} from {instance_id}"
            )

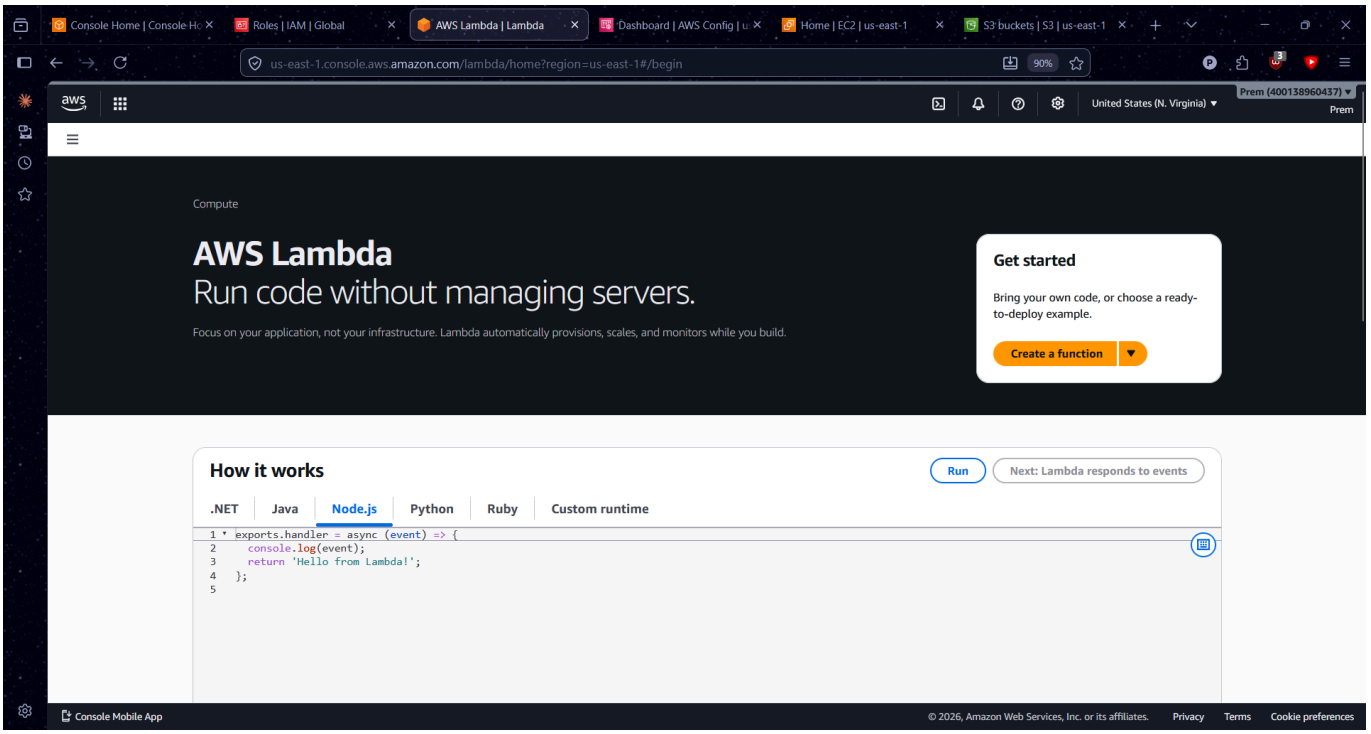
        wrong_tags = {
            k: v for k, v in APPROVED_TAGS.items()
            if current_tags.get(k) != v
        }

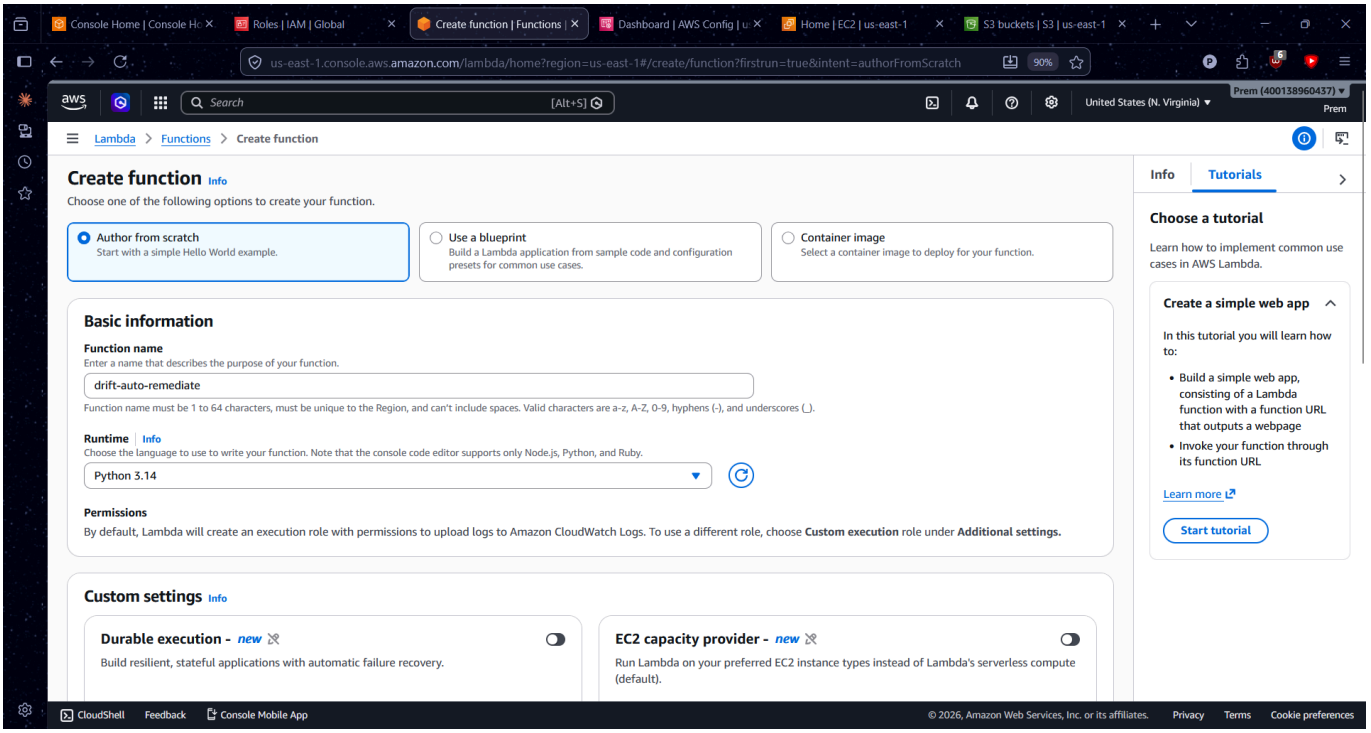
        if wrong_tags:
            ec2.create_tags(
                Resources=[instance_id],
                Tags=[{'Key': k, 'Value': v} for k, v in wrong_tags.items()]
            )
            logger.info(
                f"REMEDIATED: Restored changed tags "
                f"{list(wrong_tags.keys())} on {instance_id}"
            )

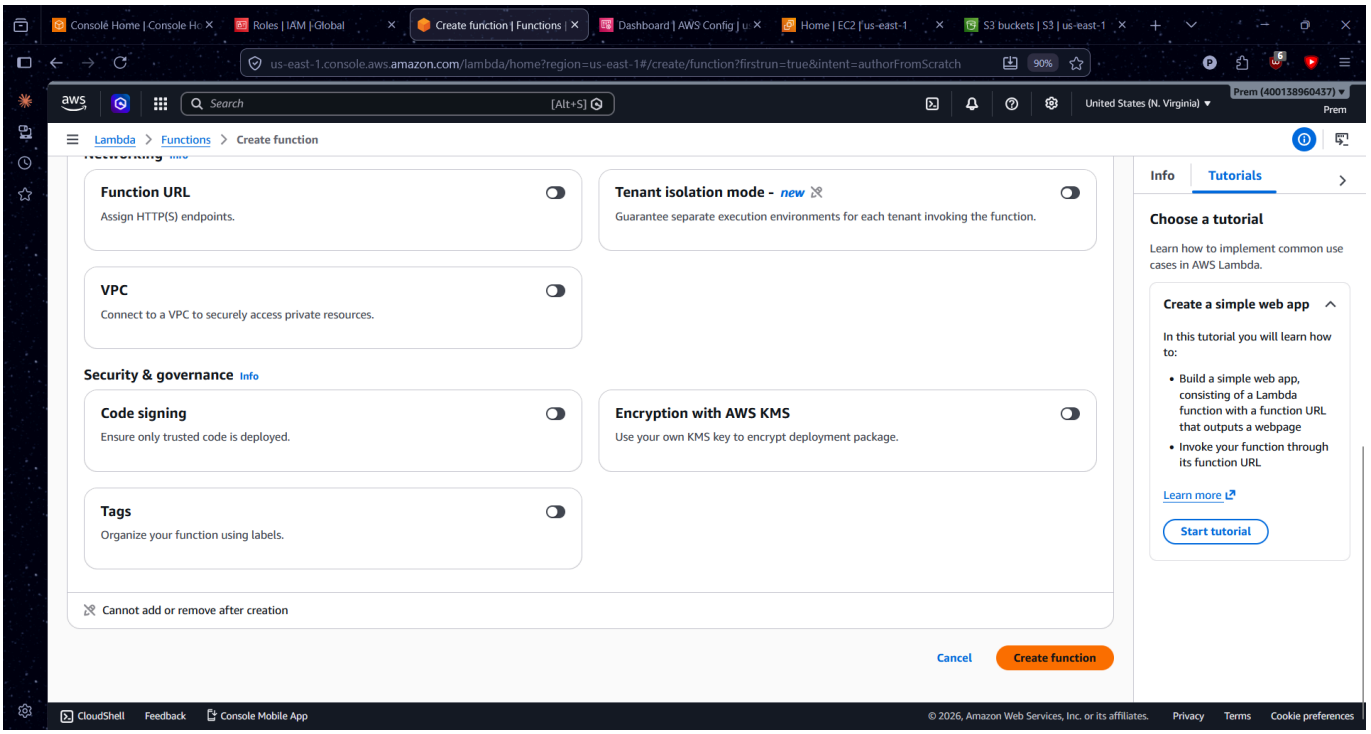
    except Exception as e:
        logger.error(f"Error in remediate_ec2_tags: {e}")
        raise
```

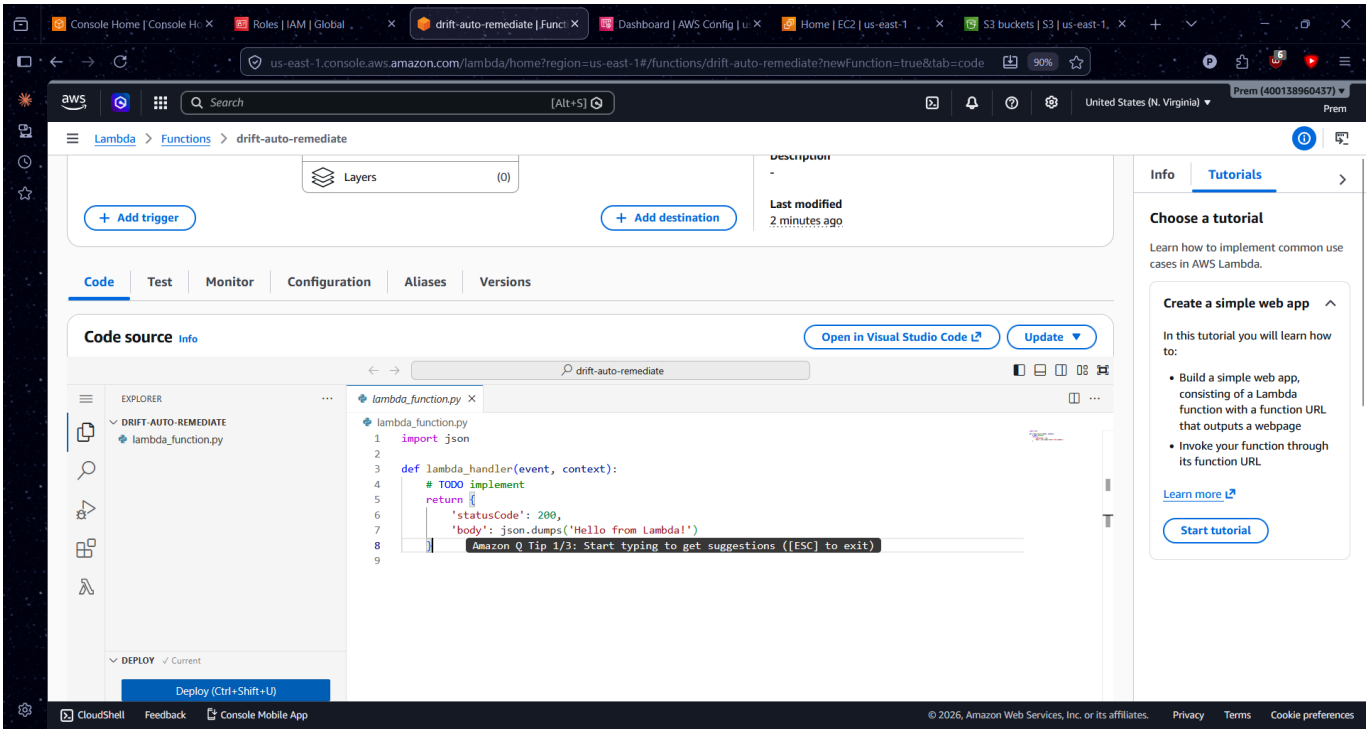

File Edit Selection View Go Run Terminal Help
drift-detection-project
EXPLORER
DRIFT-DETECTION-PROJECT
.terraform
.terraform.lock.hcl
drift.md
lambda_function.py 1
main.tf
outputs.tf
provider.tf
terraform.tfstate
terraform.tfstate.backup
variables.tf
main.tf
lambda_function.py 1
drift.md
provider.tf
variables.tf
outputs.tf
lambda_function.py > remediate_ec2_tags
import boto3
import json
import logging
Fix 3: removed unused 'from datetime import datetime'
logger = logging.getLogger()
logger.setLevel(logging.INFO)
ec2 = boto3.client('ec2')
Approved rules — both port AND cidr must match
Fix 1: APPROVED_CIDR is now actually used in the check
APPROVED_RULES = [
{
'port': 22,
'protocol': 'tcp',
'cidr': '103.171.55.124/32' # your IP from Terraform
}
]
APPROVED_TAGS = {
'Name': 'drift-demo-ec2',
'ManagedBy': 'Terraform',
'Environment': 'lab'
}
def lambda_handler(event, context):
Fix 2: wrap entire handler in try/except
try:
logger.info(f"Event received: {json.dumps(event)}")
detail = event.get('detail', {})
resource_id = detail.get('resourceId', '')
resource_type = detail.get('resourceType', '')
OUTLINE
TIMELINE
Ln 195, Col 14 Spaces: 4 UTF-8 CRLF Python Python 3.13 (64-bit) Go Live

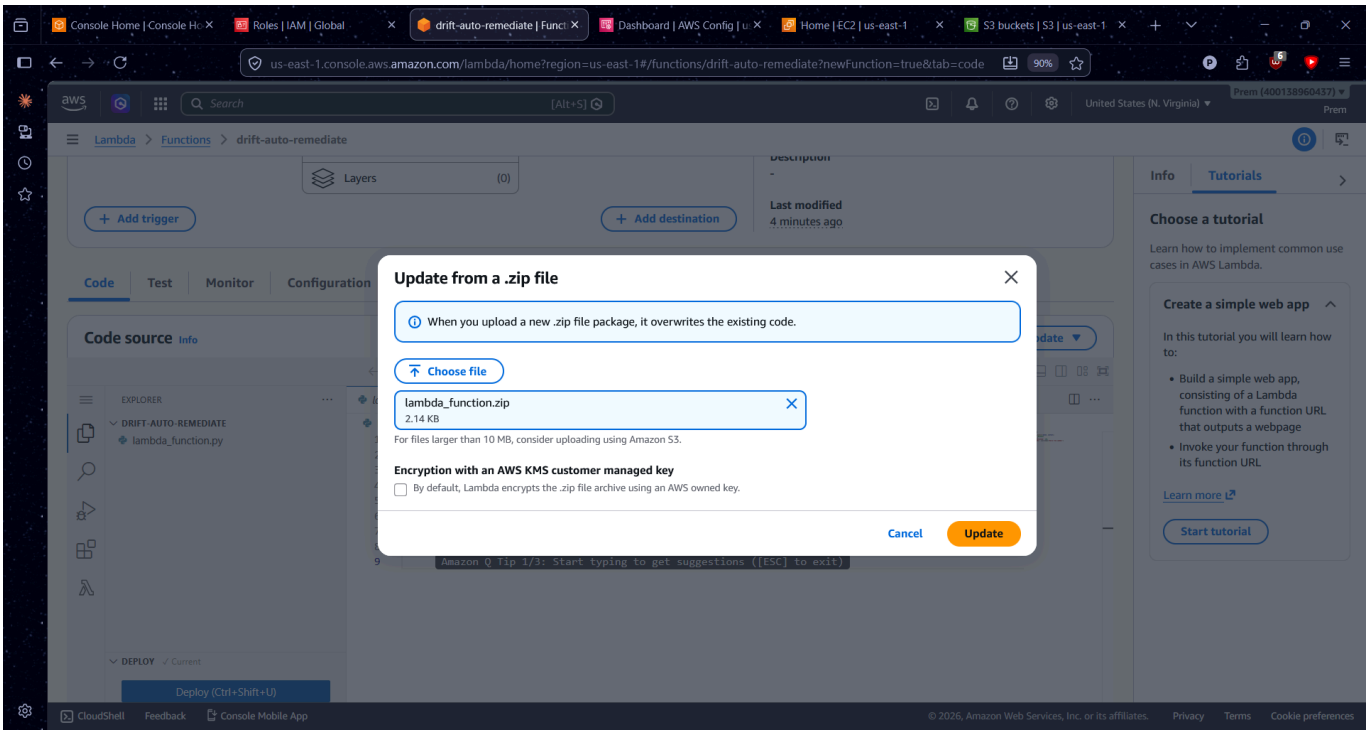

Attachments
Desktop
Documents
Pictures
esktop
ownloads
deos
ocuments
ctures
usic
legram Deskt
Video Downl
ift-detection-p
reenshots
.terraform 26-06-2026 15:00 File folder
.terraform.lock.hcl 01-07-2026 14:32 HCL File 2 KB
drift 01-07-2026 17:52 Markdown Source ... 1 KB
lambda_function 08-07-2026 20:44 Python Source File 7 KB
main 01-07-2026 16:15 TF File 2 KB
outputs 26-06-2026 14:51 TF File 1 KB
provider 01-07-2026 13:19 TF File 1 KB
terraform.tfstate 01-07-2026 17:36 TFSTATE File 11 KB
terraform.tfstate.backup 01-07-2026 17:36 BACKUP File 1 KB
variables 30-06-2026 22:46 TF File 1 KB
lambda_function 08-07-2026 20:49 Compressed (zipp... 3 KB


Console Home | Console Hc
Roles | IAM | Global
AWS Lambda | Lambda
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/begin
United States (N. Virginia)
Prem (400138960437)
Prem
Compute
AWS Lambda
Run code without managing servers.
Focus on your application, not your infrastructure. Lambda automatically provisions, scales, and monitors while you build.
Get started
Bring your own code, or choose a ready-to-deploy example.
Create a function
How it works
Run
Next: Lambda responds to events
.NET
Java
Node.js
Python
Ruby
Custom runtime
1 * exports.handler = async (event) => {
2 console.log(event);
3 return 'Hello from Lambda!';
4 };
5
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Hc
Roles | IAM | Global
Create function | Functions
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/create/function?firstrun=true&intent=authorFromScratch
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > Create function
Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
drift-auto-remediate
Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).
Runtime Info
Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.
Python 3.14
Permissions
By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. To use a different role, choose Custom execution role under Additional settings.
Custom settings Info
Durable execution - new
Build resilient, stateful applications with automatic failure recovery.
EC2 capacity provider - new
Run Lambda on your preferred EC2 instance types instead of Lambda's serverless compute (default).
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Create function | Functions
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/create/function?firstrun=true&intent=authorFromScratch
Search
[Alt+S]
United States (N. Virginia)
Lambda > Functions > Create function
Function URL
Assign HTTP(S) endpoints.
Tenant isolation mode - new
Guarantee separate execution environments for each tenant invoking the function.
VPC
Connect to a VPC to securely access private resources.
Security & governance Info
Code signing
Ensure only trusted code is deployed.
Encryption with AWS KMS
Use your own KMS key to encrypt deployment package.
Tags
Organize your function using labels.
Cannot add or remove after creation
Cancel
Create function
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


drift-auto-remediate | Funct
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?newFunction=true&tab=code
Lambda > Functions > drift-auto-remediate
Layers (0)
Add trigger
Add destination
Last modified
2 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Code source Info
Open in Visual Studio Code
Update
drift-auto-remediate
EXPLORER
DRIFT-AUTO-REMEDIATE
lambda_function.py
import json
def lambda_handler(event, context):
TODO implement
return {
'statusCode': 200,
'body': json.dumps('Hello from Lambda!')
}
Amazon Q Tip 1/3: Start typing to get suggestions ([ESC] to exit)
DEPLOY
Deploy (Ctrl+Shift+U)
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

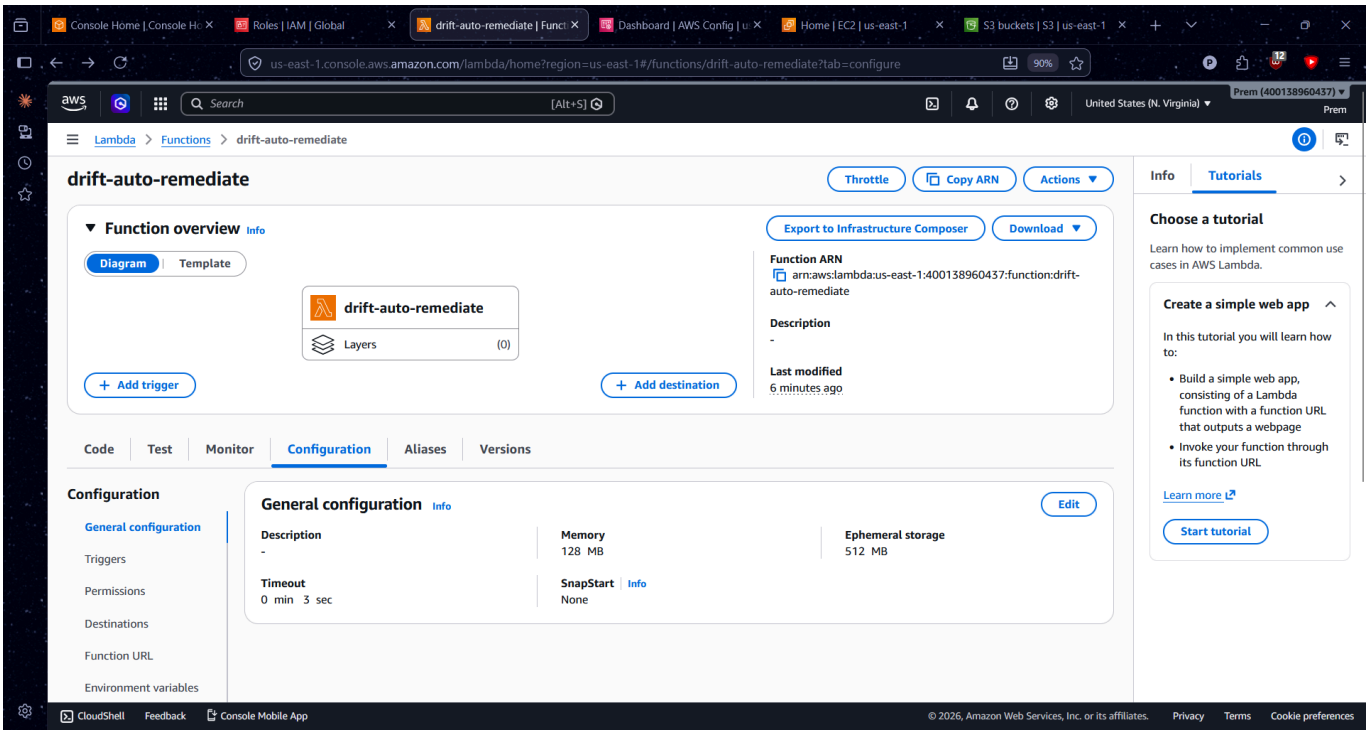


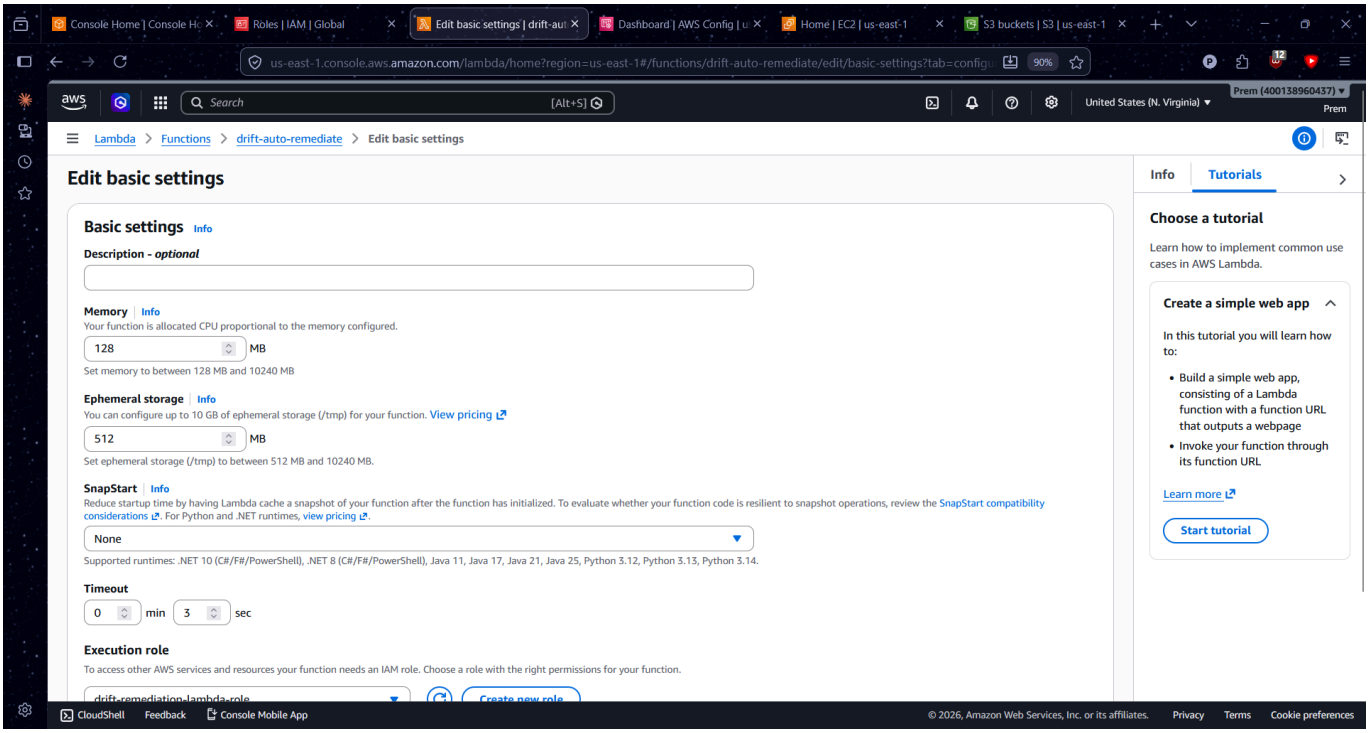
Deploy Lambda

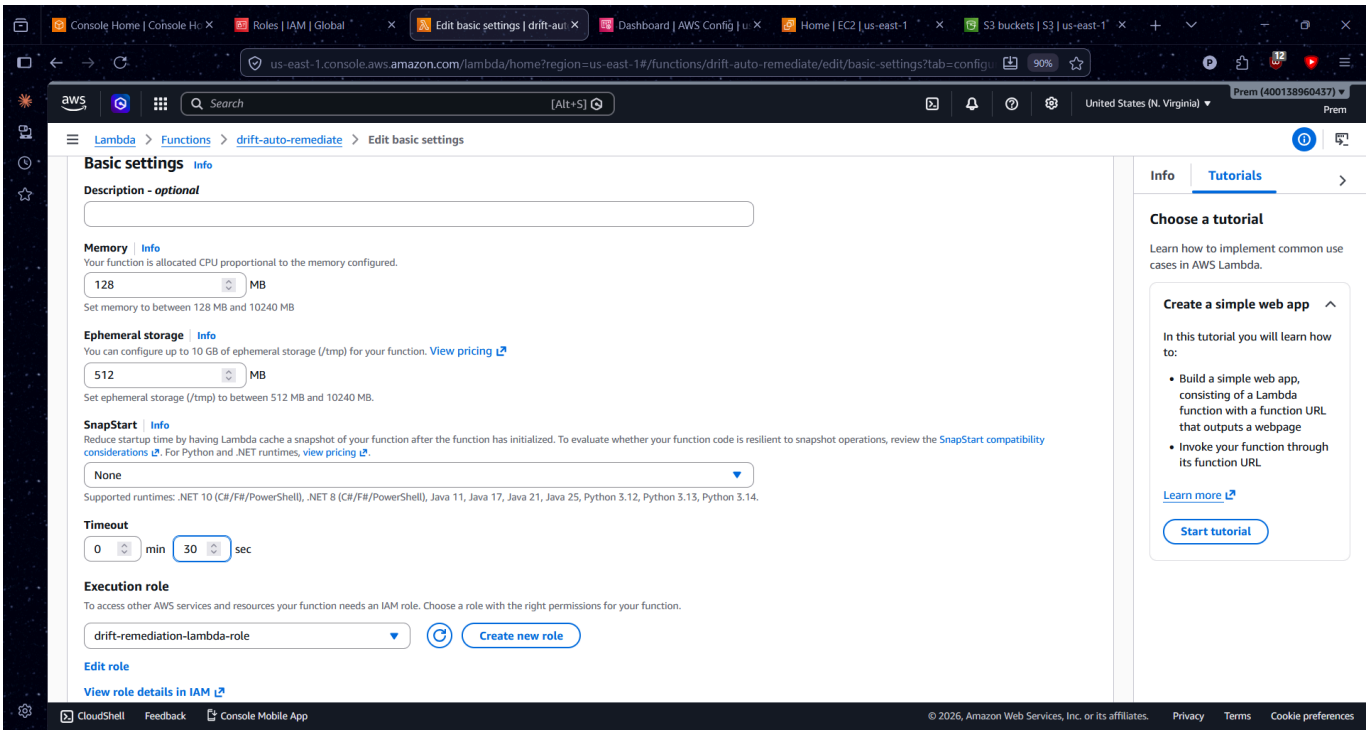
```
zip function.zip lambda_function.py

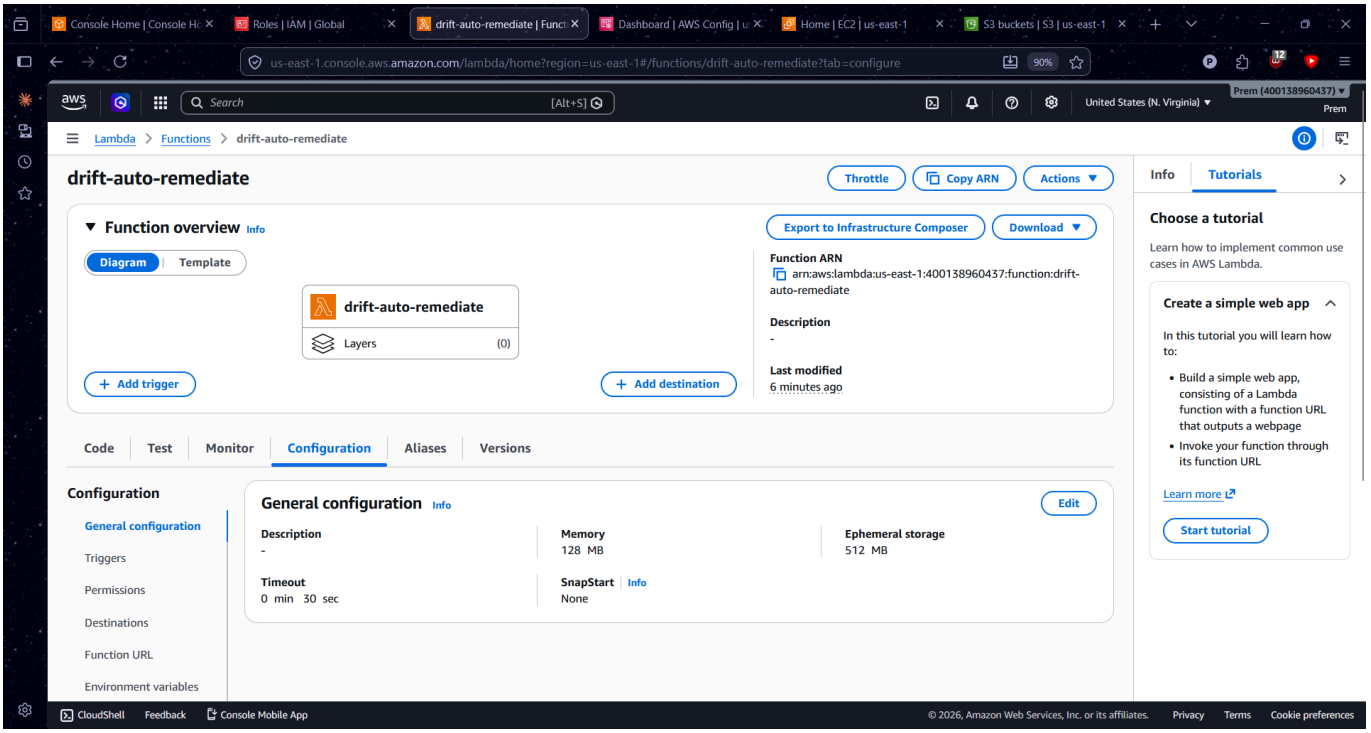
# AWS Console:
Lambda → Create function
  Function name:    drift-auto-remediate
  Runtime:          Python 3.12
  Execution role:   Use existing → drift-remediation-lambda-role
  Timeout:          30 seconds   ← important, default 3s is too short

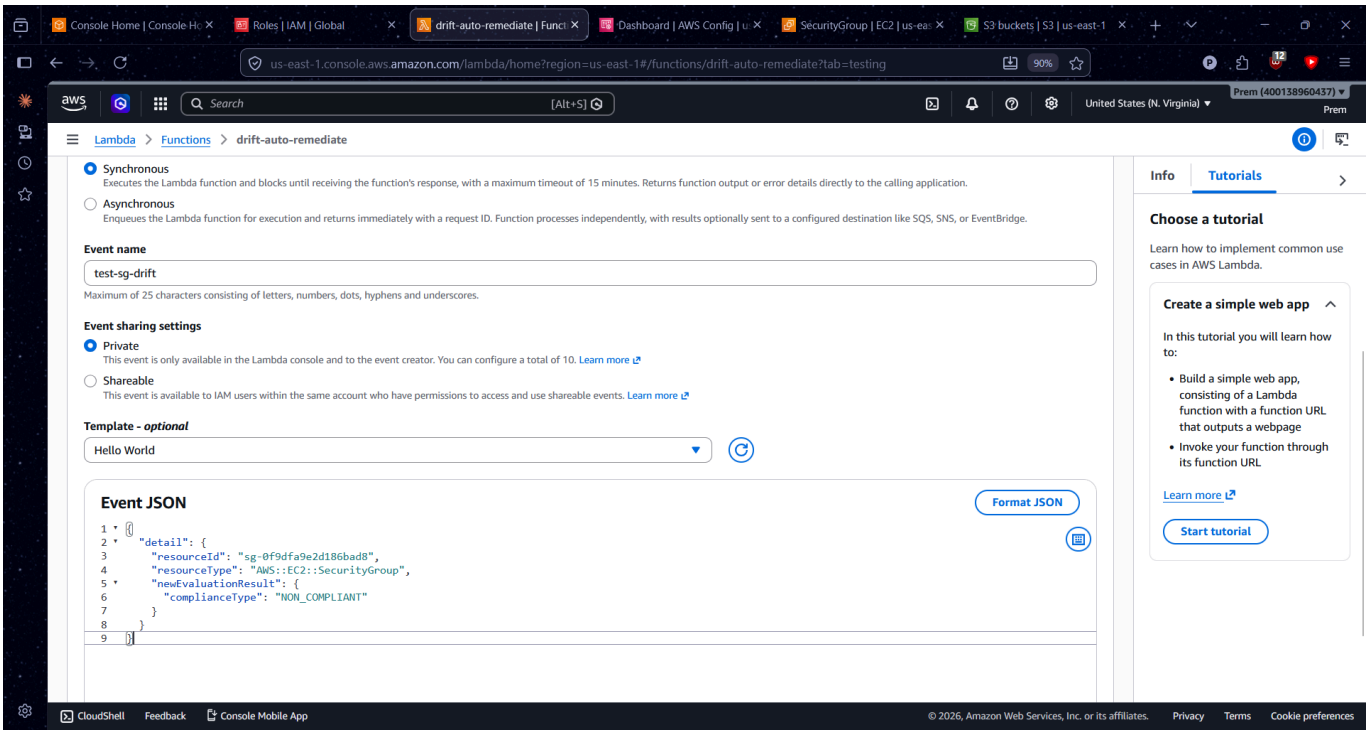
→ Upload function.zip → Deploy
```

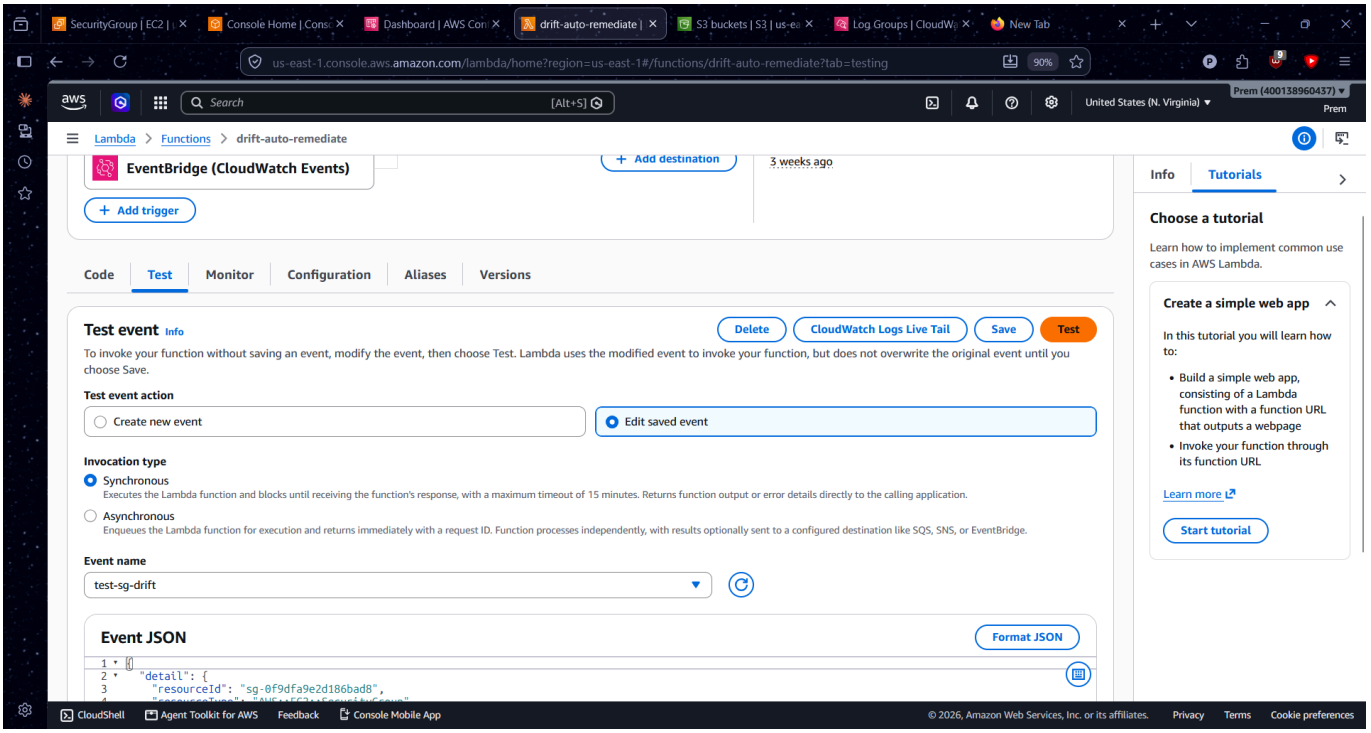

Console Home | Console Hc
Roles | IAM | Global
drift-auto-remediate | Funct
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=configure
90%
aws
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
drift-auto-remediate
Throttle
Copy ARN
Actions
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
drift-auto-remediate
Layers (0)
+ Add trigger
+ Add destination
Function ARN
arn:aws:lambda:us-east-1:400138960437:function:drift-auto-remediate
Description
-
Last modified
6 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
General configuration Info
Edit
Description
-
Memory
128 MB
Ephemeral storage
512 MB
Timeout
0 min 3 sec
SnapStart Info
None
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

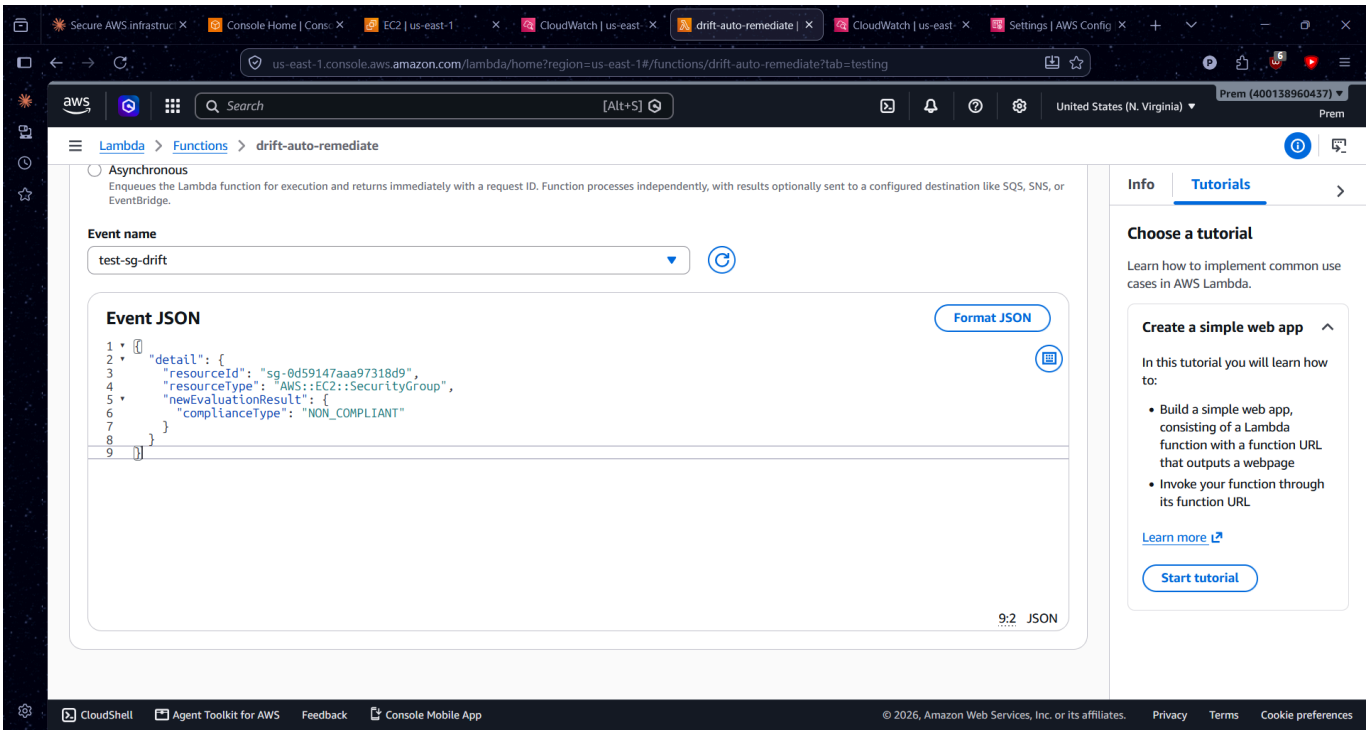

Console Home | Console Hc
Roles | IAM | Global
Edit basic settings | drift-aut
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate/edit/basic-settings?tab=config
90%
aws
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate > Edit basic settings
Edit basic settings
Basic settings Info
Description - optional
Memory Info
Your function is allocated CPU proportional to the memory configured.
128 MB
Set memory to between 128 MB and 10240 MB
Ephemeral storage Info
You can configure up to 10 GB of ephemeral storage (/tmp) for your function. View pricing
512 MB
Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.
SnapStart Info
Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the SnapStart compatibility considerations. For Python and .NET runtimes, view pricing.
None
Supported runtimes: .NET 10 (C#/F#/PowerShell), .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.
Timeout
0 min 3 sec
Execution role
To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.
drift-remediation-lambda-role
Create new role
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

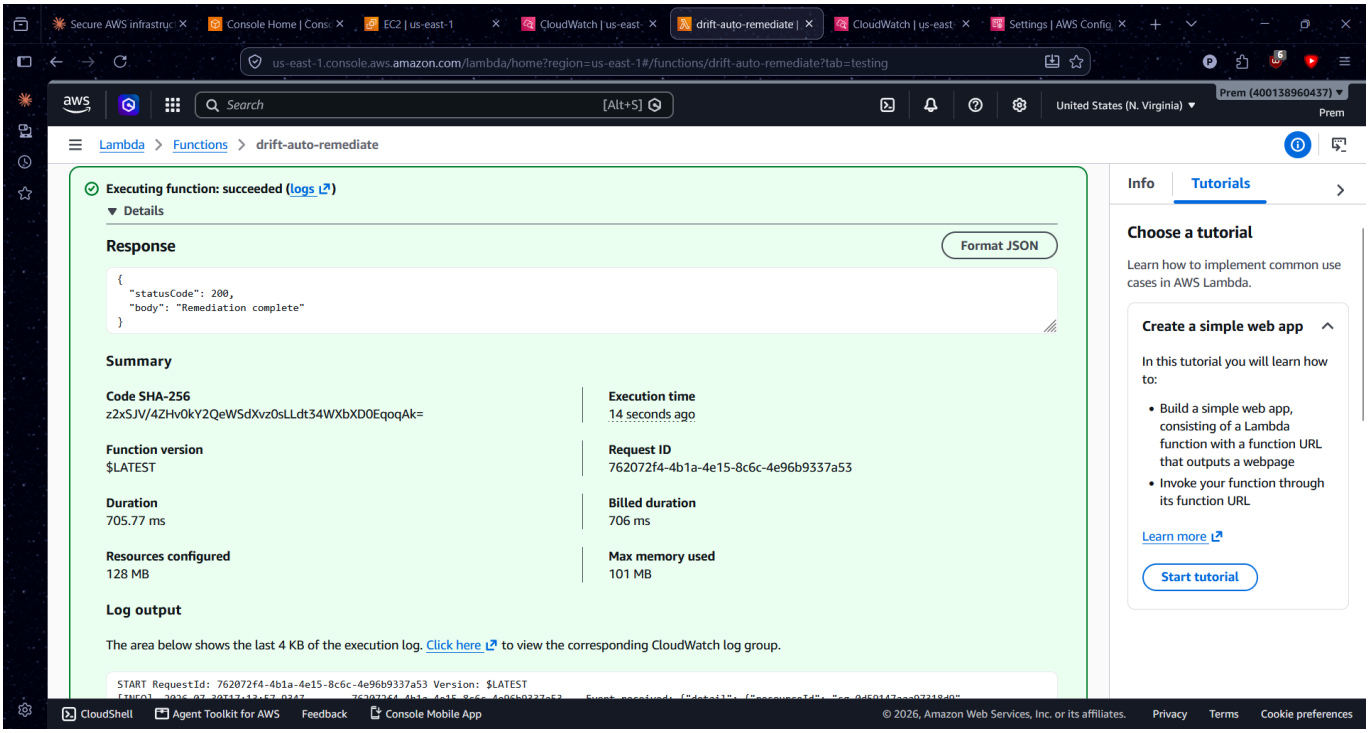

Console Home | Console Hc
Roles | IAM | Global
Edit basic settings | drift-aut
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate/edit/basic-settings?tab=config
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate > Edit basic settings
Basic settings Info
Description - optional
Memory Info
Your function is allocated CPU proportional to the memory configured.
128 MB
Set memory to between 128 MB and 10240 MB
Ephemeral storage Info
You can configure up to 10 GB of ephemeral storage (/tmp) for your function. View pricing
512 MB
Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.
SnapStart Info
Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the SnapStart compatibility considerations. For Python and .NET runtimes, view pricing.
None
Supported runtimes: .NET 10 (C#/F#/PowerShell), .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14.
Timeout
0 min 30 sec
Execution role
To access other AWS services and resources your function needs an IAM role. Choose a role with the right permissions for your function.
drift-remediation-lambda-role
Create new role
Edit role
View role details in IAM
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Hc
Roles | IAM | Global
drift-auto-remediate | Funct
Dashboard | AWS Config | u
Home | EC2 | us-east-1
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=configure
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
drift-auto-remediate
Throttle
Copy ARN
Actions
Function overview Info
Export to Infrastructure Composer
Download
Diagram
Template
drift-auto-remediate
Layers (0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:us-east-1:400138960437:function:drift-auto-remediate
Description
-
Last modified
6 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
General configuration Info
Edit
Description
-
Memory
128 MB
Ephemeral storage
512 MB
Timeout
0 min 30 sec
SnapStart Info
None
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Console Home | Console Hc
Roles | IAM | Global
drift-auto-remediate | Funct
Dashboard | AWS Config | u
SecurityGroup | EC2 | us-eas
S3 buckets | S3 | us-east-1
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=testing
90%
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.
Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.
Event name
test-sg-drift
Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.
Event sharing settings
Private
This event is only available in the Lambda console and to the event creator. You can configure a total of 10. Learn more
Shareable
This event is available to IAM users within the same account who have permissions to access and use shareable events. Learn more
Template - optional
Hello World
Event JSON
Format JSON
{
"detail": {
"resourceId": "sg-0f9dfa9e2d186bad8",
"resourceType": "AWS::EC2::SecurityGroup",
"newEvaluationResult": {
"complianceType": "NON_COMPLIANT"
}
}
}
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


SecurityGroup | EC2 |
Console Home | Cons
Dashboard | AWS Con
drift-auto-remediate |
S3 buckets | S3 | us-ea
Log Groups | CloudW
New Tab
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=testing
90%
Search
[Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
EventBridge (CloudWatch Events)
Add destination
3 weeks ago
Add trigger
Code
Test
Monitor
Configuration
Aliases
Versions
Test event Info
Delete
CloudWatch Logs Live Tail
Save
Test
To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.
Test event action
Create new event
Edit saved event
Invocation type
Synchronous
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.
Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.
Event name
test-sg-drift
Event JSON
Format JSON
{
"detail": {
"resourceId": "sg-0f9dfa9e2d186bad8",
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences





Create EventBridge Rule

```
EventBridge → Rules → Create rule
Mode: Advanced builder

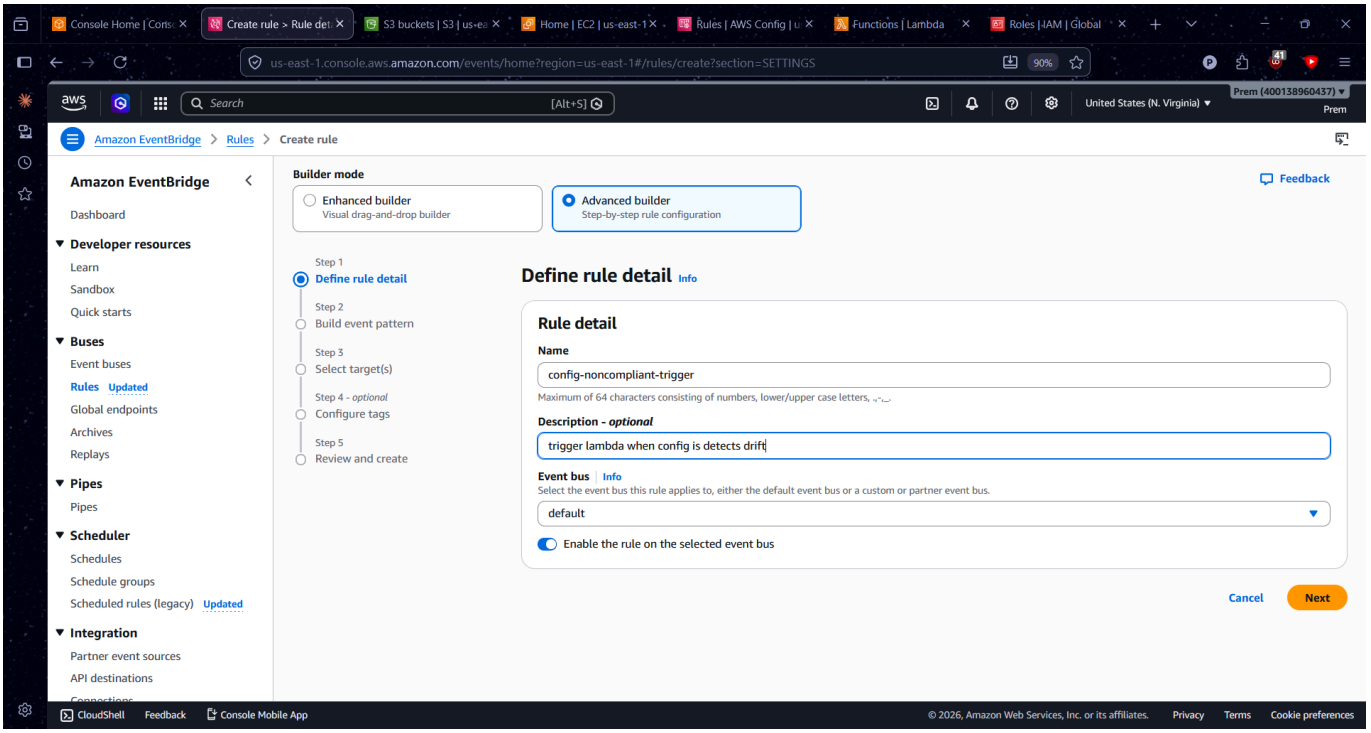
Name: config-noncompliant-trigger

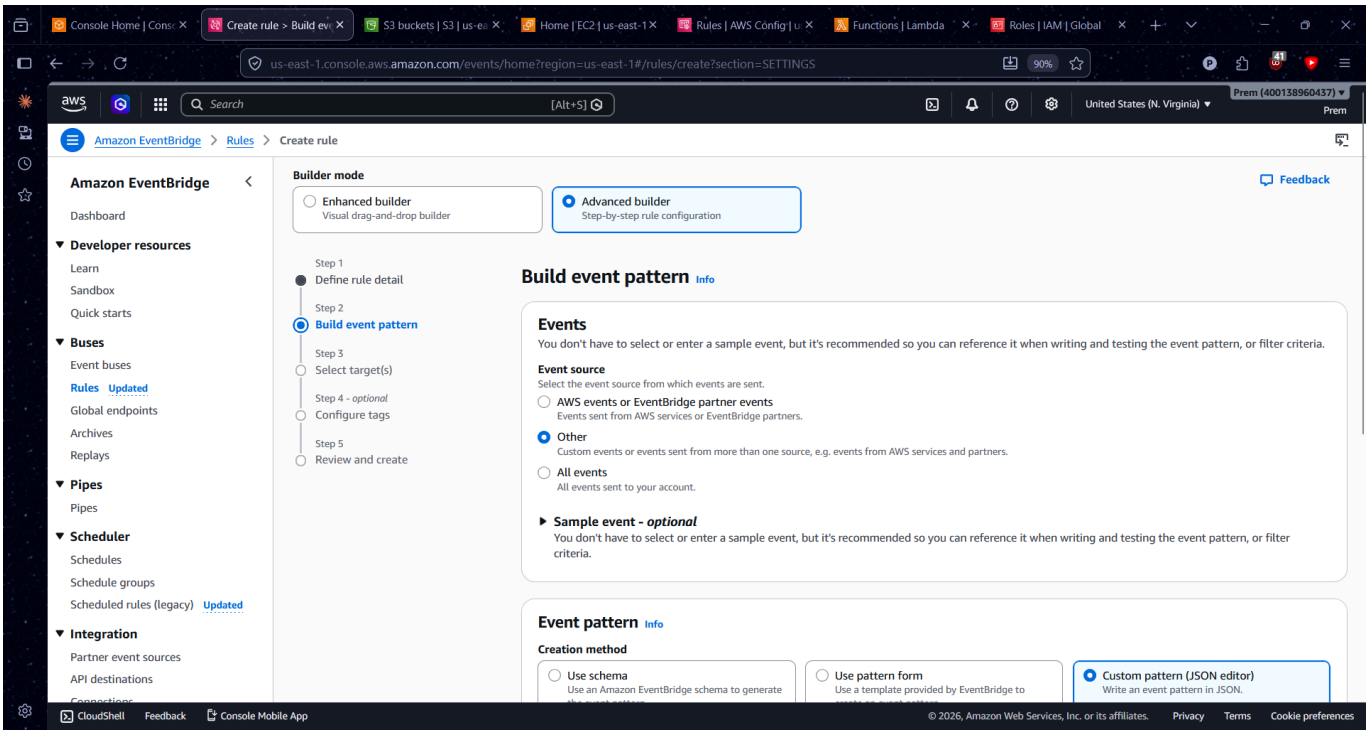
Event pattern (Custom pattern):
{
  "source": ["aws.config"],
  "detail-type": ["Config Rules Compliance Change"],
  "detail": {
    "newEvaluationResult": {
```

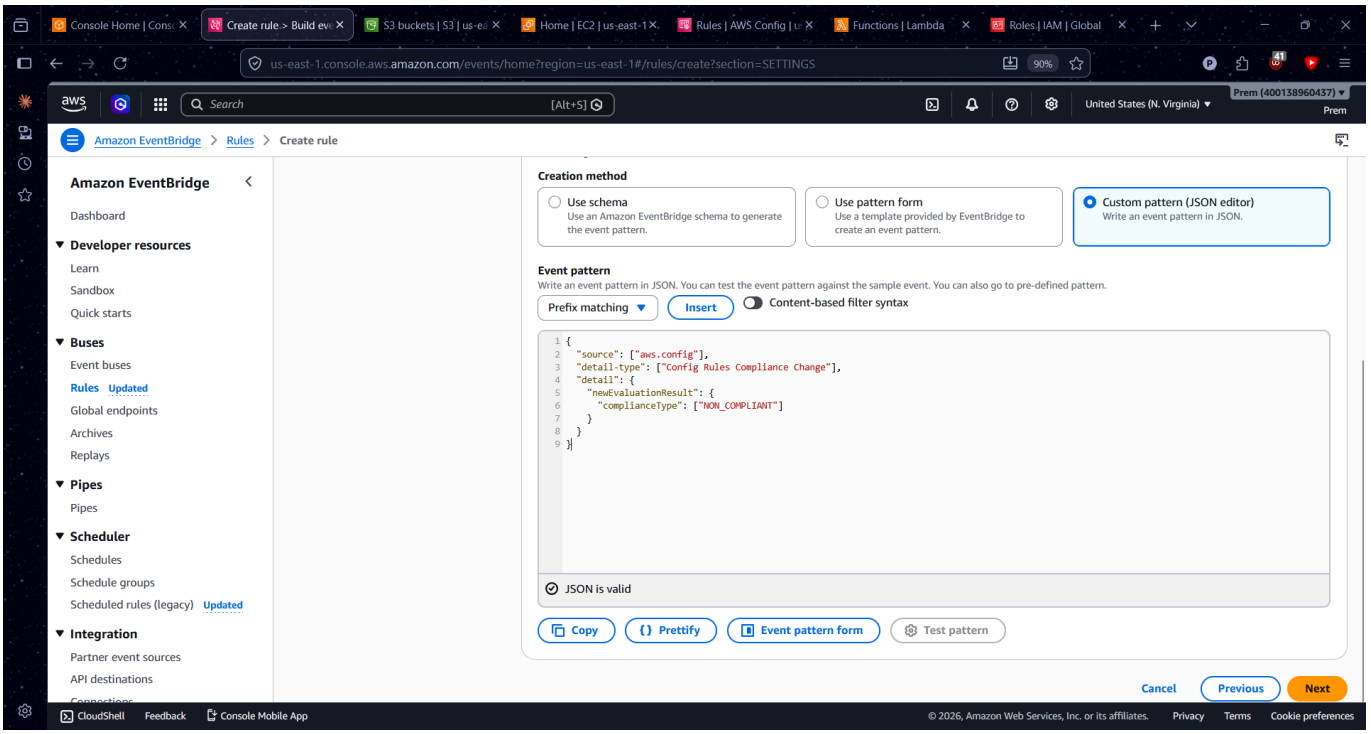
```
      "complianceType": ["NON_COMPLIANT"]
    }
  }
}

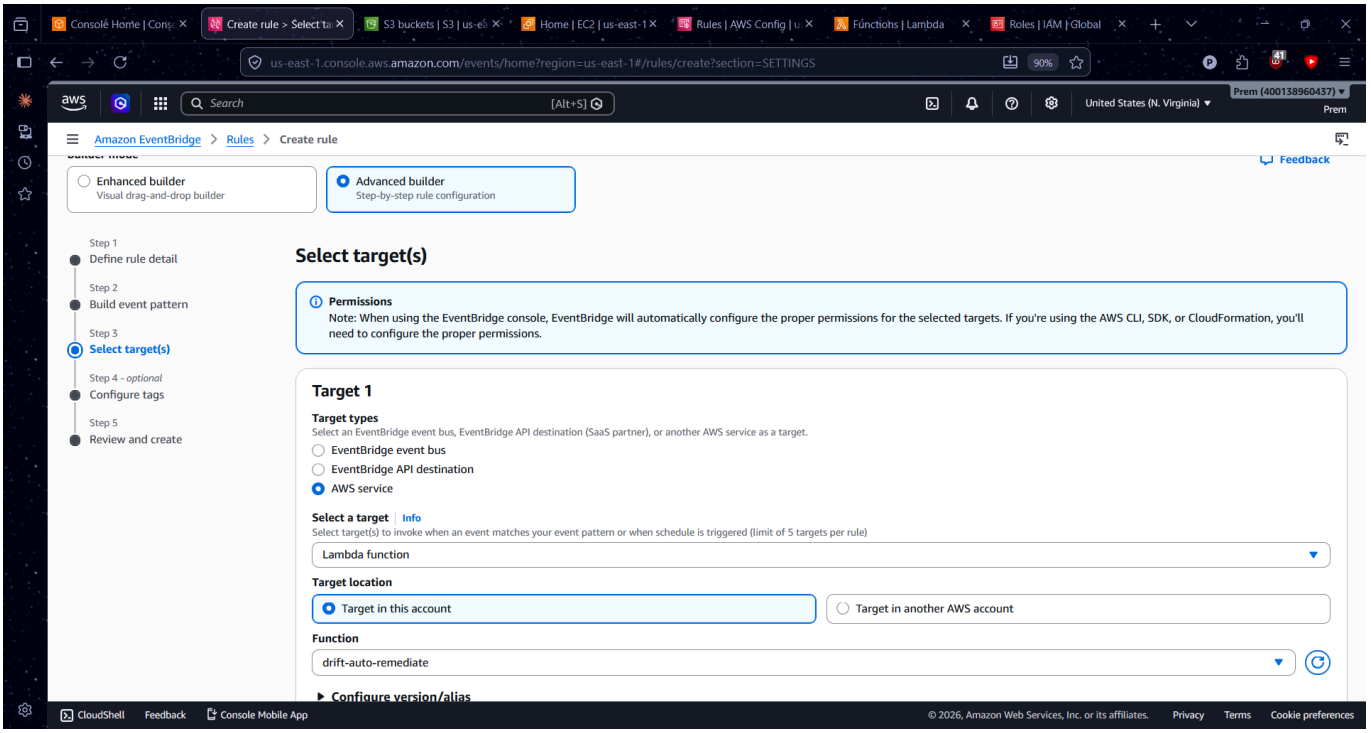
Target: Lambda function → drift-auto-remediate
→ Create rule
```

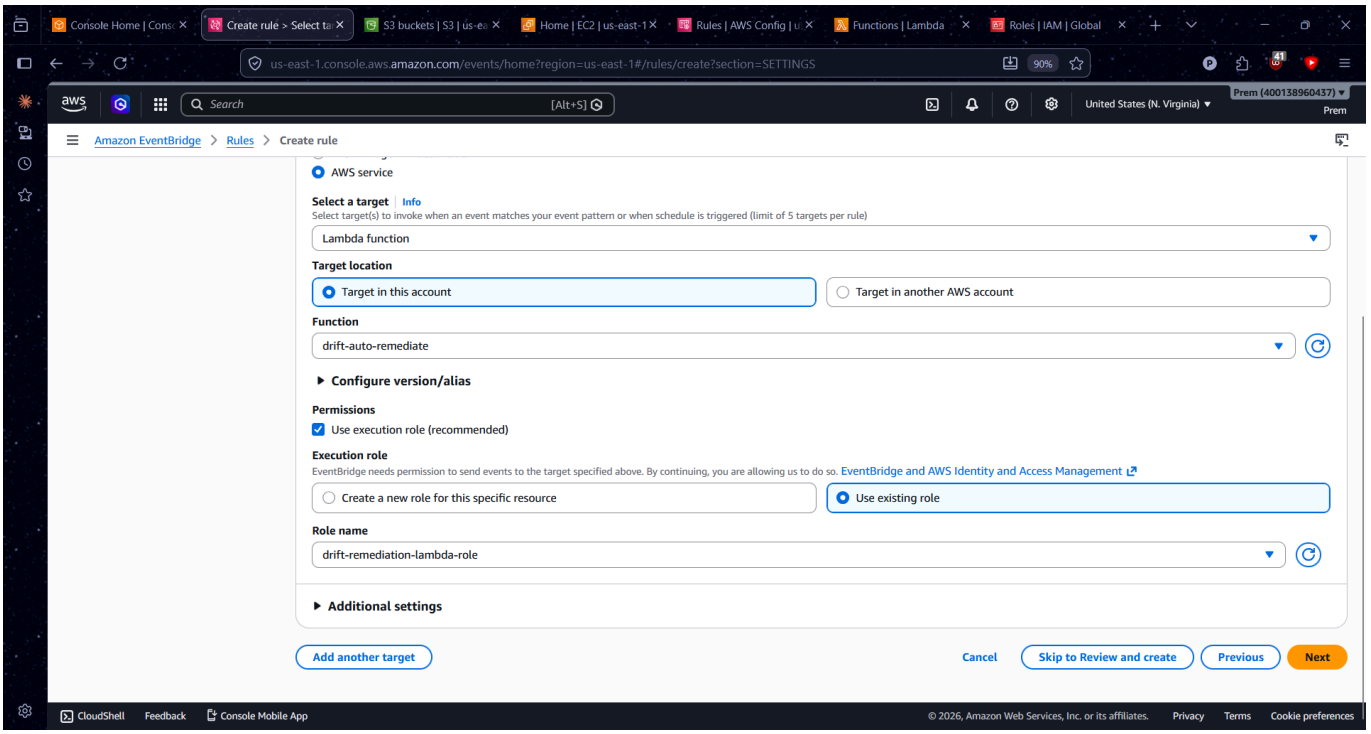
The EventBridge rule is what connects Config to Lambda. Config does not call Lambda directly — it fires an event, EventBridge catches it and routes it. This decoupling means you can later add more targets to the same event (a Slack notification, a DynamoDB audit record) without touching either Config or Lambda.

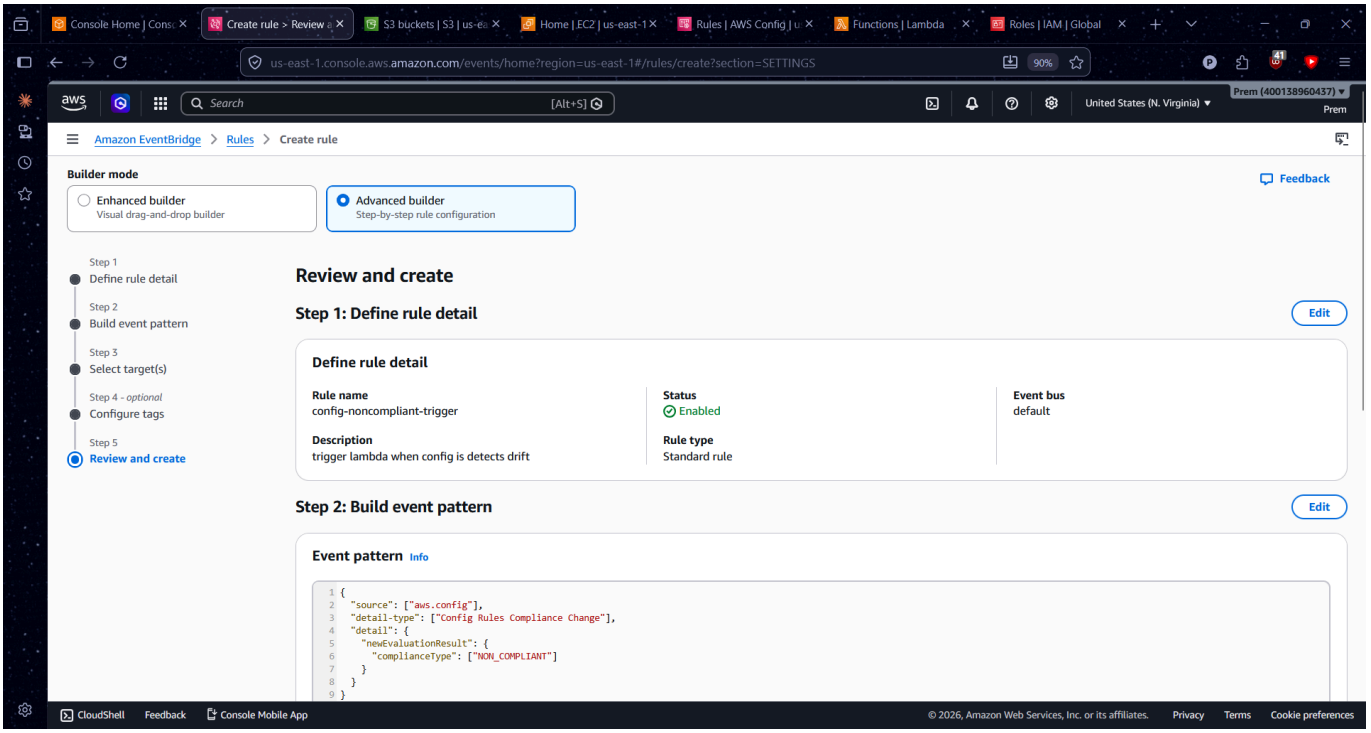


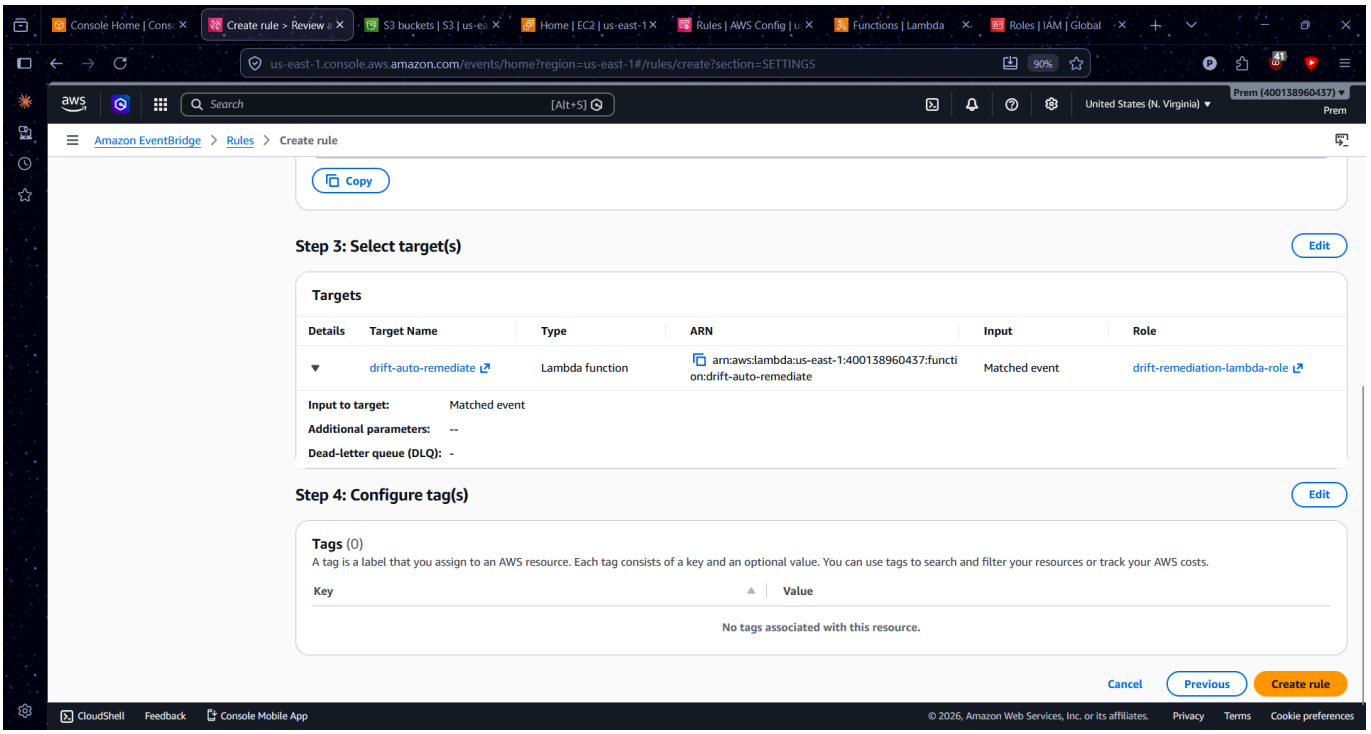


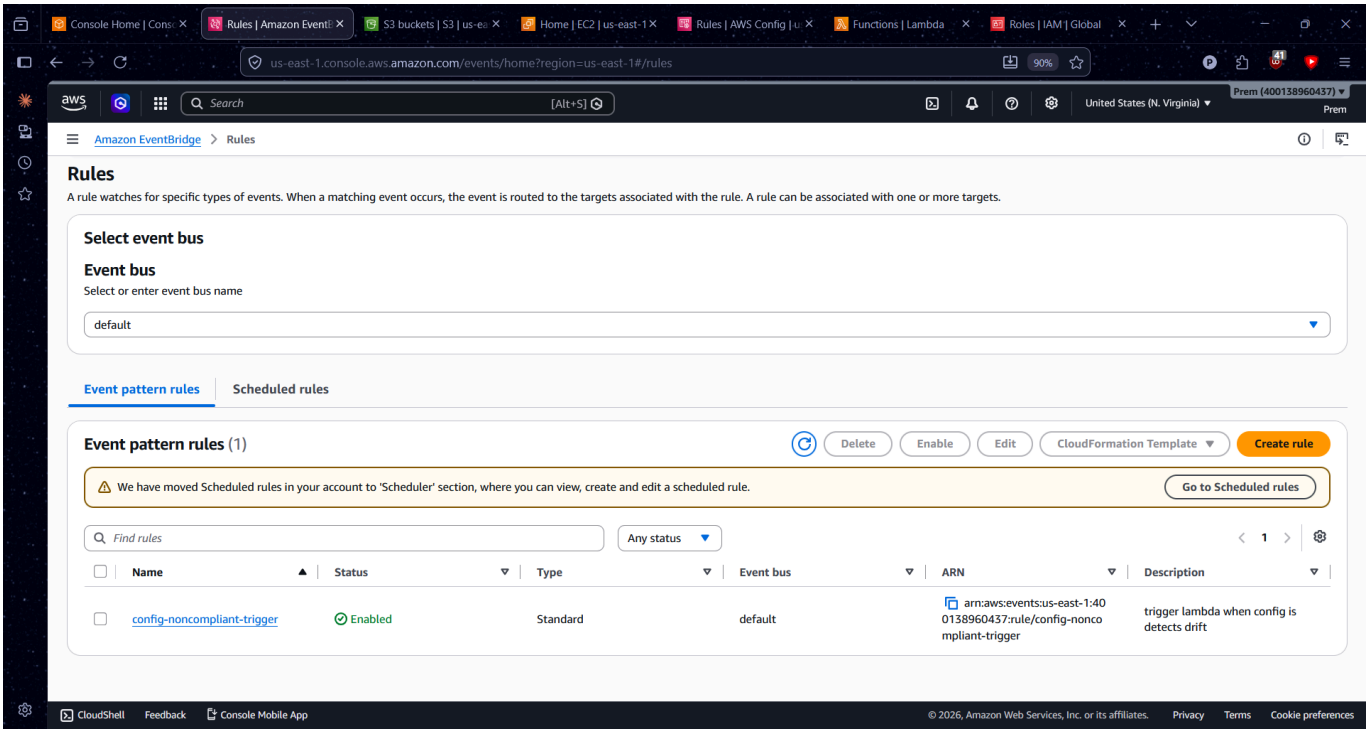

Amazon EventBridge > Rules > Create rule
Amazon EventBridge
Dashboard
Developer resources
Learn
Sandbox
Quick starts
Buses
Event buses
Rules Updated
Global endpoints
Archives
Replays
Pipes
Pipes
Scheduler
Schedules
Schedule groups
Scheduled rules (legacy) Updated
Integration
Partner event sources
API destinations
Creation method
Use schema
Use an Amazon EventBridge schema to generate the event pattern.
Use pattern form
Use a template provided by EventBridge to create an event pattern.
Custom pattern (JSON editor)
Write an event pattern in JSON.
Event pattern
Write an event pattern in JSON. You can test the event pattern against the sample event. You can also go to pre-defined pattern.
Prefix matching
Insert
Content-based filter syntax
{
"source": ["aws.config"],
"detail-type": ["Config Rules Compliance Change"],
"detail": {
"newEvaluationResult": {
"complianceType": ["NON_COMPLIANT"]
}
}
}
JSON is valid
Copy
Prettify
Event pattern form
Test pattern
Cancel
Previous
Next

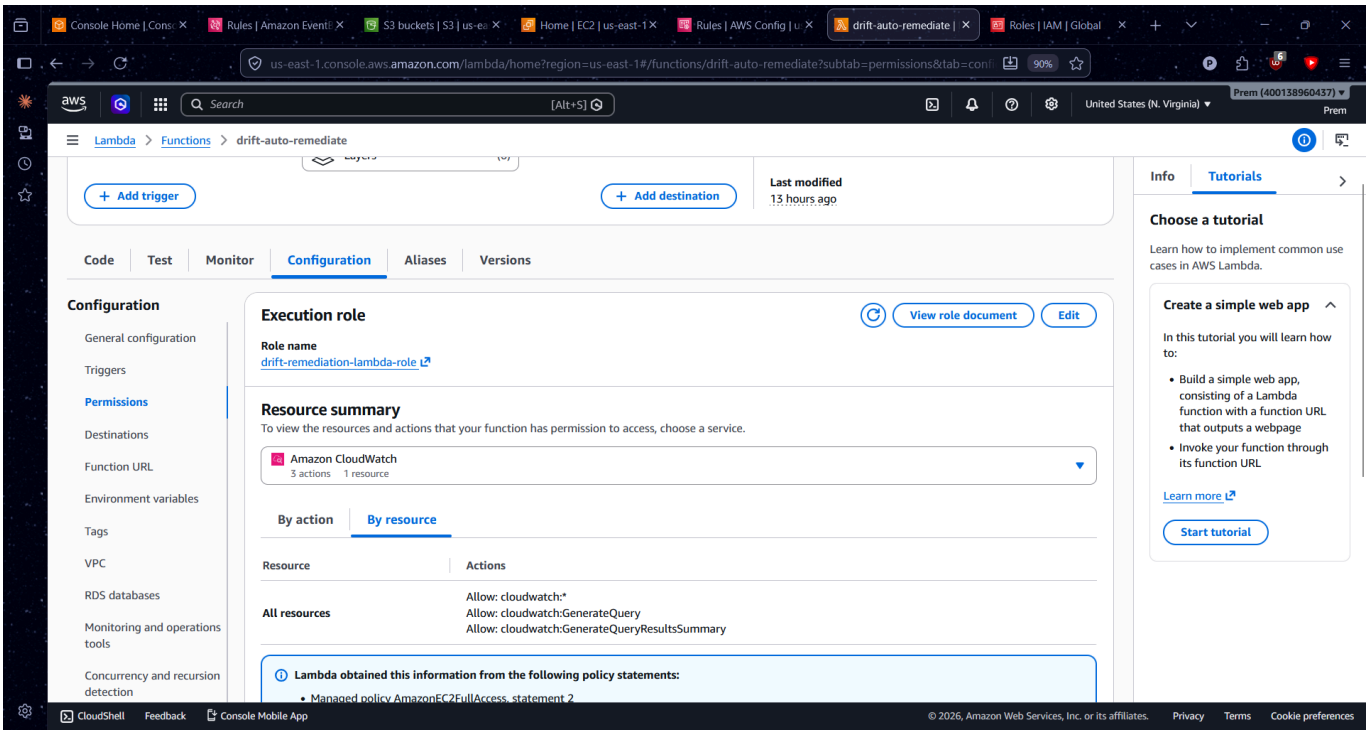

Amazon EventBridge > Rules > Create rule
Enhanced builder
Visual drag-and-drop builder
Advanced builder
Step-by-step rule configuration
Feedback
Step 1
Define rule detail
Step 2
Build event pattern
Step 3
Select target(s)
Step 4 - optional
Configure tags
Step 5
Review and create
Select target(s)
Permissions
Note: When using the EventBridge console, EventBridge will automatically configure the proper permissions for the selected targets. If you're using the AWS CLI, SDK, or CloudFormation, you'll need to configure the proper permissions.
Target 1
Target types
Select an EventBridge event bus, EventBridge API destination (SaaS partner), or another AWS service as a target.
EventBridge event bus
EventBridge API destination
AWS service
Select a target Info
Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)
Lambda function
Target location
Target in this account
Target in another AWS account
Function
drift-auto-remediate
Configure version/alias

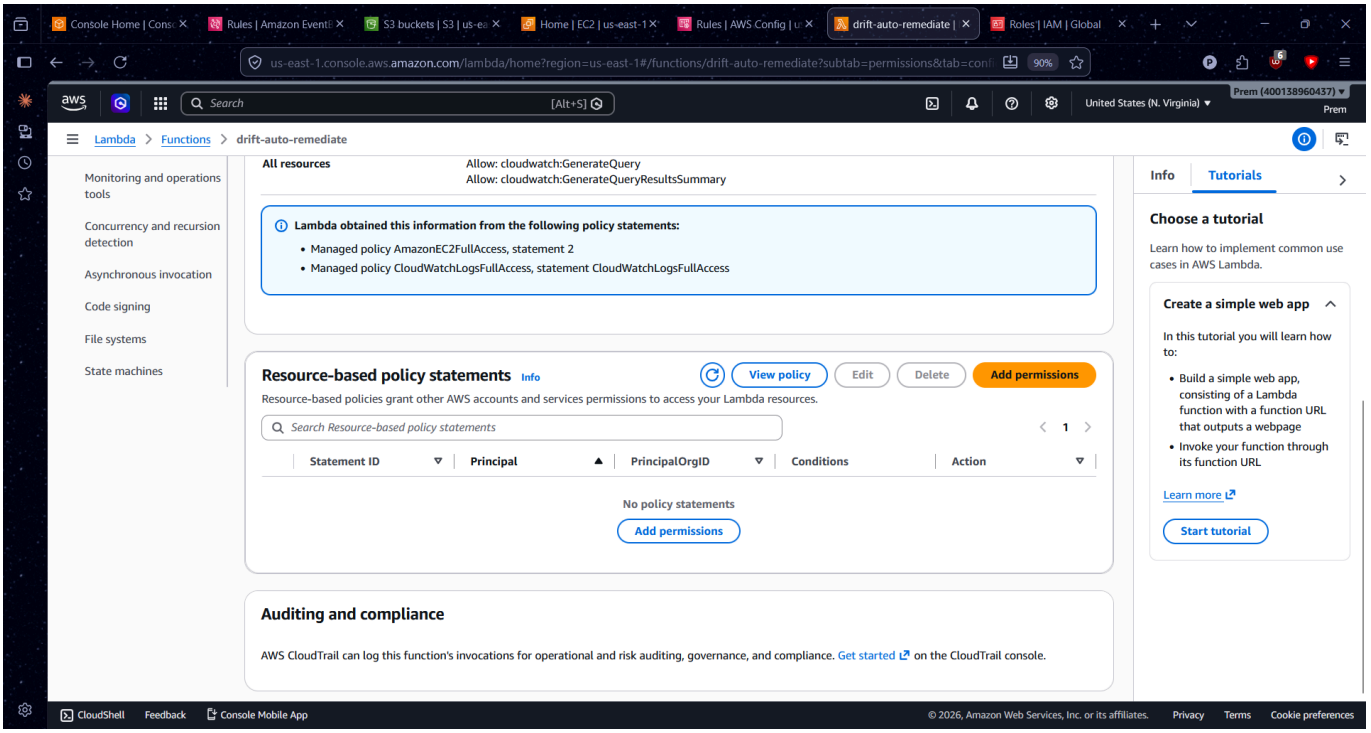

Amazon EventBridge > Rules > Create rule
AWS service
Select a target | Info
Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)
Lambda function
Target location
Target in this account
Target in another AWS account
Function
drift-auto-remediate
Configure version/alias
Permissions
Use execution role (recommended)
Execution role
EventBridge needs permission to send events to the target specified above. By continuing, you are allowing us to do so. EventBridge and AWS Identity and Access Management
Create a new role for this specific resource
Use existing role
Role name
drift-remediation-lambda-role
Additional settings
Add another target
Cancel
Skip to Review and create
Previous
Next

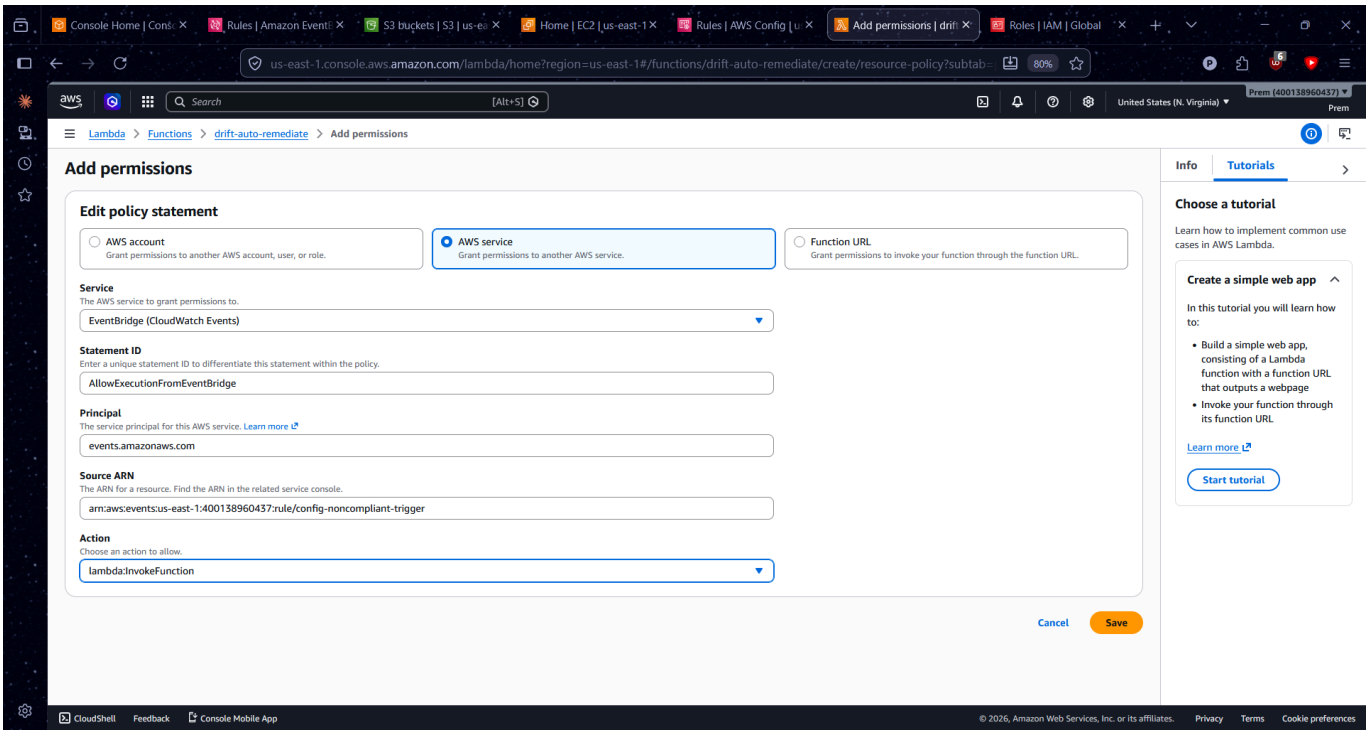

Amazon EventBridge > Rules > Create rule
Builder mode
Enhanced builder
Visual drag-and-drop builder
Advanced builder
Step-by-step rule configuration
Feedback
Step 1 Define rule detail
Step 2 Build event pattern
Step 3 Select target(s)
Step 4 - optional Configure tags
Step 5 Review and create
Review and create
Step 1: Define rule detail
Edit
Define rule detail
Rule name
config-noncompliant-trigger
Status
Enabled
Event bus
default
Description
trigger lambda when config is detects drift
Rule type
Standard rule
Step 2: Build event pattern
Edit
Event pattern Info
{
"source": ["aws.config"],
"detail-type": ["Config Rules Compliance Change"],
"detail": {
"newEvaluationResult": {
"complianceType": ["NON_COMPLIANT"]
}
}
}

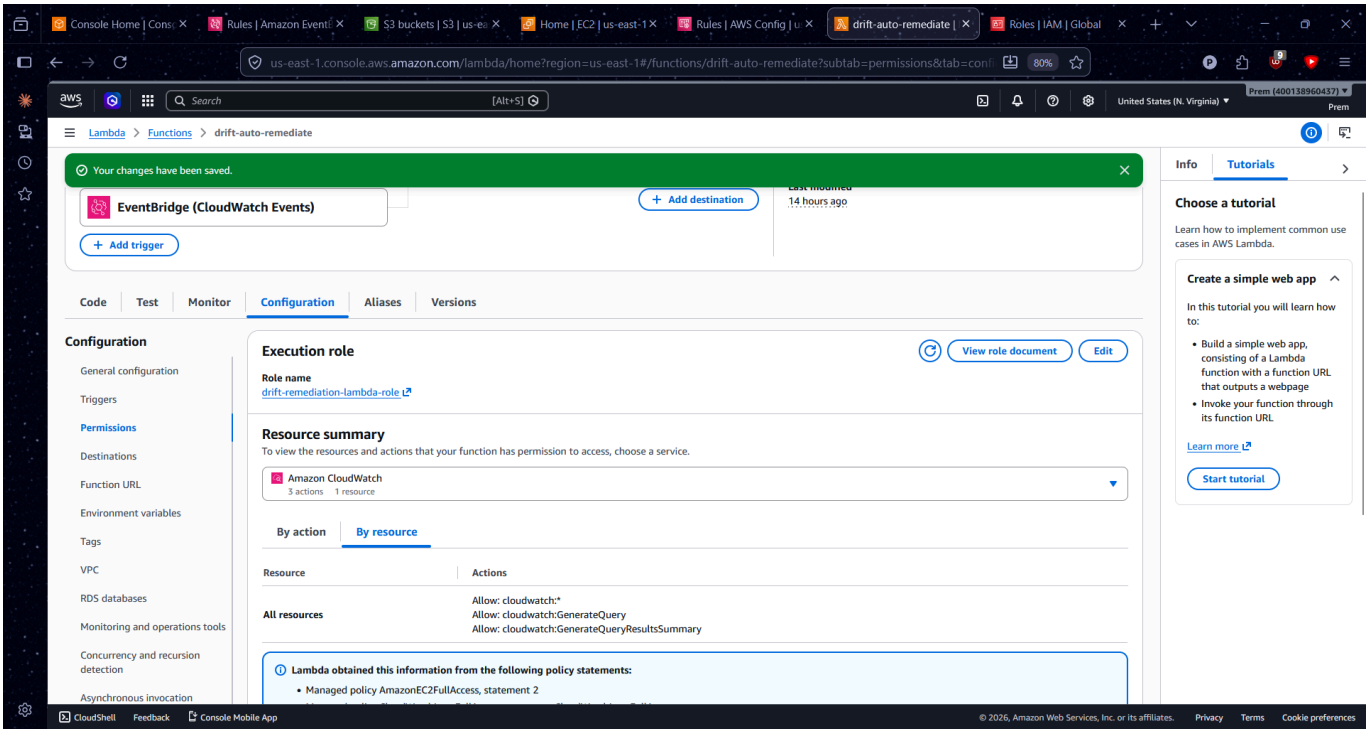

Amazon EventBridge > Rules > Create rule
Copy
Step 3: Select target(s)
Edit
Targets
Details
Target Name
Type
ARN
Input
Role
drift-auto-remediate
Lambda function
arn:aws:lambda:us-east-1:400138960437:function:drift-auto-remediate
Matched event
drift-remediation-lambda-role
Input to target: Matched event
Additional parameters: --
Dead-letter queue (DLQ): -
Step 4: Configure tag(s)
Edit
Tags (0)
A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.
Key
Value
No tags associated with this resource.
Cancel
Previous
Create rule


Amazon EventBridge > Rules
Rules
A rule watches for specific types of events. When a matching event occurs, the event is routed to the targets associated with the rule. A rule can be associated with one or more targets.
Select event bus
Event bus
Select or enter event bus name
default
Event pattern rules
Scheduled rules
Event pattern rules (1)
Delete
Enable
Edit
CloudFormation Template
Create rule
We have moved Scheduled rules in your account to 'Scheduler' section, where you can view, create and edit a scheduled rule.
Go to Scheduled rules
Find rules
Any status
Name
Status
Type
Event bus
ARN
Description
config-noncompliant-trigger
Enabled
Standard
default
arn:aws:events:us-east-1:400138960437:rule/config-noncompliant-trigger
trigger lambda when config is detects drift


Lambda > Functions > drift-auto-remediate
+ Add trigger
+ Add destination
Last modified
13 hours ago
Code Test Monitor Configuration Aliases Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Monitoring and operations tools
Concurrency and recursion detection
Execution role
View role document
Edit
Role name
drift-remediation-lambda-role
Resource summary
To view the resources and actions that your function has permission to access, choose a service.
Amazon CloudWatch
3 actions 1 resource
By action By resource
Resource
Actions
All resources
Allow: cloudwatch:*
Allow: cloudwatch:GenerateQuery
Allow: cloudwatch:GenerateQueryResultsSummary
Lambda obtained this information from the following policy statements:
Managed policy AmazonEC2FullAccess, statement 2
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


Lambda > Functions > drift-auto-remediate
Monitoring and operations tools
Concurrency and recursion detection
Asynchronous invocation
Code signing
File systems
State machines
All resources
Allow: cloudwatch:GenerateQuery
Allow: cloudwatch:GenerateQueryResultsSummary
Lambda obtained this information from the following policy statements:
Managed policy AmazonEC2FullAccess, statement 2
Managed policy CloudWatchLogsFullAccess, statement CloudWatchLogsFullAccess
Resource-based policy statements
Info
View policy
Edit
Delete
Add permissions
Resource-based policies grant other AWS accounts and services permissions to access your Lambda resources.
Search Resource-based policy statements
Statement ID
Principal
PrincipalOrgID
Conditions
Action
No policy statements
Add permissions
Auditing and compliance
AWS CloudTrail can log this function's invocations for operational and risk auditing, governance, and compliance. Get started on the CloudTrail console.
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences





Phase 5 — End-to-End Test and Verification

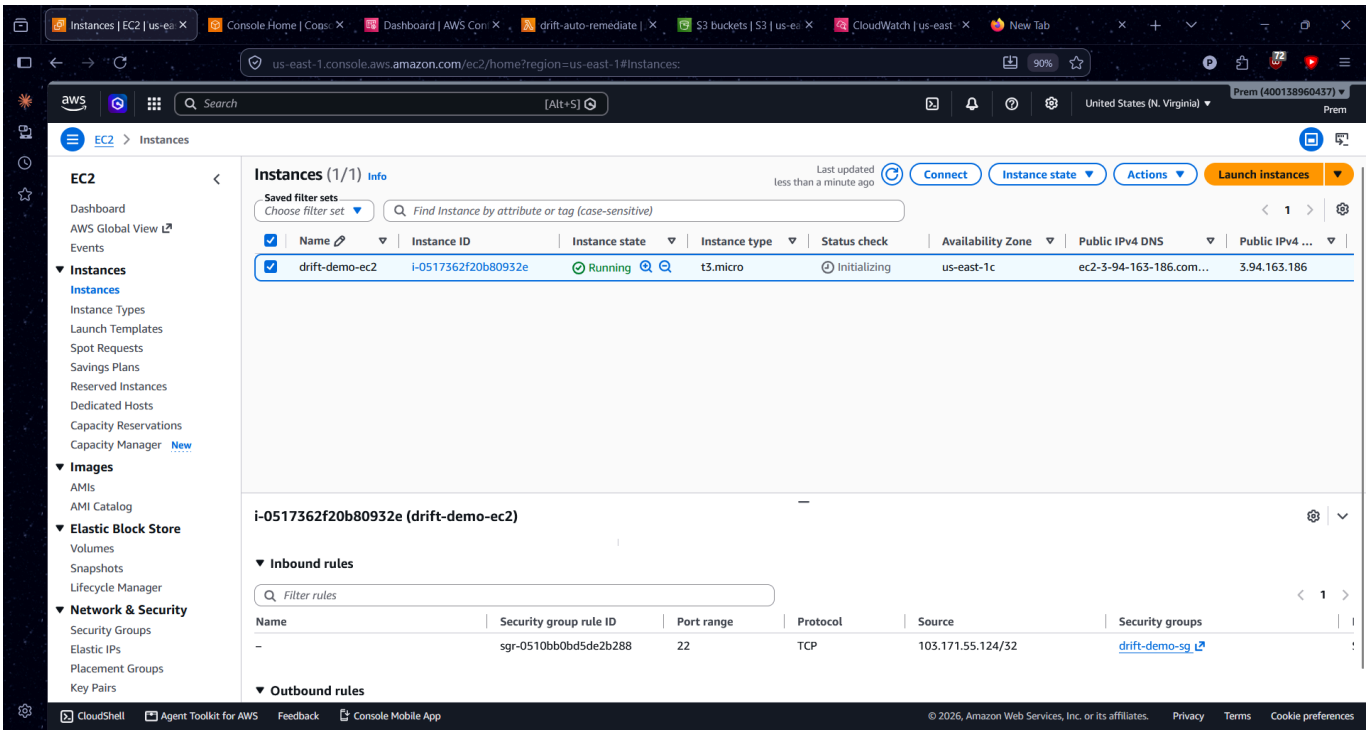
Time to prove the whole system works together, not just in isolation.

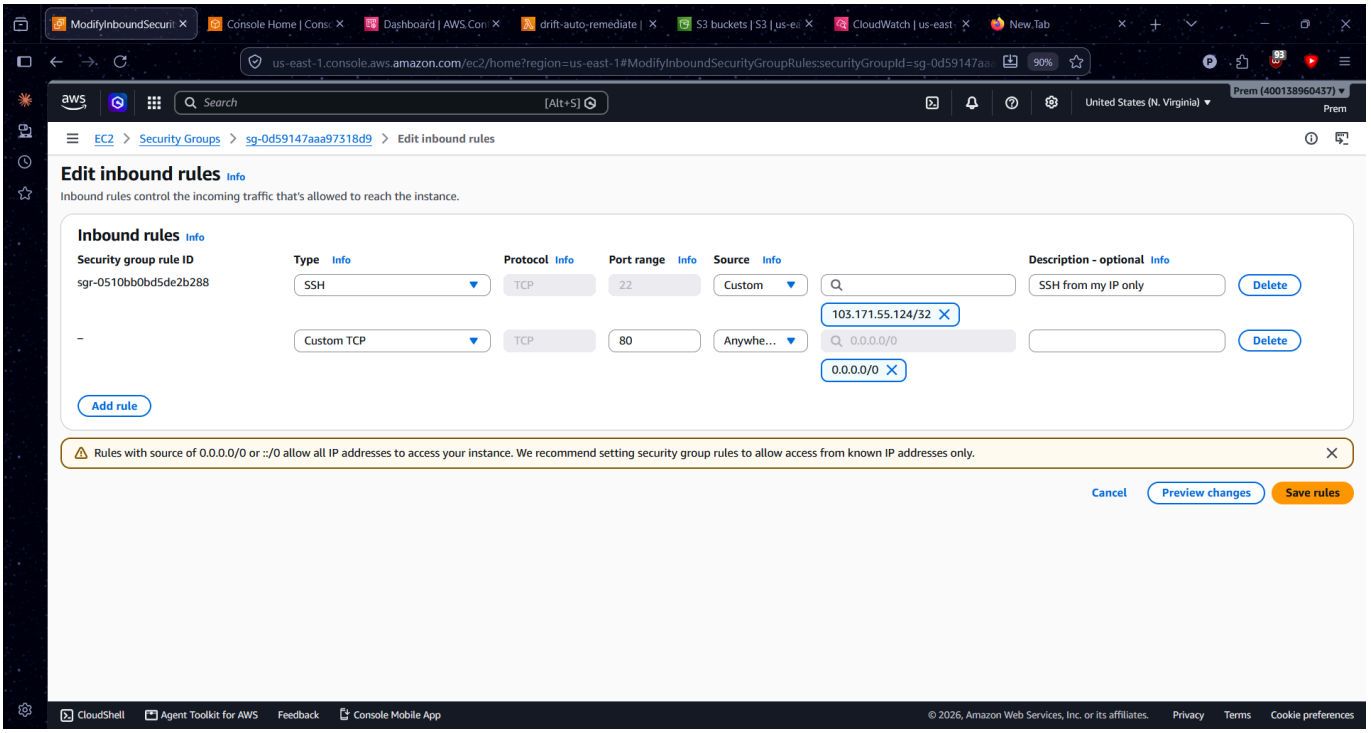
```
# Step 1: Add drift via console
# AWS Console → drift-demo-sg → Edit inbound rules
# Add: HTTP, port 80, source 0.0.0.0/0 → Save

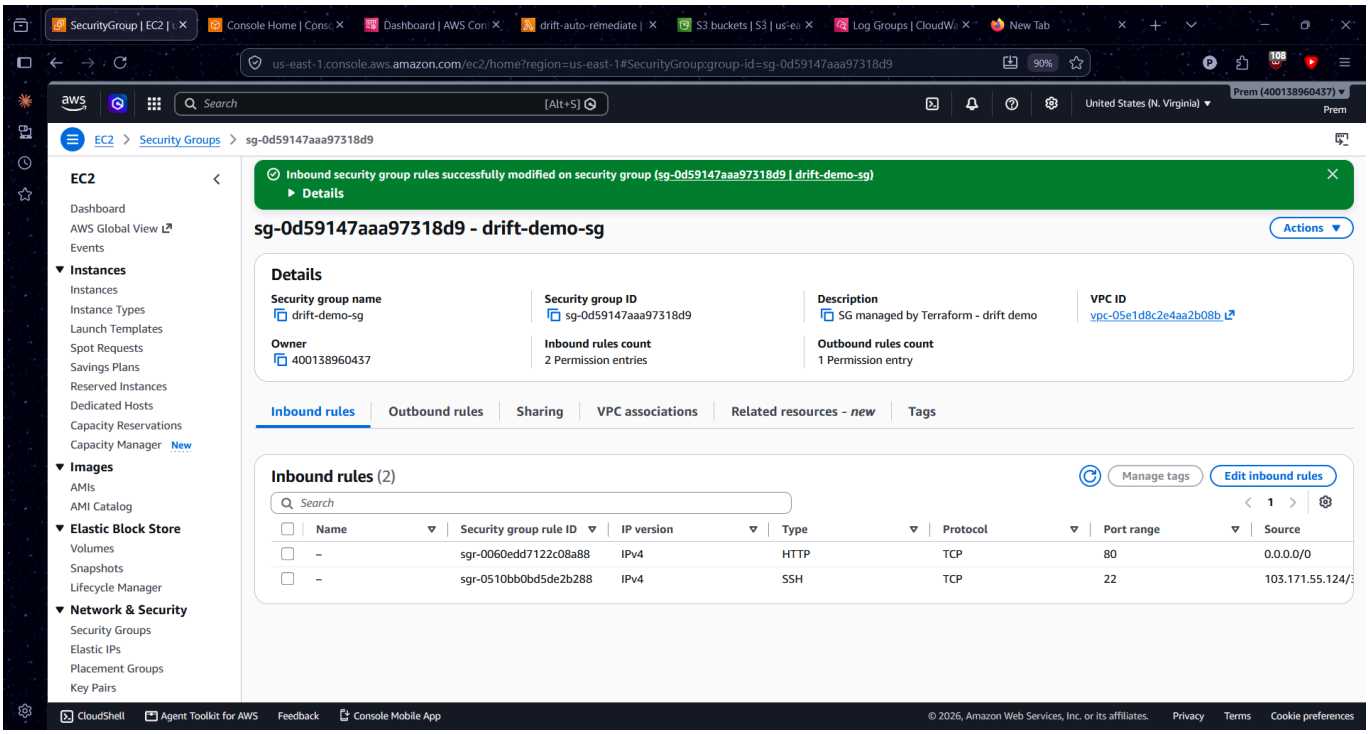
# Step 2: Force Config to evaluate immediately
# AWS Config → Rules → terraform-drift-sg-rule
# → Actions → Re-evaluate
# (Otherwise you wait up to 15 minutes for automatic evaluation)
```

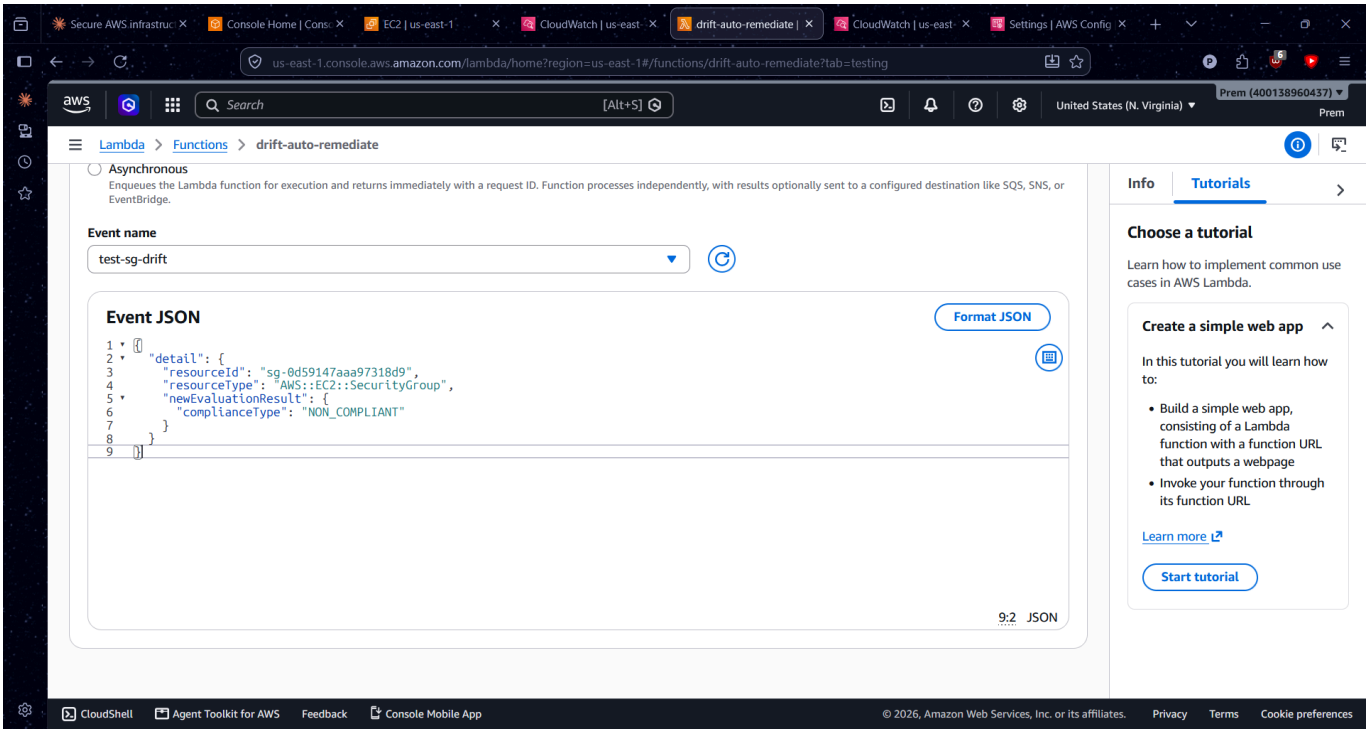
```
# Step 3: Check the Security Group 30 seconds later
# EC2 → Security Groups → drift-demo-sg
# Port 80 should be gone — automatically
```

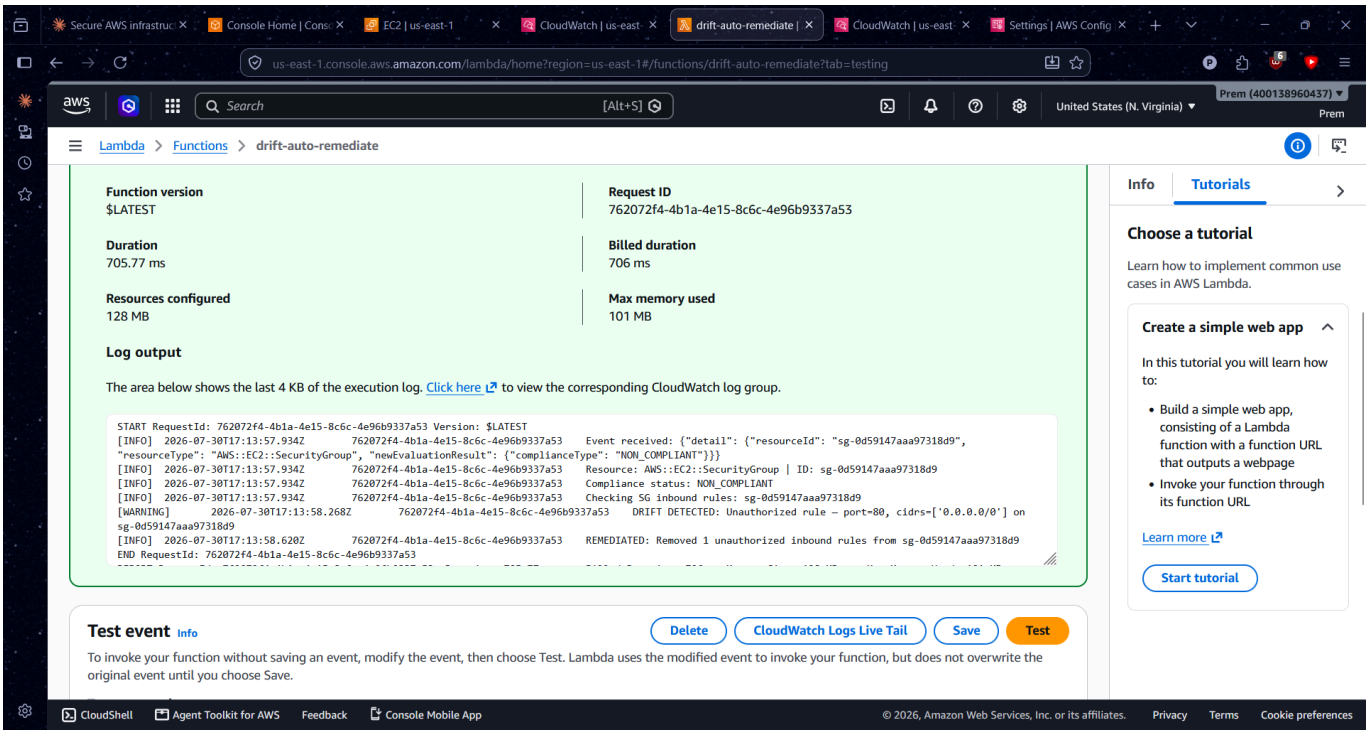
The moment Config marks the rule NON_COMPLIANT, EventBridge fires, Lambda runs, and the unauthorized rule is revoked. No ticket raised. No engineer paged. No runbook followed.

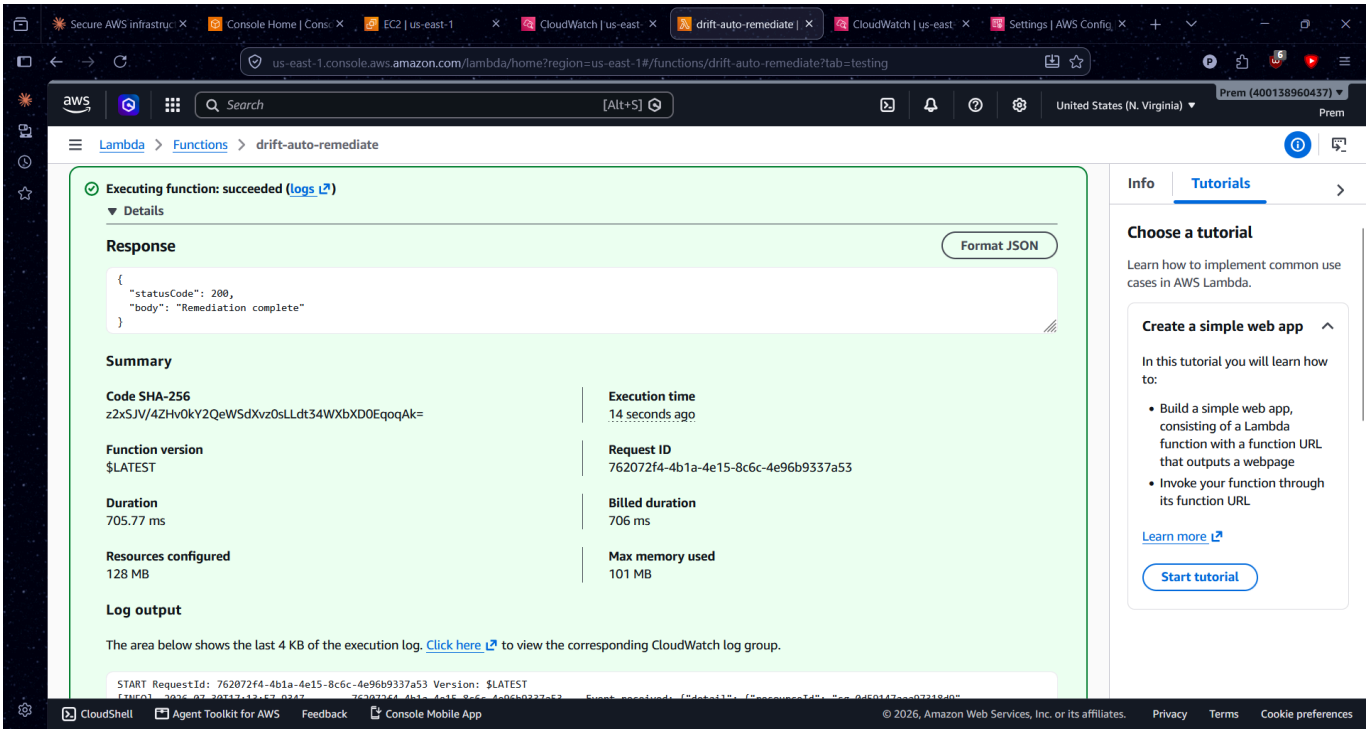


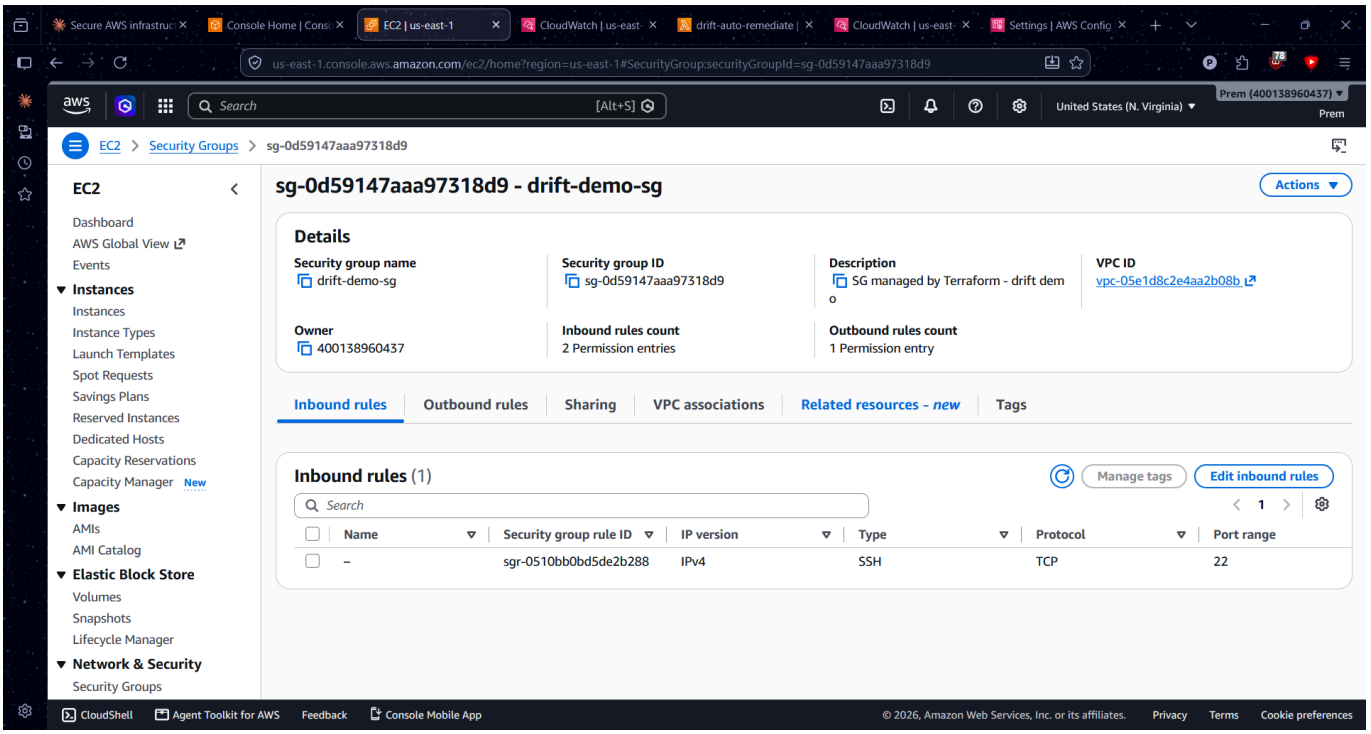



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#SecurityGroup:group-id=sg-0d59147aaa97318d9
EC2 > Security Groups > sg-0d59147aaa97318d9
Inbound security group rules successfully modified on security group (sg-0d59147aaa97318d9 | drift-demo-sg)
Details
sg-0d59147aaa97318d9 - drift-demo-sg
Actions
Details
Security group name
drift-demo-sg
Security group ID
sg-0d59147aaa97318d9
Description
SG managed by Terraform - drift demo
VPC ID
vpc-05e1d8c2e4aa2b08b
Owner
400138960437
Inbound rules count
2 Permission entries
Outbound rules count
1 Permission entry
Inbound rules
Outbound rules
Sharing
VPC associations
Related resources - new
Tags
Inbound rules (2)
Manage tags
Edit inbound rules
Name
Security group rule ID
IP version
Type
Protocol
Port range
Source
sgr-0060edd7122c08a88
IPv4
HTTP
TCP
80
0.0.0.0/0
sgr-0510bb0bd5de2b288
IPv4
SSH
TCP
22
103.171.55.124/
EC2
Dashboard
AWS Global View
Events
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager
Images
AMIs
AMI Catalog
Elastic Block Store
Volumes
Snapshots
Lifecycle Manager
Network & Security
Security Groups
Elastic IPs
Placement Groups
Key Pairs
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences


us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=testing
Lambda > Functions > drift-auto-remediate
Asynchronous
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.
Event name
test-sg-drift
Event JSON
Format JSON
{
"detail": {
"resourceId": "sg-0d59147aaa97318d9",
"resourceType": "AWS::EC2::SecurityGroup",
"newEvaluationResult": {
"complianceType": "NON_COMPLIANT"
}
}
}
9:2 JSON
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

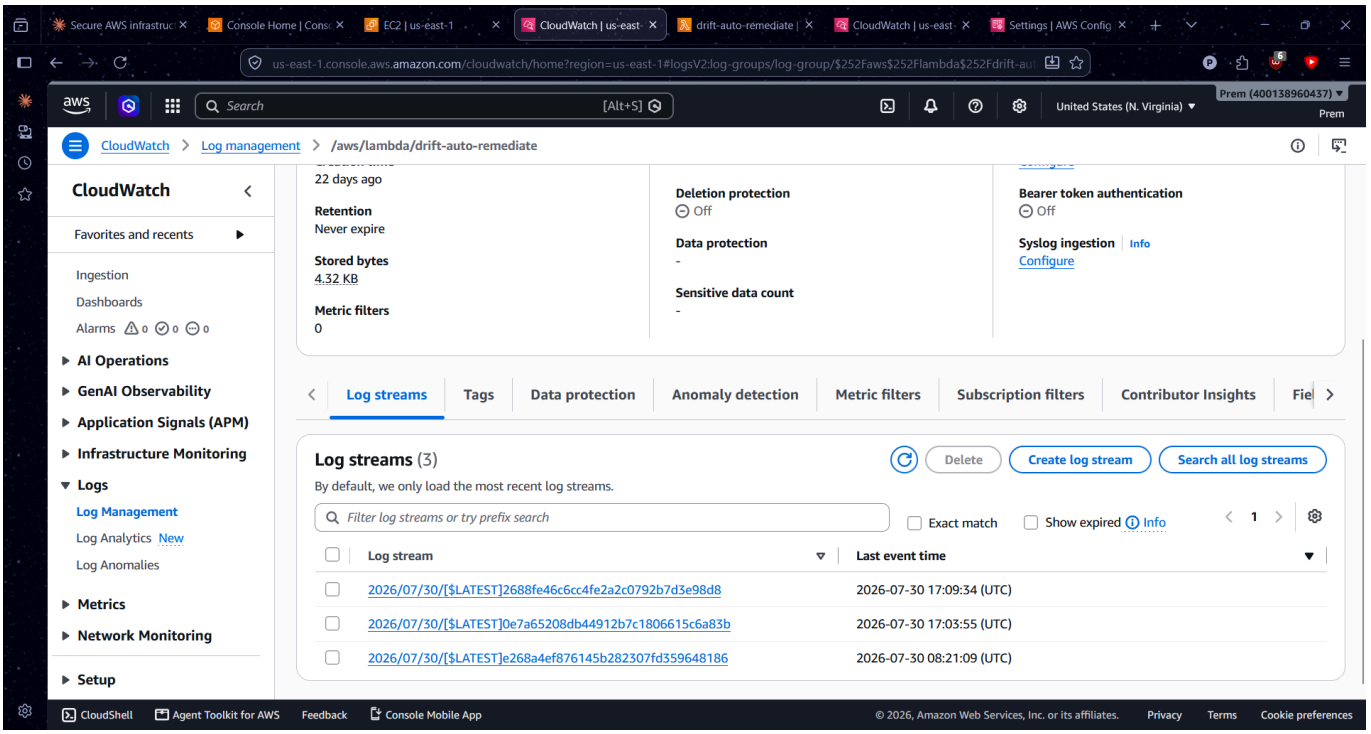

Secure AWS infrastruc
Console Home | Cons
EC2 | us-east-1
CloudWatch | us-east
drift-auto-remediate |
CloudWatch | us-east
Settings | AWS Config
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=testing
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
Function version
$LATEST
Request ID
762072f4-4b1a-4e15-8c6c-4e96b9337a53
Duration
705.77 ms
Billed duration
706 ms
Resources configured
128 MB
Max memory used
101 MB
Log output
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.
START RequestId: 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Version: $LATEST
[INFO] 2026-07-30T17:13:57.934Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Event received: {"detail": {"resourceId": "sg-0d59147aaa97318d9",
"resourceType": "AWS::EC2::SecurityGroup", "newEvaluationResult": {"complianceType": "NON_COMPLIANT"}}}
[INFO] 2026-07-30T17:13:57.934Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Resource: AWS::EC2::SecurityGroup | ID: sg-0d59147aaa97318d9
[INFO] 2026-07-30T17:13:57.934Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Compliance status: NON_COMPLIANT
[INFO] 2026-07-30T17:13:57.934Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Checking SG inbound rules: sg-0d59147aaa97318d9
[WARNING] 2026-07-30T17:13:58.268Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 DRIFT DETECTED: Unauthorized rule — port-80, cidrs=['0.0.0.0/0'] on
sg-0d59147aaa97318d9
[INFO] 2026-07-30T17:13:58.620Z 762072f4-4b1a-4e15-8c6c-4e96b9337a53 REMEDIATED: Removed 1 unauthorized inbound rules from sg-0d59147aaa97318d9
END RequestId: 762072f4-4b1a-4e15-8c6c-4e96b9337a53
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
Test event Info
Delete
CloudWatch Logs Live Tail
Save
Test
To invoke your function without saving an event, modify the event, then choose Test. Lambda uses the modified event to invoke your function, but does not overwrite the original event until you choose Save.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

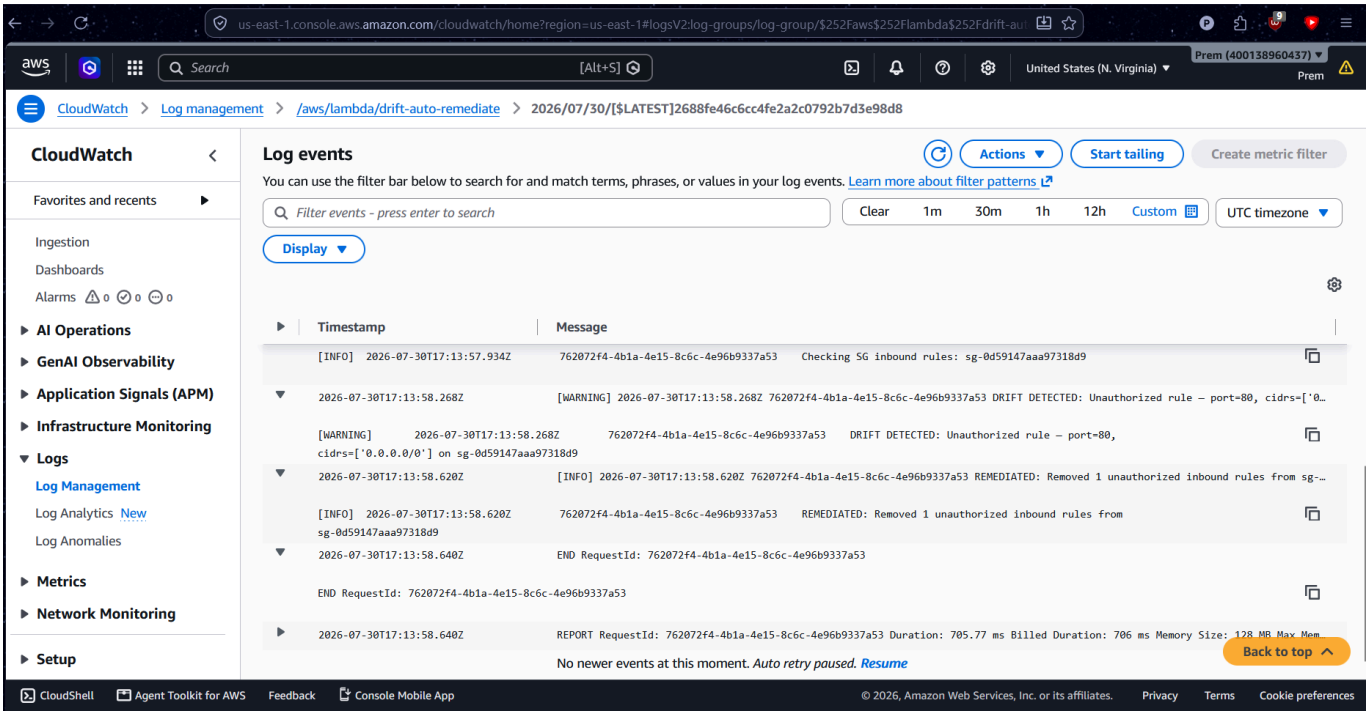

Secure AWS infrastruc
Console Home | Cons
EC2 | us-east-1
CloudWatch | us-east
drift-auto-remediate |
CloudWatch | us-east
Settings | AWS Config
us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/drift-auto-remediate?tab=testing
Search [Alt+S]
United States (N. Virginia)
Prem (400138960437)
Prem
Lambda > Functions > drift-auto-remediate
Executing function: succeeded (logs)
Details
Response
Format JSON
{
"statusCode": 200,
"body": "Remediation complete"
}
Summary
Code SHA-256
z2xSJV/4ZHv0kY2QeWSdXvz0sLLdt34WXbXD0EqoqAk=
Execution time
14 seconds ago
Function version
$LATEST
Request ID
762072f4-4b1a-4e15-8c6c-4e96b9337a53
Duration
705.77 ms
Billed duration
706 ms
Resources configured
128 MB
Max memory used
101 MB
Log output
The area below shows the last 4 KB of the execution log. Click here to view the corresponding CloudWatch log group.
START RequestId: 762072f4-4b1a-4e15-8c6c-4e96b9337a53 Version: $LATEST
Info
Tutorials
Choose a tutorial
Learn how to implement common use cases in AWS Lambda.
Create a simple web app
In this tutorial you will learn how to:
Build a simple web app, consisting of a Lambda function with a function URL that outputs a webpage
Invoke your function through its function URL
Learn more
Start tutorial
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences



Phase 6 — CloudWatch Logs Audit Trail

Every step of the remediation cycle is logged in CloudWatch. This is what you hand to a compliance auditor — a timestamped record of what drifted, when, and exactly what was done about it.





What the logs look like

```
[INFO]    Checking SG inbound rules: sg-0d59147aaa97318d9
[WARNING] DRIFT DETECTED: Unauthorized rule — port=80, cidrs=['0.0.0.0/0'] on sg-
0d59147aaa97318d9
[INFO]    REMEDIATED: Removed 1 unauthorized inbound rules from sg-
0d59147aaa97318d9
```

Four log lines. Complete story. Timestamp on every entry. This is the audit trail that makes compliance teams happy and keeps security teams informed.

Drift Types This System Handles

Drift Type	Example	How It Is Detected	How It Is Fixed
Unauthorized inbound rule	Port 80 added via console	Config rule flags NON_COMPLIANT	Lambda revokes the rule
Wrong source CIDR	Port 22 changed to 0.0.0.0/0	Config rule flags NON_COMPLIANT	Lambda revokes and the approved rule stays
Unauthorized EC2 tag	Owner tag added manually	Config change event fires	Lambda deletes the unauthorized tag
Changed tag value	Environment changed from lab to production	Config change event fires	Lambda restores the original value

---

🔧 Key Design Decisions

Why AWS Config instead of a scheduled Terraform plan

I chose AWS Config because it can detect configuration changes as they happen instead of waiting for the next scheduled Terraform plan.

For example, if someone changes a Security Group at midnight, a scheduled `terraform plan` might not detect it until its next run. With AWS Config, the change can be evaluated shortly after it happens.

For security-sensitive resources such as Security Groups, detecting the change quickly gives us a better chance of correcting it before it becomes a bigger issue.

Why `is_rule_approved()` checks both port and CIDR

Initially, the function only checked the port number. That created a serious problem.

For example, if port 22 was allowed in the approved configuration, a rule allowing `22` from `0.0.0.0/0` could also be treated as approved, even though it exposes SSH to the entire internet.

The check therefore needs to validate the complete rule — including the port, protocol, and source CIDR. This makes sure that a rule is considered valid only when it actually matches the expected configuration.

Why EventBridge sits between Config and Lambda

AWS Config doesn't need to call the Lambda function directly. Instead, Config generates an event when a resource becomes `NON_COMPLIANT`, and EventBridge receives and routes that event to Lambda.

This also keeps the components separate. If I later want the same event to trigger an SNS notification, store an audit record in DynamoDB, or send a Slack notification, I can add another EventBridge target without having to change the Config rule or the Lambda function.

This makes it easier to extend the system as the project grows.

Why the Lambda timeout is set to 30 seconds

The default Lambda timeout is 3 seconds. For this project, that can be a little tight because the function needs to make AWS API calls such as `describe_security_groups` and `revoke_security_group_ingress`.

If multiple unauthorized rules need to be removed, or if an AWS API call takes longer than expected, a 3-second timeout could cause the function to stop before the remediation is completed.

I set the timeout to 30 seconds to give the function enough time to complete the remediation without making the timeout unnecessarily long.

⏱️ How Long Does Remediation Take

Trigger Method	Time to Remediate
Lambda Test button — manual	5–10 seconds
Config → Re-evaluate → EventBridge → Lambda	2–5 minutes
Config on configuration changes (automatic)	5–15 minutes
Config periodic evaluation (1 hour cycle)	Up to 1 hour

For demos, click **Config** → **Actions** → **Re-evaluate** to force immediate evaluation without waiting for the periodic cycle. For end-to-end validation, the automatic flow is more impressive — add the rule, wait a few minutes, and watch it disappear on its own.

💡 What I Learned

Configuration drift isn't only a technical problem — it's also a governance problem. The port 80 rule itself wasn't the biggest issue. The real problem was that someone bypassed the normal code review process, so there was no second pair of eyes on the change, nothing in Git, and no clear way to know it happened until someone noticed the difference or an auditor found it.

One thing that surprised me was how different Terraform state and AWS Config are when it comes to tracking changes. Terraform knows the state of the infrastructure it manages, but it doesn't keep a complete history of every change made between applies. AWS Config does provide that historical view, including timestamps for configuration changes. You can use it to look back at what a Security Group looked like at a particular point in time and see what changed. That's especially useful when investigating drift or dealing with audit requirements.

I also got the remediation logic wrong on my first pass. My initial `is_rule_approved()` function only checked port numbers. That meant SSH on port 22 could have been treated as approved even if it was open to `0.0.0.0/0`, simply because the port matched. I had to change the logic to validate the port, protocol, and source CIDR together. Otherwise, the system could have marked an insecure rule as valid and missed the actual drift.

Routing everything through EventBridge instead of connecting Config directly to Lambda felt like extra complexity at first. But it gives the system more flexibility. I can add another target later — such as an SNS notification, Slack integration, or another Lambda function — without changing the existing Config or remediation logic.

I also learned that Lambda's default 3-second timeout can be too short for a remediation workflow. The function timed out once while making the required AWS API calls, leaving the Security Group only partially fixed. Increasing the timeout gave the function enough time to complete the remediation reliably.

These were small details individually, but they made a big difference in how I approached the final design of the system.

📌 Project Summary

This project started with a simple observation: telling engineers not to touch the AWS Console directly works fine — until there's a production incident and someone needs to make a quick change to fix it.

That's where drift creeps in.

Here's what the system does when that happens: someone manually opens a port in a Security Group, AWS Config detects the change, EventBridge routes the event to Lambda, and Lambda checks whether the change matches the approved configuration. If it doesn't, the system automatically reverts it. CloudWatch logs the entire process — what changed, when it changed, and what action was taken.

I'm not trying to prevent emergency console changes altogether. Sometimes they're genuinely necessary, especially during a production incident. The goal is to prevent those temporary changes from quietly becoming permanent without anyone noticing.

That's the gap this project is designed to close.

🧹 Cleanup Order

```
1. Disable AWS Config recording          → Settings → Edit → turn off
2. Delete EventBridge rule               → config-noncompliant-trigger
3. Delete Lambda function                → drift-auto-remediate
4. Delete IAM Role                       → drift-remediation-lambda-role
5. Delete AWS Config rule                → terraform-drift-sg-rule
6. Terraform destroy                     → terraform destroy -auto-approve
7. Empty Config delivery S3 bucket       → then delete the bucket
8. Delete CloudWatch Log Group           → /aws/lambda/drift-auto-remediate
```

> ⚠️ Run `terraform destroy` before manually deleting resources. Deleting resources from the console first puts Terraform state out of sync and makes the destroy fail.

👷 Author

Premkumar Keshav Patil Cloud & DevOps Enthusiast | Learning AWS, Terraform, and Cloud Automation

LinkedIn Connect | Medium Follow | GitHub Follow

📌 Tags

AWS Terraform Lambda AWS Config EventBridge CloudWatch IAM Python boto3 Infrastructure as Code Drift Detection Auto Remediation Cloud Automation DevOps Compliance Security EC2 Security Groups